United States Patent	12417210
Kind Code	B2
Date of Patent	September 16, 2025
Inventor(s)	Liang; Zhimin et al.

Log data extraction from data chunks of an isolated execution environment

Abstract

Systems and methods are disclosed for processing data associated with isolated execution environments. A chunk of data associated with an isolated execution environment can include log data and non-log data. At least a portion of the log data can include log data generated by the isolated execution environment. The system can parse the chunk of data to identify the log data and the non-log data and extract at least a portion of the log data from the chunk of data. The extracted data can be further processed to generate one or more events.

Inventors: **Liang; Zhimin (West Vancouver, CA), Modestino; Matthew (Toronto, CA), Baldwin; David Christopher (Dublin, CA), Chéné; Marc Andre (Seattle, WA), Wastell; Blaine (Woodinville, WA)**

Applicant: **Splunk Inc.** (San Francisco, CA)

Family ID: **1000008820234**

Assignee: **SPLUNK Inc. (San Francisco, CA)**

Appl. No.: **18/504491**

Filed: **November 08, 2023**

Prior Publication Data

Document Identifier	Publication Date
US 20240152488 A1	May. 09, 2024

Related U.S. Application Data

continuation parent-doc US 17646372 20211229 US 11829330 child-doc US 18504491
continuation parent-doc US 15979933 20180515 US 11238012 20220201 child-doc US 17646372

Publication Classification

Int. Cl.: **G06F17/00** (20190101); **G06F7/00** (20060101); **G06F8/41** (20180101); **G06F9/455** (20180101); **G06F16/17** (20190101); **G06F16/901** (20190101); **G06F16/9038** (20190101); **G06F16/907** (20190101)

U.S. Cl.:

CPC **G06F16/1734** (20190101); **G06F8/427** (20130101); **G06F9/45533** (20130101); **G06F16/901** (20190101); **G06F16/9038** (20190101); **G06F16/907** (20190101); G06F2009/45587 (20130101); G06F2009/45591 (20130101)

Field of Classification Search

CPC: G06F (16/1734); G06F (16/907); G06F (16/901); G06F (16/9038); G06F (8/427); G06F (9/45533); G06F (2009/45587); G06F (2009/45591)

USPC: 707/672

References Cited

U.S. PATENT DOCUMENTS

Patent No.	Issued Date	Patentee Name	U.S. Cl.	CPC
7937344	12/2010	Baum et al.	N/A	N/A
8060466	12/2010	Round et al.	N/A	N/A
8112425	12/2011	Baum et al.	N/A	N/A
8751529	12/2013	Zhang et al.	N/A	N/A
8788525	12/2013	Neels et al.	N/A	N/A
8983912	12/2014	Beedgen et al.	N/A	N/A
9135560	12/2014	Saurabh et al.	N/A	N/A
9215240	12/2014	Merza et al.	N/A	N/A
9262519	12/2015	Saurabh	N/A	N/A
9286413	12/2015	Coates et al.	N/A	N/A
9342571	12/2015	Kurtic et al.	N/A	N/A
9853986	12/2016	Dash et al.	N/A	N/A
10127258	12/2017	Lamas et al.	N/A	N/A
10171312	12/2018	Kannan et al.	N/A	N/A
10223145	12/2018	Neogy et al.	N/A	N/A
10242062	12/2018	Turner	N/A	N/A
10346775	12/2018	Xu et al.	N/A	N/A
10419469	12/2018	Singh et al.	N/A	N/A
10445311	12/2018	Saurabh et al.	N/A	N/A
10474656	12/2018	Bronnikov et al.	N/A	N/A
10547521	12/2019	Roy et al.	N/A	N/A
10762746	12/2019	Liang et al.	N/A	N/A
10929415	12/2020	Shcherbakov et al.	N/A	N/A
11238012	12/2021	Liang	N/A	G06F 11/3006
11537627	12/2021	Baskaran et al.	N/A	N/A
11567960	12/2022	Shcherbkov et al.	N/A	N/A

11829330	12/2022	Liang	N/A	G06F 16/9038
11886455	12/2023	Baskaran et al.	N/A	N/A
2005/0114707	12/2004	DeStefano et al.	N/A	N/A
2007/0174429	12/2006	Mazzaferri	709/218	H04L 63/102
2007/0283194	12/2006	Villella et al.	N/A	N/A
2008/0162592	12/2007	Huang et al.	N/A	N/A
2010/0185961	12/2009	Fisher et al.	N/A	N/A
2011/0035248	12/2010	Juillard	N/A	N/A
2012/0246303	12/2011	Petersen et al.	N/A	N/A
2012/0259825	12/2011	Tashiro et al.	N/A	N/A
2012/0310899	12/2011	Wasserman et al.	N/A	N/A
2013/0042123	12/2012	Smith et al.	N/A	N/A
2013/0117676	12/2012	De Pauw et al.	N/A	N/A
2013/0227349	12/2012	Nodir et al.	N/A	N/A
2013/0332424	12/2012	Nos et al.	N/A	N/A
2013/0332588	12/2012	Maytal et al.	N/A	N/A
2014/0040182	12/2013	Gilder et al.	N/A	N/A
2014/0047099	12/2013	Flores et al.	N/A	N/A
2014/0229607	12/2013	Jung et al.	N/A	N/A
2014/0278807	12/2013	Bohacek	N/A	N/A
2014/0278808	12/2013	Iyoob et al.	N/A	N/A
2014/0279201	12/2013	Iyoob et al.	N/A	N/A
2014/0324647	12/2013	Iyoob et al.	N/A	N/A
2014/0330832	12/2013	Viau	N/A	N/A
2015/0039651	12/2014	Kinsely et al.	N/A	N/A
2015/0039757	12/2014	Petersen et al.	N/A	N/A
2015/0149879	12/2014	Miller et al.	N/A	N/A
2015/0180891	12/2014	Seward et al.	N/A	N/A
2015/0271109	12/2014	Bullotta et al.	N/A	N/A
2015/0309710	12/2014	Ashoori et al.	N/A	N/A
2015/0341240	12/2014	Iyoob et al.	N/A	N/A
2015/0363851	12/2014	Stella et al.	N/A	N/A
2015/0369664	12/2014	Garsha et al.	N/A	N/A
2016/0019636	12/2015	Adapalli et al.	N/A	N/A
2016/0036903	12/2015	Pal et al.	N/A	N/A
2016/0043892	12/2015	Hason et al.	N/A	N/A
2016/0092475	12/2015	Stojanovic et al.	N/A	N/A
2016/0092558	12/2015	Ago et al.	N/A	N/A
2016/0094477	12/2015	Bai et al.	N/A	N/A
2016/0180557	12/2015	Yousaf et al.	N/A	N/A
2016/0198003	12/2015	Luft	N/A	N/A
2016/0246844	12/2015	Turner	N/A	N/A
2016/0271500	12/2015	Nath et al.	N/A	N/A
2016/0282858	12/2015	Michalscheck et al.	N/A	N/A
2016/0292166	12/2015	Russell	N/A	N/A
2016/0359955	12/2015	Gill et al.	N/A	N/A
2017/0046445	12/2016	Cormier et al.	N/A	N/A
2017/0061339	12/2016	Littlejohn et al.	N/A	N/A
2017/0085446	12/2016	Zhong et al.	N/A	N/A
2017/0085447	12/2016	Chen et al.	N/A	N/A

2017/0093645	12/2016	Zhong et al.	N/A	N/A
2017/0116321	12/2016	Jain	N/A	G06F 16/27
2017/0228460	12/2016	Amel et al.	N/A	N/A
2017/0295181	12/2016	Parimi et al.	N/A	N/A
2017/0364538	12/2016	Jacob et al.	N/A	N/A
2017/0364540	12/2016	Sigler	N/A	N/A
2018/0012166	12/2017	Devadas et al.	N/A	N/A
2018/0027006	12/2017	Zimmermann et al.	N/A	N/A
2018/0115463	12/2017	Sinha et al.	N/A	N/A
2018/0165142	12/2017	Harutyunyan et al.	N/A	N/A
2018/0225345	12/2017	Gilder et al.	N/A	N/A
2018/0246797	12/2017	Modi et al.	N/A	N/A
2018/0321927	12/2017	Borthakur et al.	N/A	N/A
2018/0336027	12/2017	Narayanan et al.	N/A	N/A
2018/0367412	12/2017	Sethi et al.	N/A	N/A
2019/0018717	12/2018	Feijoo et al.	N/A	N/A
2019/0018844	12/2018	Bhagwat et al.	N/A	N/A
2019/0052542	12/2018	Passante et al.	N/A	N/A
2019/0098106	12/2018	Mungel et al.	N/A	N/A
2019/0155953	12/2018	Brown et al.	N/A	N/A
2019/0190773	12/2018	Shi et al.	N/A	N/A
2019/0306236	12/2018	Wiener et al.	N/A	N/A
2019/0310977	12/2018	Pal et al.	N/A	N/A
2019/0312939	12/2018	Noble	N/A	N/A
2019/0324962	12/2018	Turner	N/A	N/A
2019/0342372	12/2018	Lee et al.	N/A	N/A
2019/0379590	12/2018	Rimar et al.	N/A	N/A
2019/0386891	12/2018	Chitalia et al.	N/A	N/A
2020/0026624	12/2019	Parthasarathy et al.	N/A	N/A
2020/0034484	12/2019	Arora et al.	N/A	N/A
2020/0099610	12/2019	Heron	N/A	N/A
2020/0134359	12/2019	Kim et al.	N/A	N/A
2021/0105597	12/2020	Wright et al.	N/A	N/A
2021/0224259	12/2020	Shcherbakov	N/A	N/A
2022/0300464	12/2021	Liang et al.	N/A	N/A
2023/0169084	12/2022	Shcherbakov	N/A	N/A

FOREIGN PATENT DOCUMENTS

Patent No.	Application Date	Country	CPC
101810762	12/2016	KR	N/A

OTHER PUBLICATIONS

U.S. Appl. No. 15/979,933, filed May 15, 2018, Liang et al. cited by applicant
U.S. Appl. No. 15/980,008, filed May 15, 2018, Modestino et al. cited by applicant
U.S. Appl. No. 16/262,746, filed Jan. 30, 2019, Liang et al. cited by applicant
U.S. Appl. No. 16/148,918, filed Oct. 1, 2018, Shcherbakov et al. cited by applicant
U.S. Appl. No. 18/146,256, filed Dec. 23, 2022, Baskaran et al. cited by applicant
U.S. Appl. No. 18/160,972, filed Jan. 27, 2023, Shcherbakov et al. cited by applicant
Add Docker metadata, Filebeat Reference 6.0,
https://www.elastic.co/guide/en/beats/filebeat/6.0/add-docker-metadata.html, software version

(6.0.0) realized 2017. cited by applicant
Beach, et.: Pro PowerShell for Amazon Web Services; 2nd edition (Year: 2019). 1 of 3. cited by applicant
Beach, et.: Pro PowerShell for Amazon Web Services; 2nd edition (Year: 2019). 2 of 3. cited by applicant
Beach, et.: Pro PowerShell for Amazon Web Services; 2nd edition (Year: 2019). 3 of 3. cited by applicant
Beats Version 5.0.0. Release Notes [relating to version 5.0, allegedly released Oct. 2016] [online], [retrieved on Jan. 31, 2020]. Retrieved from the Internet :< URL: https://www.elastic.co/guide/en/beats/libbeat/current/release-notes-5.0.0.html>. cited by applicant
Bitincka, Ledion et al., "Optimizing Data Analysis with a Semi-structured Time Series Database," self-published, first presented at "Workshop on Managing Systems via Log Analysis and Machine Learning Techniques (SLAML)", Vancouver, British Columbia, Oct. 3, 2010. cited by applicant
Carraso, David, "Exploring Splunk," published by CITO Research, New York, NY, Apr. 2012. cited by applicant
Filebeat Prospectors Configuration. Filebeat Reference [relating to version 5.0, allegedly released Oct. 2016] [version 5.0 allegedly released Oct. 2016] [online], [retrieved on Jan. 31, 2020]. Retrieved from the Internet: < URL:https://www.elastic.co/guide/en/beats/filebeat/5.0/configuration-filebeat-options.html>. cited by applicant
Perez-Aradros, C. Enriching Logs with Docker Metadata Using Filebeat. Elastic Blog [online], Jul. 2017 [retrieved on Jan. 31, 2020]. Retrieved from the Internet: < URL: https://www.elastic.co/blog/enrich-docker-logs-with-filebeat>. cited by applicant
Perez-Aradros, C. Shipping Kubernetes Logs to Elasticsearch with Filebeat. Elastic Blog [online], Nov. 2017 [retrieved on Jan. 31, 2020]. Retrieved from the Internet: < URL: https://www.elastic.co/blog/shipping-kubernetes-logs-to-elasticsearch-with-filebeat>. cited by applicant
Set up Prospectors. Filebeat Reference [relating to version 6.2, allegedly released Feb. 2018] [online], [retrieved on Jan. 31, 2020]. Retrieved from the Internet: < URL: https://www.elastic.co/guide/en/beats/filebeat/6.2/configuration-filebeat-options.html>. cited by applicant
SLAML 10 Reports, Workshop On Managing Systems via Log Analysis and Machine Learning Techniques, ;login: Feb. 2011 Conference Reports. cited by applicant
Splunk Enterprise 8.0.0 Overview, available online, retrieved May 20, 2020 from docs.splunk.com. cited by applicant
Splunk Cloud 8.0.2004 User Manual, available online, retrieved May 20, 2020 from docs.splunk.com. cited by applicant
Splunk Quick Reference Guide, updated 2019, available online at https://www.splunk.com/pdfs/solution-guides/splunk-quick-reference-guide.pdf, retrieved May 20, 2020. cited by applicant
TSG. 'Second proposal for JSON support'. In Elastic/Beats Pull Requests [online], Mar. 2016 [retrieved on Jan. 31, 2020]. Retrieved from the internet: <URL: https://github.com/elastic/beats/pull/1143>. cited by applicant
Vaid, Workshop on Managing Systems via log Analysis and Machine Learning Techniques (SLAML '10), ; login: vol. 36, No. 1, Oct. 3, 2010, Vancouver, BC, Canada. cited by applicant
Office Action in U.S. Appl. No. 15/979,933, dated Feb. 20, 2020, 17 pages. cited by applicant
Final Office Action in U.S. Appl. No. 15/979,933, dated Aug. 19, 2020 in 23 pages. cited by applicant
Office Action in U.S. Appl. No. 15/979,933, dated May 18, 2021 in 22 pages. cited by applicant
Notice of Allowance in U.S. Appl. No. 15/979,933, dated Sep. 17, 2021 in 12 pages. cited by

applicant
Office Action in U.S. Appl. No. 16/262,746, dated Mar. 29, 2019, 14 pages. cited by applicant
Final Office Action in U.S. Appl. No. 16/262,746, dated Oct. 16, 2019, 15 pages. cited by applicant
Notice of Allowance in U.S. Appl. No. 16/262,746, dated Jan. 30, 2020, 17 pages. cited by applicant
Notice of Allowance in U.S. Appl. No. 16/262,746, dated Mar. 23, 2020, 9 pages. cited by applicant
Office Action in U.S. Appl. No. 17/646,372, dated Mar. 29, 2023, in 16 pages. cited by applicant
Notice of Allowance in U.S. Appl. No. 17/646,372, dated Jul. 19, 2023, in 12 pages. cited by applicant
Office Action in U.S. Appl. No. 15/980,008, dated May 7, 2020, 11 pages. cited by applicant
Final Office Action in U.S. Appl. No. 15/980,008, dated Oct. 20, 2020, in 19 pages. cited by applicant
Notice of Allowance in U.S. Appl. No. 15/980,008, dated May 13, 2021, in 13 pages. cited by applicant
Office Action in U.S. Appl. No. 16/148,918 dated May 18, 2020, in 12 pages. cited by applicant
Notice of Allowance in U.S. Appl. No. 16/148,918 dated Oct. 6, 2020, in 10 pages. cited by applicant
Office Action in U.S. Appl. No. 17/143,063 dated Feb. 18, 2022 in 15 pages. cited by applicant
Final Office Action in U.S. Appl. No. 17/143,063 dated Jul. 18, 2022 in 7 pages. cited by applicant
Notice of Allowance in U.S. Appl. No. 17/143,063 dated Sep. 28, 2022 in 9 pages. cited by applicant
Office Action in U.S. Appl. No. 16/147,181 dated Dec. 21, 2020 in 31 pages. cited by applicant
Final Office Action in U.S. App. No. 16/147,181 dated Jun. 25, 2021 in 28 pages. cited by applicant
Office Action in U.S. App. No. 16/147,181 dated Nov. 12, 2021 in 27 pages. cited by applicant
Notice of Allowance in U.S. App. No. 16/147,181 dated Aug. 19, 2022 in 10 pages. cited by applicant
Office Action in U.S. Appl. No. 18/146,256 dated May 12, 2023 in 22 pages. cited by applicant
Office Action in U.S. Appl. No. 17/305,550 dated May 26, 2023 in 41 pages. cited by applicant

---

Primary Examiner: Mamillapalli; Pavan

Attorney, Agent or Firm: Kilpatrick Townsend & Stockton LLP

Background/Summary

RELATED APPLICATIONS (1) The present application is a continuation of U.S. application Ser. No. 17/646,372, filed Dec. 29, 2021, entitled “Log Data Extraction from Data Chunks of an Isolated Execution Environment”, which is a continuation of U.S. application Ser. No. 15/979,933, filed May 15, 2023, entitled “Log Data Extraction from Data Chunks of an Isolated Execution Environment”, now U.S. Pat. No. 11,238,012. The present application incorporates each of the foregoing disclosures herein by reference.

BACKGROUND
(1) Information technology (IT) environments can include diverse types of data systems that store large amounts of diverse data types generated by numerous devices. For example, a big data ecosystem may include databases such as MySQL and Oracle databases, cloud computing services such as Amazon web services (AWS), and other data systems that store passively or actively

generated data, including machine-generated data ("machine data"). The machine data can include performance data, diagnostic data, or any other data that can be analyzed to diagnose equipment performance problems, monitor user interactions, and to derive other insights.
(2) The large amount and diversity of data systems containing large amounts of structured, semi-structured, and unstructured data relevant to any search query can be massive, and continues to grow rapidly. This technological evolution can give rise to various challenges in relation to managing, understanding and effectively utilizing the data. To reduce the potentially vast amount of data that may be generated, some data systems pre-process data based on anticipated data analysis needs. In particular, specified data items may be extracted from the generated data and stored in a data system to facilitate efficient retrieval and analysis of those data items at a later time. At least some of the remainder of the generated data is typically discarded during pre-processing.
(3) However, storing massive quantities of minimally processed or unprocessed data (collectively and individually referred to as "raw data") for later retrieval and analysis is becoming increasingly more feasible as storage capacity becomes more inexpensive and plentiful. In general, storing raw data and performing analysis on that data later can provide greater flexibility because it enables an analyst to analyze all of the generated data instead of only a fraction of it.
(4) Although the availability of vastly greater amounts of diverse data on diverse data systems provides opportunities to derive new insights, it also gives rise to technical challenges to search and analyze the data. Tools exist that allow an analyst to search data systems separately and collect results over a network for the analyst to derive insights in a piecemeal manner. However, UI tools that allow analysts to quickly search and analyze large set of raw machine data to visually identify data subsets of interest, particularly via straightforward and easy-to-understand sets of tools and search functionality do not exist.

Description

BRIEF DESCRIPTION OF THE DRAWINGS
(1) The present disclosure is illustrated by way of example, and not limitation, in the figures of the accompanying drawings, in which like reference numerals indicate similar elements and in which:
(2) FIG. **1** is a block diagram of an example networked computer environment, in accordance with example embodiments.
(3) FIG. **2** is a block diagram of an example data intake and query system, in accordance with example embodiments.
(4) FIG. **3** is a block diagram of an example cloud-based data intake and query system, in accordance with example embodiments.
(5) FIG. **4** is a block diagram of an example data intake and query system that performs searches across external data systems, in accordance with example embodiments.
(6) FIG. **5**A is a flowchart of an example method that illustrates how indexers process, index, and store data received from forwarders, in accordance with example embodiments.
(7) FIG. **5**B is a block diagram of a data structure in which time-stamped event data can be stored in a data store, in accordance with example embodiments.
(8) FIG. **5**C provides a visual representation of the manner in which a pipelined search language or query operates, in accordance with example embodiments.
(9) FIG. **6**A is a flow diagram of an example method that illustrates how a search head and indexers perform a search query, in accordance with example embodiments.
(10) FIG. **6**B provides a visual representation of an example manner in which a pipelined command language or query operates, in accordance with example embodiments.
(11) FIG. **7**A is a diagram of an example scenario where a common customer identifier is found among log data received from three disparate data sources, in accordance with example

embodiments.
(12) FIG. **7**B illustrates an example of processing keyword searches and field searches, in accordance with disclosed embodiments.
(13) FIG. **7**C illustrates an example of creating and using an inverted index, in accordance with example embodiments.
(14) FIG. **7**D depicts a flowchart of example use of an inverted index in a pipelined search query, in accordance with example embodiments.
(15) FIG. **8**A is an interface diagram of an example user interface for a search screen, in accordance with example embodiments.
(16) FIG. **8**B is an interface diagram of an example user interface for a data summary dialog that enables a user to select various data sources, in accordance with example embodiments.
(17) FIGS. **9**, **10**, **11**A, **11**B, **11**C, **11**D, **12**, **13**, **14**, and **15** are interface diagrams of example report generation user interfaces, in accordance with example embodiments.
(18) FIG. **16** is an example search query received from a client and executed by search peers, in accordance with example embodiments.
(19) FIG. **17**A is an interface diagram of an example user interface of a key indicators view, in accordance with example embodiments.
(20) FIG. **17**B is an interface diagram of an example user interface of an incident review dashboard, in accordance with example embodiments.
(21) FIG. **17**C is a tree diagram of an example a proactive monitoring tree, in accordance with example embodiments.
(22) FIG. **17**D is an interface diagram of an example a user interface displaying both log data and performance data, in accordance with example embodiments.
(23) FIG. **18** is a block diagram of an embodiment of a data processing environment that includes a data adapter.
(24) FIG. **19** is a block diagram of an embodiment of a data store that stores log files associated with a data source.
(25) FIGS. **20**A and **20**B are interface diagrams of an example user interface displaying event data associated with events generated by a data intake and query system.
(26) FIG. **21** is a flow diagram illustrative of an embodiment of a routine implemented by the data intake and query system **108** for parsing chunks of data.
(27) FIG. **22** is a flow diagram illustrative of an embodiment of a routine implemented by the data intake and query system for identifying and associating metadata with one or more events.

DETAILED DESCRIPTION

(28) Embodiments are described herein according to the following outline:

1.0. General Overview

(30) Modern data centers and other computing environments can comprise anywhere from a few host computer systems to thousands of systems configured to process data, service requests from remote clients, and perform numerous other computational tasks. During operation, various components within these computing environments often generate significant volumes of machine

data. Machine data is any data produced by a machine or component in an information technology (IT) environment and that reflects activity in the IT environment. For example, machine data can be raw machine data that is generated by various components in IT environments, such as servers, sensors, routers, mobile devices, Internet of Things (IoT) devices, etc. Machine data can include system logs, network packet data, sensor data, application program data, error logs, stack traces, system performance data, etc. In general, machine data can also include performance data, diagnostic information, and many other types of data that can be analyzed to diagnose performance problems, monitor user interactions, and to derive other insights.

(31) A number of tools are available to analyze machine data. In order to reduce the size of the potentially vast amount of machine data that may be generated, many of these tools typically pre-process the data based on anticipated data-analysis needs. For example, pre-specified data items may be extracted from the machine data and stored in a database to facilitate efficient retrieval and analysis of those data items at search time. However, the rest of the machine data typically is not saved and is discarded during pre-processing. As storage capacity becomes progressively cheaper and more plentiful, there are fewer incentives to discard these portions of machine data and many reasons to retain more of the data.

(32) This plentiful storage capacity is presently making it feasible to store massive quantities of minimally processed machine data for later retrieval and analysis. In general, storing minimally processed machine data and performing analysis operations at search time can provide greater flexibility because it enables an analyst to search all of the machine data, instead of searching only a pre-specified set of data items. This may enable an analyst to investigate different aspects of the machine data that previously were unavailable for analysis.

(33) However, analyzing and searching massive quantities of machine data presents a number of challenges. For example, a data center, servers, or network appliances may generate many different types and formats of machine data (e.g., system logs, network packet data (e.g., wire data, etc.), sensor data, application program data, error logs, stack traces, system performance data, operating system data, virtualization data, etc.) from thousands of different components, which can collectively be very time-consuming to analyze. In another example, mobile devices may generate large amounts of information relating to data accesses, application performance, operating system performance, network performance, etc. There can be millions of mobile devices that report these types of information.

(34) These challenges can be addressed by using an event-based data intake and query system, such as the SPLUNK® ENTERPRISE system developed by Splunk Inc. of San Francisco, California. The SPLUNK® ENTERPRISE system is the leading platform for providing real-time operational intelligence that enables organizations to collect, index, and search machine data from various websites, applications, servers, networks, and mobile devices that power their businesses. The data intake and query system is particularly useful for analyzing data which is commonly found in system log files, network data, and other data input sources. Although many of the techniques described herein are explained with reference to a data intake and query system similar to the SPLUNK® ENTERPRISE system, these techniques are also applicable to other types of data systems.

(35) In the data intake and query system, machine data are collected and stored as “events”. An event comprises a portion of machine data and is associated with a specific point in time. The portion of machine data may reflect activity in an IT environment and may be produced by a component of that IT environment, where the events may be searched to provide insight into the IT environment, thereby improving the performance of components in the IT environment. Events may be derived from “time series data,” where the time series data comprises a sequence of data points (e.g., performance measurements from a computer system, etc.) that are associated with successive points in time. In general, each event has a portion of machine data that is associated with a timestamp that is derived from the portion of machine data in the event. A timestamp of an

event may be determined through interpolation between temporally proximate events having known timestamps or may be determined based on other configurable rules for associating timestamps with events.
(36) In some instances, machine data can have a predefined format, where data items with specific data formats are stored at predefined locations in the data. For example, the machine data may include data associated with fields in a database table. In other instances, machine data may not have a predefined format (e.g., may not be at fixed, predefined locations), but may have repeatable (e.g., non-random) patterns. This means that some machine data can comprise various data items of different data types that may be stored at different locations within the data. For example, when the data source is an operating system log, an event can include one or more lines from the operating system log containing machine data that includes different types of performance and diagnostic information associated with a specific point in time (e.g., a timestamp).
(37) Examples of components which may generate machine data from which events can be derived include, but are not limited to, web servers, application servers, databases, firewalls, routers, operating systems, and software applications that execute on computer systems, mobile devices, sensors, Internet of Things (IoT) devices, etc. The machine data generated by such data sources can include, for example and without limitation, server log files, activity log files, configuration files, messages, network packet data, performance measurements, sensor measurements, etc.
(38) The data intake and query system uses a flexible schema to specify how to extract information from events. A flexible schema may be developed and redefined as needed. Note that a flexible schema may be applied to events “on the fly,” when it is needed (e.g., at search time, index time, ingestion time, etc.). When the schema is not applied to events until search time, the schema may be referred to as a “late-binding schema.”
(39) During operation, the data intake and query system receives machine data from any type and number of sources (e.g., one or more system logs, streams of network packet data, sensor data, application program data, error logs, stack traces, system performance data, etc.). The system parses the machine data to produce events each having a portion of machine data associated with a timestamp. The system stores the events in a data store. The system enables users to run queries against the stored events to, for example, retrieve events that meet criteria specified in a query, such as criteria indicating certain keywords or having specific values in defined fields. As used herein, the term “field” refers to a location in the machine data of an event containing one or more values for a specific data item. A field may be referenced by a field name associated with the field. As will be described in more detail herein, a field is defined by an extraction rule (e.g., a regular expression) that derives one or more values or a sub-portion of text from the portion of machine data in each event to produce a value for the field for that event. The set of values produced are semantically-related (such as IP address), even though the machine data in each event may be in different formats (e.g., semantically-related values may be in different positions in the events derived from different sources).
(40) As described above, the system stores the events in a data store. The events stored in the data store are field-searchable, where field-searchable herein refers to the ability to search the machine data (e.g., the raw machine data) of an event based on a field specified in search criteria. For example, a search having criteria that specifies a field name “UserID” may cause the system to field-search the machine data of events to identify events that have the field name “UserID.” In another example, a search having criteria that specifies a field name “UserID” with a corresponding field value “12345” may cause the system to field-search the machine data of events to identify events having that field-value pair (e.g., field name “UserID” with a corresponding field value of “12345”). Events are field-searchable using one or more configuration files associated with the events. Each configuration file includes one or more field names, where each field name is associated with a corresponding extraction rule and a set of events to which that extraction rule applies. The set of events to which an extraction rule applies may be identified by metadata

associated with the set of events. For example, an extraction rule may apply to a set of events that are each associated with a particular host, source, or source type. When events are to be searched based on a particular field name specified in a search, the system uses one or more configuration files to determine whether there is an extraction rule for that particular field name that applies to each event that falls within the criteria of the search. If so, the event is considered as part of the search results (and additional processing may be performed on that event based on criteria specified in the search). If not, the next event is similarly analyzed, and so on.

(41) As noted above, the data intake and query system utilizes a late-binding schema while performing queries on events. One aspect of a late-binding schema is applying extraction rules to events to extract values for specific fields during search time. More specifically, the extraction rule for a field can include one or more instructions that specify how to extract a value for the field from an event. An extraction rule can generally include any type of instruction for extracting values from events. In some cases, an extraction rule comprises a regular expression, where a sequence of characters form a search pattern. An extraction rule comprising a regular expression is referred to herein as a regex rule. The system applies a regex rule to an event to extract values for a field associated with the regex rule, where the values are extracted by searching the event for the sequence of characters defined in the regex rule.

(42) In the data intake and query system, a field extractor may be configured to automatically generate extraction rules for certain fields in the events when the events are being created, indexed, or stored, or possibly at a later time. Alternatively, a user may manually define extraction rules for fields using a variety of techniques. In contrast to a conventional schema for a database system, a late-binding schema is not defined at data ingestion time. Instead, the late-binding schema can be developed on an ongoing basis until the time a query is actually executed. This means that extraction rules for the fields specified in a query may be provided in the query itself, or may be located during execution of the query. Hence, as a user learns more about the data in the events, the user can continue to refine the late-binding schema by adding new fields, deleting fields, or modifying the field extraction rules for use the next time the schema is used by the system. Because the data intake and query system maintains the underlying machine data and uses a late-binding schema for searching the machine data, it enables a user to continue investigating and learn valuable insights about the machine data.

(43) In some embodiments, a common field name may be used to reference two or more fields containing equivalent and/or similar data items, even though the fields may be associated with different types of events that possibly have different data formats and different extraction rules. By enabling a common field name to be used to identify equivalent and/or similar fields from different types of events generated by disparate data sources, the system facilitates use of a “common information model” (CIM) across the disparate data sources (further discussed with respect to FIG. **7**A).

2.0. Operating Environment

(44) FIG. **1** is a block diagram of an example networked computer environment **100**, in accordance with example embodiments. Those skilled in the art would understand that FIG. **1** represents one example of a networked computer system and other embodiments may use different arrangements.

(45) The networked computer system **100** comprises one or more computing devices. These one or more computing devices comprise any combination of hardware and software configured to implement the various logical components described herein. For example, the one or more computing devices may include one or more memories that store instructions for implementing the various components described herein, one or more hardware processors configured to execute the instructions stored in the one or more memories, and various data repositories in the one or more memories for storing data structures utilized and manipulated by the various components.

(46) In some embodiments, one or more client devices **102** are coupled to one or more host devices **106** and a data intake and query system **108** via one or more networks **104**. Networks **104** broadly

represent one or more LANs, WANs, cellular networks (e.g., LTE, HSPA, 3G, and other cellular technologies), and/or networks using any of wired, wireless, terrestrial microwave, or satellite links, and may include the public Internet.

2.1. Host Devices

(47) In the illustrated embodiment, a system **100** includes one or more host devices **106**. Host devices **106** may broadly include any number of computers, virtual machine instances, and/or data centers that are configured to host or execute one or more instances of host applications **114**. In general, a host device **106** may be involved, directly or indirectly, in processing requests received from client devices **102**. Each host device **106** may comprise, for example, one or more of a network device, a web server, an application server, a database server, etc. A collection of host devices **106** may be configured to implement a network-based service. For example, a provider of a network-based service may configure one or more host devices **106** and host applications **114** (e.g., one or more web servers, application servers, database servers, etc.) to collectively implement the network-based application.

(48) In general, client devices **102** communicate with one or more host applications **114** to exchange information. The communication between a client device **102** and a host application **114** may, for example, be based on the Hypertext Transfer Protocol (HTTP) or any other network protocol. Content delivered from the host application **114** to a client device **102** may include, for example, HTML documents, media content, etc. The communication between a client device **102** and host application **114** may include sending various requests and receiving data packets. For example, in general, a client device **102** or application running on a client device may initiate communication with a host application **114** by making a request for a specific resource (e.g., based on an HTTP request), and the application server may respond with the requested content stored in one or more response packets.

(49) In the illustrated embodiment, one or more of host applications **114** may generate various types of performance data during operation, including event logs, network data, sensor data, and other types of machine data. For example, a host application **114** comprising a web server may generate one or more web server logs in which details of interactions between the web server and any number of client devices **102** is recorded. As another example, a host device **106** comprising a router may generate one or more router logs that record information related to network traffic managed by the router. As yet another example, a host application **114** comprising a database server may generate one or more logs that record information related to requests sent from other host applications **114** (e.g., web servers or application servers) for data managed by the database server.

2.2. Client Devices

(50) Client devices **102** of FIG. **1** represent any computing device capable of interacting with one or more host devices **106** via a network **104**. Examples of client devices **102** may include, without limitation, smart phones, tablet computers, handheld computers, wearable devices, laptop computers, desktop computers, servers, portable media players, gaming devices, and so forth. In general, a client device **102** can provide access to different content, for instance, content provided by one or more host devices **106**, etc. Each client device **102** may comprise one or more client applications **110**, described in more detail in a separate section hereinafter.

2.3. Client Device Applications

(51) In some embodiments, each client device **102** may host or execute one or more client applications **110** that are capable of interacting with one or more host devices **106** via one or more networks **104**. For instance, a client application **110** may be or comprise a web browser that a user may use to navigate to one or more websites or other resources provided by one or more host devices **106**. As another example, a client application **110** may comprise a mobile application or "app." For example, an operator of a network-based service hosted by one or more host devices **106** may make available one or more mobile apps that enable users of client devices **102** to access various resources of the network-based service. As yet another example, client applications **110**

may include background processes that perform various operations without direct interaction from a user. A client application **110** may include a “plug-in” or “extension” to another application, such as a web browser plug-in or extension.

(52) In some embodiments, a client application **110** may include a monitoring component **112**. At a high level, the monitoring component **112** comprises a software component or other logic that facilitates generating performance data related to a client device's operating state, including monitoring network traffic sent and received from the client device and collecting other device and/or application-specific information. Monitoring component **112** may be an integrated component of a client application **110**, a plug-in, an extension, or any other type of add-on component. Monitoring component **112** may also be a stand-alone process.

(53) In some embodiments, a monitoring component **112** may be created when a client application **110** is developed, for example, by an application developer using a software development kit (SDK). The SDK may include custom monitoring code that can be incorporated into the code implementing a client application **110**. When the code is converted to an executable application, the custom code implementing the monitoring functionality can become part of the application itself.

(54) In some embodiments, an SDK or other code for implementing the monitoring functionality may be offered by a provider of a data intake and query system, such as a system **108**. In such cases, the provider of the system **108** can implement the custom code so that performance data generated by the monitoring functionality is sent to the system **108** to facilitate analysis of the performance data by a developer of the client application or other users.

(55) In some embodiments, the custom monitoring code may be incorporated into the code of a client application **110** in a number of different ways, such as the insertion of one or more lines in the client application code that call or otherwise invoke the monitoring component **112**. As such, a developer of a client application **110** can add one or more lines of code into the client application **110** to trigger the monitoring component **112** at desired points during execution of the application. Code that triggers the monitoring component may be referred to as a monitor trigger. For instance, a monitor trigger may be included at or near the beginning of the executable code of the client application **110** such that the monitoring component **112** is initiated or triggered as the application is launched, or included at other points in the code that correspond to various actions of the client application, such as sending a network request or displaying a particular interface.

(56) In some embodiments, the monitoring component **112** may monitor one or more aspects of network traffic sent and/or received by a client application **110**. For example, the monitoring component **112** may be configured to monitor data packets transmitted to and/or from one or more host applications **114**. Incoming and/or outgoing data packets can be read or examined to identify network data contained within the packets, for example, and other aspects of data packets can be analyzed to determine a number of network performance statistics. Monitoring network traffic may enable information to be gathered particular to the network performance associated with a client application **110** or set of applications.

(57) In some embodiments, network performance data refers to any type of data that indicates information about the network and/or network performance. Network performance data may include, for instance, a URL requested, a connection type (e.g., HTTP, HTTPS, etc.), a connection start time, a connection end time, an HTTP status code, request length, response length, request headers, response headers, connection status (e.g., completion, response time(s), failure, etc.), and the like. Upon obtaining network performance data indicating performance of the network, the network performance data can be transmitted to a data intake and query system **108** for analysis.

(58) Upon developing a client application **110** that incorporates a monitoring component **112**, the client application **110** can be distributed to client devices **102**. Applications generally can be distributed to client devices **102** in any manner, or they can be pre-loaded. In some cases, the application may be distributed to a client device **102** via an application marketplace or other application distribution system. For instance, an application marketplace or other application

distribution system might distribute the application to a client device based on a request from the client device to download the application.
(59) Examples of functionality that enables monitoring performance of a client device are described in U.S. patent application Ser. No. 14/524,748, entitled “UTILIZING PACKET HEADERS TO MONITOR NETWORK TRAFFIC IN ASSOCIATION WITH A CLIENT DEVICE”, filed on 27 Oct. 2014, and which is hereby incorporated by reference in its entirety for all purposes.
(60) In some embodiments, the monitoring component **112** may also monitor and collect performance data related to one or more aspects of the operational state of a client application **110** and/or client device **102**. For example, a monitoring component **112** may be configured to collect device performance information by monitoring one or more client device operations, or by making calls to an operating system and/or one or more other applications executing on a client device **102** for performance information. Device performance information may include, for instance, a current wireless signal strength of the device, a current connection type and network carrier, current memory performance information, a geographic location of the device, a device orientation, and any other information related to the operational state of the client device.
(61) In some embodiments, the monitoring component **112** may also monitor and collect other device profile information including, for example, a type of client device, a manufacturer and model of the device, versions of various software applications installed on the device, and so forth.
(62) In general, a monitoring component **112** may be configured to generate performance data in response to a monitor trigger in the code of a client application **110** or other triggering application event, as described above, and to store the performance data in one or more data records. Each data record, for example, may include a collection of field-value pairs, each field-value pair storing a particular item of performance data in association with a field for the item. For example, a data record generated by a monitoring component **112** may include a “networkLatency” field (not shown in the Figure) in which a value is stored. This field indicates a network latency measurement associated with one or more network requests. The data record may include a “state” field to store a value indicating a state of a network connection, and so forth for any number of aspects of collected performance data.
2.4. Data Server System
(63) FIG. **2** is a block diagram of an example data intake and query system **108**, in accordance with example embodiments. System **108** includes one or more forwarders **204** that receive data from a variety of input data sources **202**, and one or more indexers **206** that process and store the data in one or more data stores **208**. These forwarders **204** and indexers **208** can comprise separate computer systems, or may alternatively comprise separate processes executing on one or more computer systems.
(64) Each data source **202** broadly represents a distinct source of data that can be consumed by system **108**. Examples of a data sources **202** include, without limitation, data files, directories of files, data sent over a network, event logs, registries, or one or more computer systems that generate the data files, directories of files, data sent over a network, event logs, registries, etc.
(65) During operation, the forwarders **204** identify which indexers **206** receive data collected from a data source **202** and forward the data to the appropriate indexers. Forwarders **204** can also perform operations on the data before forwarding, including removing extraneous data, detecting timestamps in the data, parsing data, indexing data, routing data based on criteria relating to the data being routed, and/or performing other data transformations.
(66) In some embodiments, a forwarder **204** may comprise a service accessible to client devices **102** and host devices **106** via a network **104**. For example, one type of forwarder **204** may be capable of consuming vast amounts of real-time data from a potentially large number of client devices **102** and/or host devices **106**. The forwarder **204** may, for example, comprise a computing device which implements multiple data pipelines or “queues” to handle forwarding of network data

to indexers **206**. A forwarder **204** may also perform many of the functions that are performed by an indexer. For example, a forwarder **204** may perform keyword extractions on raw data or parse raw data to create events. A forwarder **204** may generate time stamps for events. Additionally or alternatively, a forwarder **204** may perform routing of events to indexers **206**. Data store **208** may contain events derived from machine data from a variety of sources all pertaining to the same component in an IT environment, and this data may be produced by the machine in question or by other components in the IT environment.

2.5. Cloud-Based System Overview

(67) The example data intake and query system **108** described in reference to FIG. **2** comprises several system components, including one or more forwarders, indexers, and search heads. In some environments, a user of a data intake and query system **108** may install and configure, on computing devices owned and operated by the user, one or more software applications that implement some or all of these system components. For example, a user may install a software application on server computers owned by the user and configure each server to operate as one or more of a forwarder, an indexer, a search head, etc. This arrangement generally may be referred to as an “on-premises” solution. That is, the system **108** is installed and operates on computing devices directly controlled by the user of the system. Some users may prefer an on-premises solution because it may provide a greater level of control over the configuration of certain aspects of the system (e.g., security, privacy, standards, controls, etc.). However, other users may instead prefer an arrangement in which the user is not directly responsible for providing and managing the computing devices upon which various components of system **108** operate.

(68) In one embodiment, to provide an alternative to an entirely on-premises environment for system **108**, one or more of the components of a data intake and query system instead may be provided as a cloud-based service. In this context, a cloud-based service refers to a service hosted by one more computing resources that are accessible to end users over a network, for example, by using a web browser or other application on a client device to interface with the remote computing resources. For example, a service provider may provide a cloud-based data intake and query system by managing computing resources configured to implement various aspects of the system (e.g., forwarders, indexers, search heads, etc.) and by providing access to the system to end users via a network. Typically, a user may pay a subscription or other fee to use such a service. Each subscribing user of the cloud-based service may be provided with an account that enables the user to configure a customized cloud-based system based on the user's preferences.

(69) FIG. **3** illustrates a block diagram of an example cloud-based data intake and query system. Similar to the system of FIG. **2**, the networked computer system **300** includes input data sources **202** and forwarders **204**. These input data sources and forwarders may be in a subscriber's private computing environment. Alternatively, they might be directly managed by the service provider as part of the cloud service. In the example system **300**, one or more forwarders **204** and client devices **302** are coupled to a cloud-based data intake and query system **306** via one or more networks **304**. Network **304** broadly represents one or more LANs, WANs, cellular networks, intranetworks, internetworks, etc., using any of wired, wireless, terrestrial microwave, satellite links, etc., and may include the public Internet, and is used by client devices **302** and forwarders **204** to access the system **306**. Similar to the system of **38**, each of the forwarders **204** may be configured to receive data from an input source and to forward the data to other components of the system **306** for further processing.

(70) In some embodiments, a cloud-based data intake and query system **306** may comprise a plurality of system instances **308**. In general, each system instance **308** may include one or more computing resources managed by a provider of the cloud-based system **306** made available to a particular subscriber. The computing resources comprising a system instance **308** may, for example, include one or more servers or other devices configured to implement one or more forwarders, indexers, search heads, and other components of a data intake and query system, similar to system

108. As indicated above, a subscriber may use a web browser or other application of a client device **302** to access a web portal or other interface that enables the subscriber to configure an instance **308**.

(71) Providing a data intake and query system as described in reference to system **108** as a cloud-based service presents a number of challenges. Each of the components of a system **108** (e.g., forwarders, indexers, and search heads) may at times refer to various configuration files stored locally at each component. These configuration files typically may involve some level of user configuration to accommodate particular types of data a user desires to analyze and to account for other user preferences. However, in a cloud-based service context, users typically may not have direct access to the underlying computing resources implementing the various system components (e.g., the computing resources comprising each system instance **308**) and may desire to make such configurations indirectly, for example, using one or more web-based interfaces. Thus, the techniques and systems described herein for providing user interfaces that enable a user to configure source type definitions are applicable to both on-premises and cloud-based service contexts, or some combination thereof (e.g., a hybrid system where both an on-premises environment, such as SPLUNK® ENTERPRISE, and a cloud-based environment, such as SPLUNK CLOUD™, are centrally visible).

2.6. Searching Externally-Archived Data

(72) FIG. **4** shows a block diagram of an example of a data intake and query system **108** that provides transparent search facilities for data systems that are external to the data intake and query system. Such facilities are available in the Splunk® Analytics for Hadoop® system provided by Splunk Inc. of San Francisco, California. Splunk® Analytics for Hadoop® represents an analytics platform that enables business and IT teams to rapidly explore, analyze, and visualize data in Hadoop® and NoSQL data stores.

(73) The search head **210** of the data intake and query system receives search requests from one or more client devices **404** over network connections **420**. As discussed above, the data intake and query system **108** may reside in an enterprise location, in the cloud, etc. FIG. **4** illustrates that multiple client devices **404***a*, **404***b*, . . . , **404***n* may communicate with the data intake and query system **108**. The client devices **404** may communicate with the data intake and query system using a variety of connections. For example, one client device in FIG. **4** is illustrated as communicating over an Internet (Web) protocol, another client device is illustrated as communicating via a command line interface, and another client device is illustrated as communicating via a software developer kit (SDK).

(74) The search head **210** analyzes the received search request to identify request parameters. If a search request received from one of the client devices **404** references an index maintained by the data intake and query system, then the search head **210** connects to one or more indexers **206** of the data intake and query system for the index referenced in the request parameters. That is, if the request parameters of the search request reference an index, then the search head accesses the data in the index via the indexer. The data intake and query system **108** may include one or more indexers **206**, depending on system access resources and requirements. As described further below, the indexers **206** retrieve data from their respective local data stores **208** as specified in the search request. The indexers and their respective data stores can comprise one or more storage devices and typically reside on the same system, though they may be connected via a local network connection.

(75) If the request parameters of the received search request reference an external data collection, which is not accessible to the indexers **206** or under the management of the data intake and query system, then the search head **210** can access the external data collection through an External Result Provider (ERP) process **410**. An external data collection may be referred to as a “virtual index” (plural, “virtual indices”). An ERP process provides an interface through which the search head **210** may access virtual indices.

(76) Thus, a search reference to an index of the system relates to a locally stored and managed data

collection. In contrast, a search reference to a virtual index relates to an externally stored and managed data collection, which the search head may access through one or more ERP processes **410**, **412**. FIG. **4** shows two ERP processes **410**, **412** that connect to respective remote (external) virtual indices, which are indicated as a Hadoop or another system **414** (e.g., Amazon S3, Amazon EMR, other Hadoop® Compatible File Systems (HCFS), etc.) and a relational database management system (RDBMS) **416**. Other virtual indices may include other file organizations and protocols, such as Structured Query Language (SQL) and the like. The ellipses between the ERP processes **410**, **412** indicate optional additional ERP processes of the data intake and query system **108**. An ERP process may be a computer process that is initiated or spawned by the search head **210** and is executed by the search data intake and query system **108**. Alternatively or additionally, an ERP process may be a process spawned by the search head **210** on the same or different host system as the search head **210** resides.

(77) The search head **210** may spawn a single ERP process in response to multiple virtual indices referenced in a search request, or the search head may spawn different ERP processes for different virtual indices. Generally, virtual indices that share common data configurations or protocols may share ERP processes. For example, all search query references to a Hadoop file system may be processed by the same ERP process, if the ERP process is suitably configured. Likewise, all search query references to a SQL database may be processed by the same ERP process. In addition, the search head may provide a common ERP process for common external data source types (e.g., a common vendor may utilize a common ERP process, even if the vendor includes different data storage system types, such as Hadoop and SQL). Common indexing schemes also may be handled by common ERP processes, such as flat text files or Weblog files.

(78) The search head **210** determines the number of ERP processes to be initiated via the use of configuration parameters that are included in a search request message. Generally, there is a one-to-many relationship between an external results provider "family" and ERP processes. There is also a one-to-many relationship between an ERP process and corresponding virtual indices that are referred to in a search request. For example, using RDBMS, assume two independent instances of such a system by one vendor, such as one RDBMS for production and another RDBMS used for development. In such a situation, it is likely preferable (but optional) to use two ERP processes to maintain the independent operation as between production and development data. Both of the ERPs, however, will belong to the same family, because the two RDBMS system types are from the same vendor.

(79) The ERP processes **410**, **412** receive a search request from the search head **210**. The search head may optimize the received search request for execution at the respective external virtual index. Alternatively, the ERP process may receive a search request as a result of analysis performed by the search head or by a different system process. The ERP processes **410**, **412** can communicate with the search head **210** via conventional input/output routines (e.g., standard in/standard out, etc.). In this way, the ERP process receives the search request from a client device such that the search request may be efficiently executed at the corresponding external virtual index.

(80) The ERP processes **410**, **412** may be implemented as a process of the data intake and query system. Each ERP process may be provided by the data intake and query system, or may be provided by process or application providers who are independent of the data intake and query system. Each respective ERP process may include an interface application installed at a computer of the external result provider that ensures proper communication between the search support system and the external result provider. The ERP processes **410**, **412** generate appropriate search requests in the protocol and syntax of the respective virtual indices **414**, **416**, each of which corresponds to the search request received by the search head **210**. Upon receiving search results from their corresponding virtual indices, the respective ERP process passes the result to the search head **210**, which may return or display the results or a processed set of results based on the returned results to the respective client device.

(81) Client devices **404** may communicate with the data intake and query system **108** through a network interface **420**, e.g., one or more LANs, WANs, cellular networks, intranetworks, and/or internetworks using any of wired, wireless, terrestrial microwave, satellite links, etc., and may include the public Internet.

(82) The analytics platform utilizing the External Result Provider process described in more detail in U.S. Pat. No. 8,738,629, entitled “EXTERNAL RESULT PROVIDED PROCESS FOR RETRIEVING DATA STORED USING A DIFFERENT CONFIGURATION OR PROTOCOL”, issued on 27 May 2014, U.S. Pat. No. 8,738,587, entitled “PROCESSING A SYSTEM SEARCH REQUEST BY RETRIEVING RESULTS FROM BOTH A NATIVE INDEX AND A VIRTUAL INDEX”, issued on 25 Jul. 2013, U.S. patent application Ser. No. 14/266,832, entitled “PROCESSING A SYSTEM SEARCH REQUEST ACROSS DISPARATE DATA COLLECTION SYSTEMS”, filed on 1 May 2014, and U.S. Pat. No. 9,514,189, entitled “PROCESSING A SYSTEM SEARCH REQUEST INCLUDING EXTERNAL DATA SOURCES”, issued on 6 Dec. 2016, each of which is hereby incorporated by reference in its entirety for all purposes.

2.6.1. ERP Process Features

(83) The ERP processes described above may include two operation modes: a streaming mode and a reporting mode. The ERP processes can operate in streaming mode only, in reporting mode only, or in both modes simultaneously. Operating in both modes simultaneously is referred to as mixed mode operation. In a mixed mode operation, the ERP at some point can stop providing the search head with streaming results and only provide reporting results thereafter, or the search head at some point may start ignoring streaming results it has been using and only use reporting results thereafter.

(84) The streaming mode returns search results in real time, with minimal processing, in response to the search request. The reporting mode provides results of a search request with processing of the search results prior to providing them to the requesting search head, which in turn provides results to the requesting client device. ERP operation with such multiple modes provides greater performance flexibility with regard to report time, search latency, and resource utilization.

(85) In a mixed mode operation, both streaming mode and reporting mode are operating simultaneously. The streaming mode results (e.g., the machine data obtained from the external data source) are provided to the search head, which can then process the results data (e.g., break the machine data into events, timestamp it, filter it, etc.) and integrate the results data with the results data from other external data sources, and/or from data stores of the search head. The search head performs such processing and can immediately start returning interim (streaming mode) results to the user at the requesting client device; simultaneously, the search head is waiting for the ERP process to process the data it is retrieving from the external data source as a result of the concurrently executing reporting mode.

(86) In some instances, the ERP process initially operates in a mixed mode, such that the streaming mode operates to enable the ERP quickly to return interim results (e.g., some of the machined data or unprocessed data necessary to respond to a search request) to the search head, enabling the search head to process the interim results and begin providing to the client or search requester interim results that are responsive to the query. Meanwhile, in this mixed mode, the ERP also operates concurrently in reporting mode, processing portions of machine data in a manner responsive to the search query. Upon determining that it has results from the reporting mode available to return to the search head, the ERP may halt processing in the mixed mode at that time (or some later time) by stopping the return of data in streaming mode to the search head and switching to reporting mode only. The ERP at this point starts sending interim results in reporting mode to the search head, which in turn may then present this processed data responsive to the search request to the client or search requester. Typically the search head switches from using results from the ERP's streaming mode of operation to results from the ERP's reporting mode of operation when the higher bandwidth results from the reporting mode outstrip the amount of data

processed by the search head in the streaming mode of ERP operation.
(87) A reporting mode may have a higher bandwidth because the ERP does not have to spend time transferring data to the search head for processing all the machine data. In addition, the ERP may optionally direct another processor to do the processing.
(88) The streaming mode of operation does not need to be stopped to gain the higher bandwidth benefits of a reporting mode; the search head could simply stop using the streaming mode results—and start using the reporting mode results—when the bandwidth of the reporting mode has caught up with or exceeded the amount of bandwidth provided by the streaming mode. Thus, a variety of triggers and ways to accomplish a search head's switch from using streaming mode results to using reporting mode results may be appreciated by one skilled in the art.
(89) The reporting mode can involve the ERP process (or an external system) performing event breaking, time stamping, filtering of events to match the search query request, and calculating statistics on the results. The user can request particular types of data, such as if the search query itself involves types of events, or the search request may ask for statistics on data, such as on events that meet the search request. In either case, the search head understands the query language used in the received query request, which may be a proprietary language. One exemplary query language is Splunk Processing Language (SPL) developed by the assignee of the application, Splunk Inc. The search head typically understands how to use that language to obtain data from the indexers, which store data in a format used by the SPLUNK® Enterprise system.
(90) The ERP processes support the search head, as the search head is not ordinarily configured to understand the format in which data is stored in external data sources such as Hadoop or SQL data systems. Rather, the ERP process performs that translation from the query submitted in the search support system's native format (e.g., SPL if SPLUNK® ENTERPRISE is used as the search support system) to a search query request format that will be accepted by the corresponding external data system. The external data system typically stores data in a different format from that of the search support system's native index format, and it utilizes a different query language (e.g., SQL or MapReduce, rather than SPL or the like).
(91) As noted, the ERP process can operate in the streaming mode alone. After the ERP process has performed the translation of the query request and received raw results from the streaming mode, the search head can integrate the returned data with any data obtained from local data sources (e.g., native to the search support system), other external data sources, and other ERP processes (if such operations were required to satisfy the terms of the search query). An advantage of mixed mode operation is that, in addition to streaming mode, the ERP process is also executing concurrently in reporting mode. Thus, the ERP process (rather than the search head) is processing query results (e.g., performing event breaking, timestamping, filtering, possibly calculating statistics if required to be responsive to the search query request, etc.). It should be apparent to those skilled in the art that additional time is needed for the ERP process to perform the processing in such a configuration. Therefore, the streaming mode will allow the search head to start returning interim results to the user at the client device before the ERP process can complete sufficient processing to start returning any search results. The switchover between streaming and reporting mode happens when the ERP process determines that the switchover is appropriate, such as when the ERP process determines it can begin returning meaningful results from its reporting mode.
(92) The operation described above illustrates the source of operational latency: streaming mode has low latency (immediate results) and usually has relatively low bandwidth (fewer results can be returned per unit of time). In contrast, the concurrently running reporting mode has relatively high latency (it has to perform a lot more processing before returning any results) and usually has relatively high bandwidth (more results can be processed per unit of time). For example, when the ERP process does begin returning report results, it returns more processed results than in the streaming mode, because, e.g., statistics only need to be calculated to be responsive to the search request. That is, the ERP process doesn't have to take time to first return machine data to the search

head. As noted, the ERP process could be configured to operate in streaming mode alone and return just the machine data for the search head to process in a way that is responsive to the search request. Alternatively, the ERP process can be configured to operate in the reporting mode only. Also, the ERP process can be configured to operate in streaming mode and reporting mode concurrently, as described, with the ERP process stopping the transmission of streaming results to the search head when the concurrently running reporting mode has caught up and started providing results. The reporting mode does not require the processing of all machine data that is responsive to the search query request before the ERP process starts returning results; rather, the reporting mode usually performs processing of chunks of events and returns the processing results to the search head for each chunk.

(93) For example, an ERP process can be configured to merely return the contents of a search result file verbatim, with little or no processing of results. That way, the search head performs all processing (such as parsing byte streams into events, filtering, etc.). The ERP process can be configured to perform additional intelligence, such as analyzing the search request and handling all the computation that a native search indexer process would otherwise perform. In this way, the configured ERP process provides greater flexibility in features while operating according to desired preferences, such as response latency and resource requirements.

2.7. Data Ingestion

(94) FIG. **5**A is a flow chart of an example method that illustrates how indexers process, index, and store data received from forwarders, in accordance with example embodiments. The data flow illustrated in FIG. **5**A is provided for illustrative purposes only; those skilled in the art would understand that one or more of the steps of the processes illustrated in FIG. **5**A may be removed or that the ordering of the steps may be changed. Furthermore, for the purposes of illustrating a clear example, one or more particular system components are described in the context of performing various operations during each of the data flow stages. For example, a forwarder is described as receiving and processing machine data during an input phase; an indexer is described as parsing and indexing machine data during parsing and indexing phases; and a search head is described as performing a search query during a search phase. However, other system arrangements and distributions of the processing steps across system components may be used.

2.7.1. Input

(95) At block **502**, a forwarder receives data from an input source, such as a data source **202** shown in FIG. **2**. A forwarder initially may receive the data as a raw data stream generated by the input source. For example, a forwarder may receive a data stream from a log file generated by an application server, from a stream of network data from a network device, or from any other source of data. In some embodiments, a forwarder receives the raw data and may segment the data stream into “blocks”, possibly of a uniform data size, to facilitate subsequent processing steps.

(96) At block **504**, a forwarder or other system component annotates each block generated from the raw data with one or more metadata fields. These metadata fields may, for example, provide information related to the data block as a whole and may apply to each event that is subsequently derived from the data in the data block. For example, the metadata fields may include separate fields specifying each of a host, a source, and a source type related to the data block. A host field may contain a value identifying a host name or IP address of a device that generated the data. A source field may contain a value identifying a source of the data, such as a pathname of a file or a protocol and port related to received network data. A source type field may contain a value specifying a particular source type label for the data. Additional metadata fields may also be included during the input phase, such as a character encoding of the data, if known, and possibly other values that provide information relevant to later processing steps. In some embodiments, a forwarder forwards the annotated data blocks to another system component (typically an indexer) for further processing.

(97) The data intake and query system allows forwarding of data from one data intake and query

instance to another, or even to a third-party system. The data intake and query system can employ different types of forwarders in a configuration.
(98) In some embodiments, a forwarder may contain the essential components needed to forward data. A forwarder can gather data from a variety of inputs and forward the data to an indexer for indexing and searching. A forwarder can also tag metadata (e.g., source, source type, host, etc.).
(99) In some embodiments, a forwarder has the capabilities of the aforementioned forwarder as well as additional capabilities. The forwarder can parse data before forwarding the data (e.g., can associate a time stamp with a portion of data and create an event, etc.) and can route data based on criteria such as source or type of event. The forwarder can also index data locally while forwarding the data to another indexer.

2.7.2. Parsing

(100) At block **506**, an indexer receives data blocks from a forwarder and parses the data to organize the data into events. In some embodiments, to organize the data into events, an indexer may determine a source type associated with each data block (e.g., by extracting a source type label from the metadata fields associated with the data block, etc.) and refer to a source type configuration corresponding to the identified source type. The source type definition may include one or more properties that indicate to the indexer to automatically determine the boundaries within the received data that indicate the portions of machine data for events. In general, these properties may include regular expression-based rules or delimiter rules where, for example, event boundaries may be indicated by predefined characters or character strings. These predefined characters may include punctuation marks or other special characters including, for example, carriage returns, tabs, spaces, line breaks, etc. If a source type for the data is unknown to the indexer, an indexer may infer a source type for the data by examining the structure of the data. Then, the indexer can apply an inferred source type definition to the data to create the events.
(101) At block **508**, the indexer determines a timestamp for each event. Similar to the process for parsing machine data, an indexer may again refer to a source type definition associated with the data to locate one or more properties that indicate instructions for determining a timestamp for each event. The properties may, for example, instruct an indexer to extract a time value from a portion of data for the event, to interpolate time values based on timestamps associated with temporally proximate events, to create a timestamp based on a time the portion of machine data was received or generated, to use the timestamp of a previous event, or use any other rules for determining timestamps.
(102) At block **510**, the indexer associates with each event one or more metadata fields including a field containing the timestamp determined for the event. In some embodiments, a timestamp may be included in the metadata fields. These metadata fields may include any number of “default fields” that are associated with all events, and may also include one more custom fields as defined by a user. Similar to the metadata fields associated with the data blocks at block **504**, the default metadata fields associated with each event may include a host, source, and source type field including or in addition to a field storing the timestamp.
(103) At block **512**, an indexer may optionally apply one or more transformations to data included in the events created at block **506**. For example, such transformations can include removing a portion of an event (e.g., a portion used to define event boundaries, extraneous characters from the event, other extraneous text, etc.), masking a portion of an event (e.g., masking a credit card number), removing redundant portions of an event, etc. The transformations applied to events may, for example, be specified in one or more configuration files and referenced by one or more source type definitions.
(104) FIG. **5**C illustrates an illustrative example of machine data can be stored in a data store in accordance with various disclosed embodiments. In other embodiments, machine data can be stored in a flat file in a corresponding bucket with an associated index file, such as a time series index or “TSIDX.” As such, the depiction of machine data and associated metadata as rows and columns in

the table of FIG. **5**C is merely illustrative and is not intended to limit the data format in which the machine data and metadata is stored in various embodiments described herein. In one particular embodiment, machine data can be stored in a compressed or encrypted formatted. In such embodiments, the machine data can be stored with or be associated with data that describes the compression or encryption scheme with which the machine data is stored. The information about the compression or encryption scheme can be used to decompress or decrypt the machine data, and any metadata with which it is stored, at search time.

(105) As mentioned above, certain metadata, e.g., host **536**, source **537**, source type **538** and timestamps **535** can be generated for each event, and associated with a corresponding portion of machine data **539** when storing the event data in a data store, e.g., data store **208**. Any of the metadata can be extracted from the corresponding machine data, or supplied or defined by an entity, such as a user or computer system. The metadata fields can become part of or stored with the event. Note that while the time-stamp metadata field can be extracted from the raw data of each event, the values for the other metadata fields may be determined by the indexer based on information it receives pertaining to the source of the data separate from the machine data.

(106) While certain default or user-defined metadata fields can be extracted from the machine data for indexing purposes, all the machine data within an event can be maintained in its original condition. As such, in embodiments in which the portion of machine data included in an event is unprocessed or otherwise unaltered, it is referred to herein as a portion of raw machine data. In other embodiments, the port of machine data in an event can be processed or otherwise altered. As such, unless certain information needs to be removed for some reasons (e.g. extraneous information, confidential information), all the raw machine data contained in an event can be preserved and saved in its original form. Accordingly, the data store in which the event records are stored is sometimes referred to as a "raw record data store." The raw record data store contains a record of the raw event data tagged with the various default fields.

(107) In FIG. **5**C, the first three rows of the table represent events **531**, **532**, and **533** and are related to a server access log that records requests from multiple clients processed by a server, as indicated by entry of "access.log" in the source column **536**.

(108) In the example shown in FIG. **5**C, each of the events **531-533** is associated with a discrete request made from a client device. The raw machine data generated by the server and extracted from a server access log can include the IP address of the client **540**, the user id of the person requesting the document **541**, the time the server finished processing the request **542**, the request line from the client **543**, the status code returned by the server to the client **545**, the size of the object returned to the client (in this case, the gif file requested by the client) **546** and the time spent to serve the request in microseconds **544**. As seen in FIG. **5**C, all the raw machine data retrieved from the server access log is retained and stored as part of the corresponding events, **531-533** in the data store.

(109) Event **534** is associated with an entry in a server error log, as indicated by "error.log" in the source column **537**, that records errors that the server encountered when processing a client request. Similar to the events related to the server access log, all the raw machine data in the error log file pertaining to event **534** can be preserved and stored as part of the event **534**.

(110) Saving minimally processed or unprocessed machine data in a data store associated with metadata fields in the manner similar to that shown in FIG. **5**C is advantageous because it allows search of all the machine data at search time instead of searching only previously specified and identified fields or field-value pairs. As mentioned above, because data structures used by various embodiments of the present disclosure maintain the underlying raw machine data and use a late-binding schema for searching the raw machines data, it enables a user to continue investigating and learn valuable insights about the raw data. In other words, the user is not compelled to know about all the fields of information that will be needed at data ingestion time. As a user learns more about the data in the events, the user can continue to refine the late-binding schema by defining new

extraction rules, or modifying or deleting existing extraction rules used by the system.

2.7.3. Indexing

(111) At blocks **514** and **516**, an indexer can optionally generate a keyword index to facilitate fast keyword searching for events. To build a keyword index, at block **514**, the indexer identifies a set of keywords in each event. At block **516**, the indexer includes the identified keywords in an index, which associates each stored keyword with reference pointers to events containing that keyword (or to locations within events where that keyword is located, other location identifiers, etc.). When an indexer subsequently receives a keyword-based query, the indexer can access the keyword index to quickly identify events containing the keyword.

(112) In some embodiments, the keyword index may include entries for field name-value pairs found in events, where a field name-value pair can include a pair of keywords connected by a symbol, such as an equals sign or colon. This way, events containing these field name-value pairs can be quickly located. In some embodiments, fields can automatically be generated for some or all of the field names of the field name-value pairs at the time of indexing. For example, if the string "dest=10.0.1.2" is found in an event, a field named "dest" may be created for the event, and assigned a value of "10.0.1.2".

(113) At block **518**, the indexer stores the events with an associated timestamp in a data store **208**. Timestamps enable a user to search for events based on a time range. In some embodiments, the stored events are organized into "buckets," where each bucket stores events associated with a specific time range based on the timestamps associated with each event. This improves time-based searching, as well as allows for events with recent timestamps, which may have a higher likelihood of being accessed, to be stored in a faster memory to facilitate faster retrieval. For example, buckets containing the most recent events can be stored in flash memory rather than on a hard disk. In some embodiments, each bucket may be associated with an identifier, a time range, and a size constraint.

(114) Each indexer **206** may be responsible for storing and searching a subset of the events contained in a corresponding data store **208**. By distributing events among the indexers and data stores, the indexers can analyze events for a query in parallel. For example, using map-reduce techniques, each indexer returns partial responses for a subset of events to a search head that combines the results to produce an answer for the query. By storing events in buckets for specific time ranges, an indexer may further optimize the data retrieval process by searching buckets corresponding to time ranges that are relevant to a query.

(115) In some embodiments, each indexer has a home directory and a cold directory. The home directory of an indexer stores hot buckets and warm buckets, and the cold directory of an indexer stores cold buckets. A hot bucket is a bucket that is capable of receiving and storing events. A warm bucket is a bucket that can no longer receive events for storage but has not yet been moved to the cold directory. A cold bucket is a bucket that can no longer receive events and may be a bucket that was previously stored in the home directory. The home directory may be stored in faster memory, such as flash memory, as events may be actively written to the home directory, and the home directory may typically store events that are more frequently searched and thus are accessed more frequently. The cold directory may be stored in slower and/or larger memory, such as a hard disk, as events are no longer being written to the cold directory, and the cold directory may typically store events that are not as frequently searched and thus are accessed less frequently. In some embodiments, an indexer may also have a quarantine bucket that contains events having potentially inaccurate information, such as an incorrect time stamp associated with the event or a time stamp that appears to be an unreasonable time stamp for the corresponding event. The quarantine bucket may have events from any time range; as such, the quarantine bucket may always be searched at search time. Additionally, an indexer may store old, archived data in a frozen bucket that is not capable of being searched at search time. In some embodiments, a frozen bucket may be stored in slower and/or larger memory, such as a hard disk, and may be stored in offline and/or remote storage.

(116) Moreover, events and buckets can also be replicated across different indexers and data stores to facilitate high availability and disaster recovery as described in U.S. Pat. No. 9,130,971, entitled "Site-Based Search Affinity", issued on 8 Sep. 2015, and in U.S. patent Ser. No. 14/266,817, entitled "Multi-Site Clustering", issued on 1 Sep. 2015, each of which is hereby incorporated by reference in its entirety for all purposes.

(117) FIG. **5**B is a block diagram of an example data store **501** that includes a directory for each index (or partition) that contains a portion of data managed by an indexer. FIG. **5**B further illustrates details of an embodiment of an inverted index **507**B and an event reference array **515** associated with inverted index **507**B.

(118) The data store **501** can correspond to a data store **208** that stores events managed by an indexer **206** or can correspond to a different data store associated with an indexer **206**. In the illustrated embodiment, the data store **501** includes a _main directory **503** associated with a _main index and a _test directory **505** associated with a _test index. However, the data store **501** can include fewer or more directories. In some embodiments, multiple indexes can share a single directory or all indexes can share a common directory. Additionally, although illustrated as a single data store **501**, it will be understood that the data store **501** can be implemented as multiple data stores storing different portions of the information shown in FIG. **5**B. For example, a single index or partition can span multiple directories or multiple data stores, and can be indexed or searched by multiple corresponding indexers.

(119) In the illustrated embodiment of FIG. **5**B, the index-specific directories **503** and **505** include inverted indexes **507**A, **507**B and **509**A, **509**B, respectively. The inverted indexes **507**A . . . **507**B, and **509**A . . . **509**B can be keyword indexes or field-value pair indexes described herein and can include less or more information that depicted in FIG. **5**B.

(120) In some embodiments, the inverted index **507**A . . . **507**B, and **509**A . . . **509**B can correspond to a distinct time-series bucket that is managed by the indexer **206** and that contains events corresponding to the relevant index (e.g., _main index, _test index). As such, each inverted index can correspond to a particular range of time for an index. Additional files, such as high performance indexes for each time-series bucket of an index, can also be stored in the same directory as the inverted indexes **507**A . . . **507**B, and **509**A . . . **509**B. In some embodiments inverted index **507**A . . . **507**B, and **509**A . . . **509**B can correspond to multiple time-series buckets or inverted indexes **507**A . . . **507**B, and **509**A . . . **509**B can correspond to a single time-series bucket.

(121) Each inverted index **507**A . . . **507**B, and **509**A . . . **509**B can include one or more entries, such as keyword (or token) entries or field-value pair entries. Furthermore, in certain embodiments, the inverted indexes **507**A . . . **507**B, and **509**A . . . **509**B can include additional information, such as a time range **523** associated with the inverted index or an index identifier **525** identifying the index associated with the inverted index **507**A . . . **507**B, and **509**A . . . **509**B. However, each inverted index **507**A . . . **507**B, and **509**A . . . **509**B can include less or more information than depicted.

(122) Token entries, such as token entries **511** illustrated in inverted index **507**B, can include a token **511**A (e.g., "error," "itemID," etc.) and event references **511**B indicative of events that include the token. For example, for the token "error," the corresponding token entry includes the token "error" and an event reference, or unique identifier, for each event stored in the corresponding time-series bucket that includes the token "error." In the illustrated embodiment of FIG. **5**B, the error token entry includes the identifiers 3, 5, 6, 8, 11, and 12 corresponding to events managed by the indexer **206** and associated with the index_main **503** that are located in the time-series bucket associated with the inverted index **507**B.

(123) In some cases, some token entries can be default entries, automatically determined entries, or user specified entries. In some embodiments, the indexer **206** can identify each word or string in an event as a distinct token and generate a token entry for it. In some cases, the indexer **206** can

identify the beginning and ending of tokens based on punctuation, spaces, as described in greater detail herein. In certain cases, the indexer **206** can rely on user input or a configuration file to identify tokens for token entries **511**, etc. It will be understood that any combination of token entries can be included as a default, automatically determined, a or included based on user-specified criteria.

(124) Similarly, field-value pair entries, such as field-value pair entries **513** shown in inverted index **507**B, can include a field-value pair **513**A and event references **513**B indicative of events that include a field value that corresponds to the field-value pair. For example, for a field-value pair sourcetype::sendmail, a field-value pair entry would include the field-value pair sourcetype::sendmail and a unique identifier, or event reference, for each event stored in the corresponding time-series bucket that includes a sendmail sourcetype.

(125) In some cases, the field-value pair entries **513** can be default entries, automatically determined entries, or user specified entries. As a non-limiting example, the field-value pair entries for the fields host, source, sourcetype can be included in the inverted indexes **507**A . . . **507**B, and **509**A . . . **509**B as a default. As such, all of the inverted indexes **507**A . . . **507**B, and **509**A . . . **509**B can include field-value pair entries for the fields host, source, sourcetype. As yet another non-limiting example, the field-value pair entries for the IP_address field can be user specified and may only appear in the inverted index **507**B based on user-specified criteria. As another non-limiting example, as the indexer indexes the events, it can automatically identify field-value pairs and create field-value pair entries. For example, based on the indexers review of events, it can identify IP_address as a field in each event and add the IP_address field-value pair entries to the inverted index **507**B. It will be understood that any combination of field-value pair entries can be included as a default, automatically determined, or included based on user-specified criteria.

(126) Each unique identifier **517**, or event reference, can correspond to a unique event located in the time series bucket. However, the same event reference can be located in multiple entries. For example if an event has a sourcetype splunkd, host www1 and token “warning,” then the unique identifier for the event will appear in the field-value pair entries sourcetype::splunkd and host::www1, as well as the token entry “warning.” With reference to the illustrated embodiment of FIG. **5**B and the event that corresponds to the event reference 3, the event reference 3 is found in the field-value pair entries **513** host::hostA, source::sourceB, sourcetype::sourcetypeA, and IP_address::91.205.189.15 indicating that the event corresponding to the event references is from hostA, sourceB, of sourcetypeA, and includes 91.205.189.15 in the event data.

(127) For some fields, the unique identifier is located in only one field-value pair entry for a particular field. For example, the inverted index may include four sourcetype field-value pair entries corresponding to four different sourcetypes of the events stored in a bucket (e.g., sourcetypes: sendmail, splunkd, web_access, and web_service). Within those four sourcetype field-value pair entries, an identifier for a particular event may appear in only one of the field-value pair entries. With continued reference to the example illustrated embodiment of FIG. **5**B, since the event reference 7 appears in the field-value pair entry sourcetype::sourcetypeA, then it does not appear in the other field-value pair entries for the sourcetype field, including sourcetype::sourcetypeB, sourcetype::sourcetypeC, and sourcetype::sourcetypeD.

(128) The event references **517** can be used to locate the events in the corresponding bucket. For example, the inverted index can include, or be associated with, an event reference array **515**. The event reference array **515** can include an array entry **517** for each event reference in the inverted index **507**B. Each array entry **517** can include location information **519** of the event corresponding to the unique identifier (non-limiting example: seek address of the event), a timestamp **521** associated with the event, or additional information regarding the event associated with the event reference, etc.

(129) For each token entry **511** or field-value pair entry **513**, the event reference **501**B or unique identifiers can be listed in chronological order or the value of the event reference can be assigned

based on chronological data, such as a timestamp associated with the event referenced by the event reference. For example, the event reference 1 in the illustrated embodiment of FIG. **5**B can correspond to the first-in-time event for the bucket, and the event reference **12** can correspond to the last-in-time event for the bucket. However, the event references can be listed in any order, such as reverse chronological order, ascending order, descending order, or some other order, etc. Further, the entries can be sorted. For example, the entries can be sorted alphabetically (collectively or within a particular group), by entry origin (e.g., default, automatically generated, user-specified, etc.), by entry type (e.g., field-value pair entry, token entry, etc.), or chronologically by when added to the inverted index, etc. In the illustrated embodiment of FIG. **5**B, the entries are sorted first by entry type and then alphabetically.

(130) As a non-limiting example of how the inverted indexes **507**A . . . **507**B, and **509**A . . . **509**B can be used during a data categorization request command, the indexers can receive filter criteria indicating data that is to be categorized and categorization criteria indicating how the data is to be categorized. Example filter criteria can include, but is not limited to, indexes (or partitions), hosts, sources, sourcetypes, time ranges, field identifier, keywords, etc.

(131) Using the filter criteria, the indexer identifies relevant inverted indexes to be searched. For example, if the filter criteria includes a set of partitions, the indexer can identify the inverted indexes stored in the directory corresponding to the particular partition as relevant inverted indexes. Other means can be used to identify inverted indexes associated with a partition of interest. For example, in some embodiments, the indexer can review an entry in the inverted indexes, such as an index-value pair entry **513** to determine if a particular inverted index is relevant. If the filter criteria does not identify any partition, then the indexer can identify all inverted indexes managed by the indexer as relevant inverted indexes.

(132) Similarly, if the filter criteria includes a time range, the indexer can identify inverted indexes corresponding to buckets that satisfy at least a portion of the time range as relevant inverted indexes. For example, if the time range is last hour then the indexer can identify all inverted indexes that correspond to buckets storing events associated with timestamps within the last hour as relevant inverted indexes.

(133) When used in combination, an index filter criterion specifying one or more partitions and a time range filter criterion specifying a particular time range can be used to identify a subset of inverted indexes within a particular directory (or otherwise associated with a particular partition) as relevant inverted indexes. As such, the indexer can focus the processing to only a subset of the total number of inverted indexes that the indexer manages.

(134) Once the relevant inverted indexes are identified, the indexer can review them using any additional filter criteria to identify events that satisfy the filter criteria. In some cases, using the known location of the directory in which the relevant inverted indexes are located, the indexer can determine that any events identified using the relevant inverted indexes satisfy an index filter criterion. For example, if the filter criteria includes a partition main, then the indexer can determine that any events identified using inverted indexes within the partition main directory (or otherwise associated with the partition main) satisfy the index filter criterion.

(135) Furthermore, based on the time range associated with each inverted index, the indexer can determine that that any events identified using a particular inverted index satisfies a time range filter criterion. For example, if a time range filter criterion is for the last hour and a particular inverted index corresponds to events within a time range of 50 minutes ago to 35 minutes ago, the indexer can determine that any events identified using the particular inverted index satisfy the time range filter criterion. Conversely, if the particular inverted index corresponds to events within a time range of 59 minutes ago to 62 minutes ago, the indexer can determine that some events identified using the particular inverted index may not satisfy the time range filter criterion.

(136) Using the inverted indexes, the indexer can identify event references (and therefore events) that satisfy the filter criteria. For example, if the token “error” is a filter criterion, the indexer can

track all event references within the token entry “error.” Similarly, the indexer can identify other event references located in other token entries or field-value pair entries that match the filter criteria. The system can identify event references located in all of the entries identified by the filter criteria. For example, if the filter criteria include the token “error” and field-value pair sourcetype::web_ui, the indexer can track the event references found in both the token entry “error” and the field-value pair entry sourcetype::web_ui. As mentioned previously, in some cases, such as when multiple values are identified for a particular filter criterion (e.g., multiple sources for a source filter criterion), the system can identify event references located in at least one of the entries corresponding to the multiple values and in all other entries identified by the filter criteria. The indexer can determine that the events associated with the identified event references satisfy the filter criteria.

(137) In some cases, the indexer can further consult a timestamp associated with the event reference to determine whether an event satisfies the filter criteria. For example, if an inverted index corresponds to a time range that is partially outside of a time range filter criterion, then the indexer can consult a timestamp associated with the event reference to determine whether the corresponding event satisfies the time range criterion. In some embodiments, to identify events that satisfy a time range, the indexer can review an array, such as the event reference array **515** that identifies the time associated with the events. Furthermore, as mentioned above using the known location of the directory in which the relevant inverted indexes are located (or other index identifier), the indexer can determine that any events identified using the relevant inverted indexes satisfy the index filter criterion.

(138) In some cases, based on the filter criteria, the indexer reviews an extraction rule. In certain embodiments, if the filter criteria includes a field name that does not correspond to a field-value pair entry in an inverted index, the indexer can review an extraction rule, which may be located in a configuration file, to identify a field that corresponds to a field-value pair entry in the inverted index.

(139) For example, the filter criteria includes a field name “sessionID” and the indexer determines that at least one relevant inverted index does not include a field-value pair entry corresponding to the field name sessionID, the indexer can review an extraction rule that identifies how the sessionID field is to be extracted from a particular host, source, or sourcetype (implicitly identifying the particular host, source, or sourcetype that includes a sessionID field). The indexer can replace the field name “sessionID” in the filter criteria with the identified host, source, or sourcetype. In some cases, the field name “sessionID” may be associated with multiples hosts, sources, or sourcetypes, in which case, all identified hosts, sources, and sourcetypes can be added as filter criteria. In some cases, the identified host, source, or sourcetype can replace or be appended to a filter criterion, or be excluded. For example, if the filter criteria includes a criterion for source S**1** and the “sessionID” field is found in source S**2**, the source S**2** can replace S**1** in the filter criteria, be appended such that the filter criteria includes source S**1** and source S**2**, or be excluded based on the presence of the filter criterion source S**1**. If the identified host, source, or sourcetype is included in the filter criteria, the indexer can then identify a field-value pair entry in the inverted index that includes a field value corresponding to the identity of the particular host, source, or sourcetype identified using the extraction rule.

(140) Once the events that satisfy the filter criteria are identified, the system, such as the indexer **206** can categorize the results based on the categorization criteria. The categorization criteria can include categories for grouping the results, such as any combination of partition, source, sourcetype, or host, or other categories or fields as desired.

(141) The indexer can use the categorization criteria to identify categorization criteria-value pairs or categorization criteria values by which to categorize or group the results. The categorization criteria-value pairs can correspond to one or more field-value pair entries stored in a relevant inverted index, one or more index-value pairs based on a directory in which the inverted index is

located or an entry in the inverted index (or other means by which an inverted index can be associated with a partition), or other criteria-value pair that identifies a general category and a particular value for that category. The categorization criteria values can correspond to the value portion of the categorization criteria-value pair.

(142) As mentioned, in some cases, the categorization criteria-value pairs can correspond to one or more field-value pair entries stored in the relevant inverted indexes. For example, the categorization criteria-value pairs can correspond to field-value pair entries of host, source, and sourcetype (or other field-value pair entry as desired). For instance, if there are ten different hosts, four different sources, and five different sourcetypes for an inverted index, then the inverted index can include ten host field-value pair entries, four source field-value pair entries, and five sourcetype field-value pair entries. The indexer can use the nineteen distinct field-value pair entries as categorization criteria-value pairs to group the results.

(143) Specifically, the indexer can identify the location of the event references associated with the events that satisfy the filter criteria within the field-value pairs, and group the event references based on their location. As such, the indexer can identify the particular field value associated with the event corresponding to the event reference. For example, if the categorization criteria include host and sourcetype, the host field-value pair entries and sourcetype field-value pair entries can be used as categorization criteria-value pairs to identify the specific host and sourcetype associated with the events that satisfy the filter criteria.

(144) In addition, as mentioned, categorization criteria-value pairs can correspond to data other than the field-value pair entries in the relevant inverted indexes. For example, if partition or index is used as a categorization criterion, the inverted indexes may not include partition field-value pair entries. Rather, the indexer can identify the categorization criteria-value pair associated with the partition based on the directory in which an inverted index is located, information in the inverted index, or other information that associates the inverted index with the partition, etc. As such a variety of methods can be used to identify the categorization criteria-value pairs from the categorization criteria.

(145) Accordingly based on the categorization criteria (and categorization criteria-value pairs), the indexer can generate groupings based on the events that satisfy the filter criteria. As a non-limiting example, if the categorization criteria includes a partition and sourcetype, then the groupings can correspond to events that are associated with each unique combination of partition and sourcetype. For instance, if there are three different partitions and two different sourcetypes associated with the identified events, then the six different groups can be formed, each with a unique partition value-sourcetype value combination. Similarly, if the categorization criteria includes partition, sourcetype, and host and there are two different partitions, three sourcetypes, and five hosts associated with the identified events, then the indexer can generate up to thirty groups for the results that satisfy the filter criteria. Each group can be associated with a unique combination of categorization criteria-value pairs (e.g., unique combinations of partition value sourcetype value, and host value).

(146) In addition, the indexer can count the number of events associated with each group based on the number of events that meet the unique combination of categorization criteria for a particular group (or match the categorization criteria-value pairs for the particular group). With continued reference to the example above, the indexer can count the number of events that meet the unique combination of partition, sourcetype, and host for a particular group.

(147) Each indexer communicates the groupings to the search head. The search head can aggregate the groupings from the indexers and provide the groupings for display. In some cases, the groups are displayed based on at least one of the host, source, sourcetype, or partition associated with the groupings. In some embodiments, the search head can further display the groups based on display criteria, such as a display order or a sort order as described in greater detail above.

(148) As a non-limiting example and with reference to FIG. **5**B, consider a request received by an

indexer **206** that includes the following filter criteria: keyword=error, partition=main, time range=3/1/17 16:22.00.000-16:28.00.000, sourcetype=sourcetypeC, host=hostB, and the following categorization criteria: source.
(149) Based on the above criteria, the indexer **206** identifies _main directory **503** and can ignore _test directory **505** and any other partition-specific directories. The indexer determines that inverted partition **507**B is a relevant partition based on its location within the _main directory **503** and the time range associated with it. For sake of simplicity in this example, the indexer **206** determines that no other inverted indexes in the _main directory **503**, such as inverted index **507**A satisfy the time range criterion.
(150) Having identified the relevant inverted index **507**B, the indexer reviews the token entries **511** and the field-value pair entries **513** to identify event references, or events, that satisfy all of the filter criteria.
(151) With respect to the token entries **511**, the indexer can review the error token entry and identify event references 3, 5, 6, 8, 11, 12, indicating that the term “error” is found in the corresponding events. Similarly, the indexer can identify event references 4, 5, 6, 8, 9, 10, 11 in the field-value pair entry sourcetype::sourcetypeC and event references 2, 5, 6, 8, 10, 11 in the field-value pair entry host::hostB. As the filter criteria did not include a source or an IP_address field-value pair, the indexer can ignore those field-value pair entries.
(152) In addition to identifying event references found in at least one token entry or field-value pair entry (e.g., event references 3, 4, 5, 6, 8, 9, 10, 11, 12), the indexer can identify events (and corresponding event references) that satisfy the time range criterion using the event reference array **1614** (e.g., event references 2, 3, 4, 5, 6, 7, 8, 9, 10). Using the information obtained from the inverted index **507**B (including the event reference array **515**), the indexer **206** can identify the event references that satisfy all of the filter criteria (e.g., event references 5, 6, 8).
(153) Having identified the events (and event references) that satisfy all of the filter criteria, the indexer **206** can group the event references using the received categorization criteria (source). In doing so, the indexer can determine that event references 5 and 6 are located in the field-value pair entry source::sourceD (or have matching categorization criteria-value pairs) and event reference 8 is located in the field-value pair entry source::sourceC. Accordingly, the indexer can generate a sourceC group having a count of one corresponding to reference 8 and a sourceD group having a count of two corresponding to references 5 and 6. This information can be communicated to the search head. In turn the search head can aggregate the results from the various indexers and display the groupings. As mentioned above, in some embodiments, the groupings can be displayed based at least in part on the categorization criteria, including at least one of host, source, sourcetype, or partition.
(154) It will be understood that a change to any of the filter criteria or categorization criteria can result in different groupings. As a one non-limiting example, a request received by an indexer **206** that includes the following filter criteria: partition=main, time range=3/1/17 3/1/17 16:21:20.000-16:28:17.000, and the following categorization criteria: host, source, sourcetype would result in the indexer identifying event references 1-12 as satisfying the filter criteria. The indexer would then generate up to 24 groupings corresponding to the 24 different combinations of the categorization criteria-value pairs, including host (hostA, hostB), source (sourceA, sourceB, sourceC, sourceD), and sourcetype (sourcetypeA, sourcetypeB, sourcetypeC). However, as there are only twelve events identifiers in the illustrated embodiment and some fall into the same grouping, the indexer generates eight groups and counts as follows: Group 1 (hostA, sourceA, sourcetypeA): 1 (event reference 7) Group 2 (hostA, sourceA, sourcetypeB): 2 (event references 1, 12) Group 3 (hostA, sourceA, sourcetypeC): 1 (event reference 4) Group 4 (hostA, sourceB, sourcetypeA): 1 (event reference 3) Group 5 (hostA, sourceB, sourcetypeC): 1 (event reference 9) Group 6 (hostB, sourceC, sourcetypeA): 1 (event reference 2) Group 7 (hostB, sourceC, sourcetypeC): 2 (event references 8, 11) Group 8 (hostB, sourceD, sourcetypeC): 3 (event references 5, 6, 10)

(155) As noted, each group has a unique combination of categorization criteria-value pairs or categorization criteria values. The indexer communicates the groups to the search head for aggregation with results received from other indexers. In communicating the groups to the search head, the indexer can include the categorization criteria-value pairs for each group and the count. In some embodiments, the indexer can include more or less information. For example, the indexer can include the event references associated with each group and other identifying information, such as the indexer or inverted index used to identify the groups.
(156) As another non-limiting examples, a request received by an indexer **206** that includes the following filter criteria: partition=_main, time range=3/1/17 3/1/17 16:21:20.000-16:28:17.000, source=sourceA, sourceD, and keyword=itemID and the following categorization criteria: host, source, sourcetype would result in the indexer identifying event references 4, 7, and 10 as satisfying the filter criteria, and generate the following groups: Group 1 (hostA, sourceA, sourcetypeC): 1 (event reference 4) Group 2 (hostA, sourceA, sourcetypeA): 1 (event reference 7) Group 3 (hostB, sourceD, sourcetypeC): 1 (event references 10)
(157) The indexer communicates the groups to the search head for aggregation with results received from other indexers. As will be understand there are myriad ways for filtering and categorizing the events and event references. For example, the indexer can review multiple inverted indexes associated with an partition or review the inverted indexes of multiple partitions, and categorize the data using any one or any combination of partition, host, source, sourcetype, or other category, as desired.
(158) Further, if a user interacts with a particular group, the indexer can provide additional information regarding the group. For example, the indexer can perform a targeted search or sampling of the events that satisfy the filter criteria and the categorization criteria for the selected group, also referred to as the filter criteria corresponding to the group or filter criteria associated with the group.
(159) In some cases, to provide the additional information, the indexer relies on the inverted index. For example, the indexer can identify the event references associated with the events that satisfy the filter criteria and the categorization criteria for the selected group and then use the event reference array **515** to access some or all of the identified events. In some cases, the categorization criteria values or categorization criteria-value pairs associated with the group become part of the filter criteria for the review.
(160) With reference to FIG. **5**B for instance, suppose a group is displayed with a count of six corresponding to event references 4, 5, 6, 8, 10, 11 (i.e., event references 4, 5, 6, 8, 10, 11 satisfy the filter criteria and are associated with matching categorization criteria values or categorization criteria-value pairs) and a user interacts with the group (e.g., selecting the group, clicking on the group, etc.). In response, the search head communicates with the indexer to provide additional information regarding the group.
(161) In some embodiments, the indexer identifies the event references associated with the group using the filter criteria and the categorization criteria for the group (e.g., categorization criteria values or categorization criteria-value pairs unique to the group). Together, the filter criteria and the categorization criteria for the group can be referred to as the filter criteria associated with the group. Using the filter criteria associated with the group, the indexer identifies event references 4, 5, 6, 8, 10, 11.
(162) Based on a sampling criteria, discussed in greater detail above, the indexer can determine that it will analyze a sample of the events associated with the event references 4, 5, 6, 8, 10, 11. For example, the sample can include analyzing event data associated with the event references 5, 8, 10. In some embodiments, the indexer can use the event reference array **515** to access the event data associated with the event references 5, 8, 10. Once accessed, the indexer can compile the relevant information and provide it to the search head for aggregation with results from other indexers. By identifying events and sampling event data using the inverted indexes, the indexer can reduce the

amount of actual data this is analyzed and the number of events that are accessed in order to generate the summary of the group and provide a response in less time.

2.8. Query Processing

(163) FIG. **6**A is a flow diagram of an example method that illustrates how a search head and indexers perform a search query, in accordance with example embodiments. At block **602**, a search head receives a search query from a client. At block **604**, the search head analyzes the search query to determine what portion(s) of the query can be delegated to indexers and what portions of the query can be executed locally by the search head. At block **606**, the search head distributes the determined portions of the query to the appropriate indexers. In some embodiments, a search head cluster may take the place of an independent search head where each search head in the search head cluster coordinates with peer search heads in the search head cluster to schedule jobs, replicate search results, update configurations, fulfill search requests, etc. In some embodiments, the search head (or each search head) communicates with a master node (also known as a cluster master, not shown in FIG. **2**) that provides the search head with a list of indexers to which the search head can distribute the determined portions of the query. The master node maintains a list of active indexers and can also designate which indexers may have responsibility for responding to queries over certain sets of events. A search head may communicate with the master node before the search head distributes queries to indexers to discover the addresses of active indexers.

(164) At block **608**, the indexers to which the query was distributed, search data stores associated with them for events that are responsive to the query. To determine which events are responsive to the query, the indexer searches for events that match the criteria specified in the query. These criteria can include matching keywords or specific values for certain fields. The searching operations at block **608** may use the late-binding schema to extract values for specified fields from events at the time the query is processed. In some embodiments, one or more rules for extracting field values may be specified as part of a source type definition in a configuration file. The indexers may then either send the relevant events back to the search head, or use the events to determine a partial result, and send the partial result back to the search head.

(165) At block **610**, the search head combines the partial results and/or events received from the indexers to produce a final result for the query. In some examples, the results of the query are indicative of performance or security of the IT environment and may help improve the performance of components in the IT environment. This final result may comprise different types of data depending on what the query requested. For example, the results can include a listing of matching events returned by the query, or some type of visualization of the data from the returned events. In another example, the final result can include one or more calculated values derived from the matching events.

(166) The results generated by the system **108** can be returned to a client using different techniques. For example, one technique streams results or relevant events back to a client in real-time as they are identified. Another technique waits to report the results to the client until a complete set of results (which may include a set of relevant events or a result based on relevant events) is ready to return to the client. Yet another technique streams interim results or relevant events back to the client in real-time until a complete set of results is ready, and then returns the complete set of results to the client. In another technique, certain results are stored as “search jobs” and the client may retrieve the results by referring the search jobs.

(167) The search head can also perform various operations to make the search more efficient. For example, before the search head begins execution of a query, the search head can determine a time range for the query and a set of common keywords that all matching events include. The search head may then use these parameters to query the indexers to obtain a superset of the eventual results. Then, during a filtering stage, the search head can perform field-extraction operations on the superset to produce a reduced set of search results. This speeds up queries, which may be particularly helpful for queries that are performed on a periodic basis.

2.9. Pipelined Search Language

(168) Various embodiments of the present disclosure can be implemented using, or in conjunction with, a pipelined command language. A pipelined command language is a language in which a set of inputs or data is operated on by a first command in a sequence of commands, and then subsequent commands in the order they are arranged in the sequence. Such commands can include any type of functionality for operating on data, such as retrieving, searching, filtering, aggregating, processing, transmitting, and the like. As described herein, a query can thus be formulated in a pipelined command language and include any number of ordered or unordered commands for operating on data.

(169) Splunk Processing Language (SPL) is an example of a pipelined command language in which a set of inputs or data is operated on by any number of commands in a particular sequence. A sequence of commands, or command sequence, can be formulated such that the order in which the commands are arranged defines the order in which the commands are applied to a set of data or the results of an earlier executed command. For example, a first command in a command sequence can operate to search or filter for specific data in particular set of data. The results of the first command can then be passed to another command listed later in the command sequence for further processing.

(170) In various embodiments, a query can be formulated as a command sequence defined in a command line of a search UI. In some embodiments, a query can be formulated as a sequence of SPL commands. Some or all of the SPL commands in the sequence of SPL commands can be separated from one another by a pipe symbol “I”. In such embodiments, a set of data, such as a set of events, can be operated on by a first SPL command in the sequence, and then a subsequent SPL command following a pipe symbol “I” after the first SPL command operates on the results produced by the first SPL command or other set of data, and so on for any additional SPL commands in the sequence. As such, a query formulated using SPL comprises a series of consecutive commands that are delimited by pipe “I” characters. The pipe character indicates to the system that the output or result of one command (to the left of the pipe) should be used as the input for one of the subsequent commands (to the right of the pipe). This enables formulation of queries defined by a pipeline of sequenced commands that refines or enhances the data at each step along the pipeline until the desired results are attained. Accordingly, various embodiments described herein can be implemented with Splunk Processing Language (SPL) used in conjunction with the SPLUNK® ENTERPRISE system.

(171) While a query can be formulated in many ways, a query can start with a search command and one or more corresponding search terms at the beginning of the pipeline. Such search terms can include any combination of keywords, phrases, times, dates, Boolean expressions, fieldname-field value pairs, etc. that specify which results should be obtained from an index. The results can then be passed as inputs into subsequent commands in a sequence of commands by using, for example, a pipe character. The subsequent commands in a sequence can include directives for additional processing of the results once it has been obtained from one or more indexes. For example, commands may be used to filter unwanted information out of the results, extract more information, evaluate field values, calculate statistics, reorder the results, create an alert, create summary of the results, or perform some type of aggregation function. In some embodiments, the summary can include a graph, chart, metric, or other visualization of the data. An aggregation function can include analysis or calculations to return an aggregate value, such as an average value, a sum, a maximum value, a root mean square, statistical values, and the like.

(172) Due to its flexible nature, use of a pipelined command language in various embodiments is advantageous because it can perform “filtering” as well as “processing” functions. In other words, a single query can include a search command and search term expressions, as well as data-analysis expressions. For example, a command at the beginning of a query can perform a “filtering” step by retrieving a set of data based on a condition (e.g., records associated with server response times of

less than 1 microsecond). The results of the filtering step can then be passed to a subsequent command in the pipeline that performs a "processing" step (e.g. calculation of an aggregate value related to the filtered events such as the average response time of servers with response times of less than 1 microsecond). Furthermore, the search command can allow events to be filtered by keyword as well as field value criteria. For example, a search command can filter out all events containing the word "warning" or filter out all events where a field value associated with a field "clientip" is "10.0.1.2."

(173) The results obtained or generated in response to a command in a query can be considered a set of results data. The set of results data can be passed from one command to another in any data format. In one embodiment, the set of result data can be in the form of a dynamically created table. Each command in a particular query can redefine the shape of the table. In some implementations, an event retrieved from an index in response to a query can be considered a row with a column for each field value. Columns contain basic information about the data and also may contain data that has been dynamically extracted at search time.

(174) FIG. **6**B provides a visual representation of the manner in which a pipelined command language or query operates in accordance with the disclosed embodiments. The query **630** can be inputted by the user into a search. The query comprises a search, the results of which are piped to two commands (namely, command **1** and command **2**) that follow the search step.

(175) Disk **622** represents the event data in the raw record data store.

(176) When a user query is processed, a search step will precede other queries in the pipeline in order to generate a set of events at block **640**. For example, the query can comprise search terms "sourcetype=syslog ERROR" at the front of the pipeline as shown in FIG. **6**B. Intermediate results table **624** shows fewer rows because it represents the subset of events retrieved from the index that matched the search terms "sourcetype=syslog ERROR" from search command **630**. By way of further example, instead of a search step, the set of events at the head of the pipeline may be generating by a call to a pre-existing inverted index (as will be explained later).

(177) At block **642**, the set of events generated in the first part of the query may be piped to a query that searches the set of events for field-value pairs or for keywords. For example, the second intermediate results table **626** shows fewer columns, representing the result of the top command, "top user" which summarizes the events into a list of the top 10 users and displays the user, count, and percentage.

(178) Finally, at block **644**, the results of the prior stage can be pipelined to another stage where further filtering or processing of the data can be performed, e.g., preparing the data for display purposes, filtering the data based on a condition, performing a mathematical calculation with the data, etc. As shown in FIG. **6**B, the "fields—percent" part of command **630** removes the column that shows the percentage, thereby, leaving a final results table **628** without a percentage column. In different embodiments, other query languages, such as the Structured Query Language ("SQL"), can be used to create a query.

2.10. Field Extraction

(179) The search head **210** allows users to search and visualize events generated from machine data received from homogenous data sources. The search head **210** also allows users to search and visualize events generated from machine data received from heterogeneous data sources. The search head **210** includes various mechanisms, which may additionally reside in an indexer **206**, for processing a query. A query language may be used to create a query, such as any suitable pipelined query language. For example, Splunk Processing Language (SPL) can be utilized to make a query. SPL is a pipelined search language in which a set of inputs is operated on by a first command in a command line, and then a subsequent command following the pipe symbol "I" operates on the results produced by the first command, and so on for additional commands. Other query languages, such as the Structured Query Language ("SQL"), can be used to create a query.

(180) In response to receiving the search query, search head **210** uses extraction rules to extract

values for fields in the events being searched. The search head **210** obtains extraction rules that specify how to extract a value for fields from an event. Extraction rules can comprise regex rules that specify how to extract values for the fields corresponding to the extraction rules. In addition to specifying how to extract field values, the extraction rules may also include instructions for deriving a field value by performing a function on a character string or value retrieved by the extraction rule. For example, an extraction rule may truncate a character string or convert the character string into a different data format. In some cases, the query itself can specify one or more extraction rules.

(181) The search head **210** can apply the extraction rules to events that it receives from indexers **206**. Indexers **206** may apply the extraction rules to events in an associated data store **208**. Extraction rules can be applied to all the events in a data store or to a subset of the events that have been filtered based on some criteria (e.g., event time stamp values, etc.). Extraction rules can be used to extract one or more values for a field from events by parsing the portions of machine data in the events and examining the data for one or more patterns of characters, numbers, delimiters, etc., that indicate where the field begins and, optionally, ends.

(182) FIG. **7**A is a diagram of an example scenario where a common customer identifier is found among log data received from three disparate data sources, in accordance with example embodiments. In this example, a user submits an order for merchandise using a vendor's shopping application program **701** running on the user's system. In this example, the order was not delivered to the vendor's server due to a resource exception at the destination server that is detected by the middleware code **702**. The user then sends a message to the customer support server **703** to complain about the order failing to complete. The three systems **701**, **702**, and **703** are disparate systems that do not have a common logging format. The order application **701** sends log data **704** to the data intake and query system in one format, the middleware code **702** sends error log data **705** in a second format, and the support server **703** sends log data **706** in a third format.

(183) Using the log data received at one or more indexers **206** from the three systems, the vendor can uniquely obtain an insight into user activity, user experience, and system behavior. The search head **210** allows the vendor's administrator to search the log data from the three systems that one or more indexers **206** are responsible for searching, thereby obtaining correlated information, such as the order number and corresponding customer ID number of the person placing the order. The system also allows the administrator to see a visualization of related events via a user interface. The administrator can query the search head **210** for customer ID field value matches across the log data from the three systems that are stored at the one or more indexers **206**. The customer ID field value exists in the data gathered from the three systems, but the customer ID field value may be located in different areas of the data given differences in the architecture of the systems. There is a semantic relationship between the customer ID field values generated by the three systems. The search head **210** requests events from the one or more indexers **206** to gather relevant events from the three systems. The search head **210** then applies extraction rules to the events in order to extract field values that it can correlate. The search head may apply a different extraction rule to each set of events from each system when the event format differs among systems. In this example, the user interface can display to the administrator the events corresponding to the common customer ID field values **707**, **708**, and **709**, thereby providing the administrator with insight into a customer's experience.

(184) Note that query results can be returned to a client, a search head, or any other system component for further processing. In general, query results may include a set of one or more events, a set of one or more values obtained from the events, a subset of the values, statistics calculated based on the values, a report containing the values, a visualization (e.g., a graph or chart) generated from the values, and the like.

(185) The search system enables users to run queries against the stored data to retrieve events that meet criteria specified in a query, such as containing certain keywords or having specific values in

defined fields. FIG. **7**B illustrates the manner in which keyword searches and field searches are processed in accordance with disclosed embodiments.

(186) If a user inputs a search query into search bar **710** that includes only keywords (also known as “tokens”), e.g., the keyword “error” or “warning”, the query search engine of the data intake and query system searches for those keywords directly in the event data **711** stored in the raw record data store. Note that while FIG. **7**B only illustrates four events **712**, **713**, **714**, **715**, the raw record data store (corresponding to data store **208** in FIG. **2**) may contain records for millions of events.

(187) As disclosed above, an indexer can optionally generate a keyword index to facilitate fast keyword searching for event data. The indexer includes the identified keywords in an index, which associates each stored keyword with reference pointers to events containing that keyword (or to locations within events where that keyword is located, other location identifiers, etc.). When an indexer subsequently receives a keyword-based query, the indexer can access the keyword index to quickly identify events containing the keyword. For example, if the keyword “HTTP” was indexed by the indexer at index time, and the user searches for the keyword “HTTP”, the events **712**, **713**, and **714**, will be identified based on the results returned from the keyword index. As noted above, the index contains reference pointers to the events containing the keyword, which allows for efficient retrieval of the relevant events from the raw record data store.

(188) If a user searches for a keyword that has not been indexed by the indexer, the data intake and query system would nevertheless be able to retrieve the events by searching the event data for the keyword in the raw record data store directly as shown in FIG. **7**B. For example, if a user searches for the keyword “frank”, and the name “frank” has not been indexed at index time, the data intake and query system will search the event data directly and return the first event **712**. Note that whether the keyword has been indexed at index time or not, in both cases the raw data of the events **712-715** is accessed from the raw data record store to service the keyword search. In the case where the keyword has been indexed, the index will contain a reference pointer that will allow for a more efficient retrieval of the event data from the data store. If the keyword has not been indexed, the search engine will need to search through all the records in the data store to service the search.

(189) In most cases, however, in addition to keywords, a user's search will also include fields. The term “field” refers to a location in the event data containing one or more values for a specific data item. Often, a field is a value with a fixed, delimited position on a line, or a name and value pair, where there is a single value to each field name. A field can also be multivalued, that is, it can appear more than once in an event and have a different value for each appearance, e.g., email address fields. Fields are searchable by the field name or field name-value pairs. Some examples of fields are “clientip” for IP addresses accessing a web server, or the “From” and “To” fields in email addresses.

(190) By way of further example, consider the search, “status=404”. This search query finds events with “status” fields that have a value of “404.” When the search is run, the search engine does not look for events with any other “status” value. It also does not look for events containing other fields that share “404” as a value. As a result, the search returns a set of results that are more focused than if “404” had been used in the search string as part of a keyword search. Note also that fields can appear in events as “key=value” pairs such as “user_name=Bob.” But in most cases, field values appear in fixed, delimited positions without identifying keys. For example, the data store may contain events where the “user_name” value always appears by itself after the timestamp as illustrated by the following string: “November 15 09:33:22 johnmedlock.”

(191) The data intake and query system advantageously allows for search time field extraction. In other words, fields can be extracted from the event data at search time using late-binding schema as opposed to at data ingestion time, which was a major limitation of the prior art systems.

(192) In response to receiving the search query, search head **210** uses extraction rules to extract values for the fields associated with a field or fields in the event data being searched. The search head **210** obtains extraction rules that specify how to extract a value for certain fields from an

event. Extraction rules can comprise regex rules that specify how to extract values for the relevant fields. In addition to specifying how to extract field values, the extraction rules may also include instructions for deriving a field value by performing a function on a character string or value retrieved by the extraction rule. For example, a transformation rule may truncate a character string, or convert the character string into a different data format. In some cases, the query itself can specify one or more extraction rules.

(193) FIG. **7**B illustrates the manner in which configuration files may be used to configure custom fields at search time in accordance with the disclosed embodiments. In response to receiving a search query, the data intake and query system determines if the query references a "field." For example, a query may request a list of events where the "clientip" field equals "127.0.0.1." If the query itself does not specify an extraction rule and if the field is not a metadata field, e.g., time, host, source, source type, etc., then in order to determine an extraction rule, the search engine may, in one or more embodiments, need to locate configuration file **716** during the execution of the search as shown in FIG. **7**B.

(194) Configuration file **716** may contain extraction rules for all the various fields that are not metadata fields, e.g., the "clientip" field. The extraction rules may be inserted into the configuration file in a variety of ways. In some embodiments, the extraction rules can comprise regular expression rules that are manually entered in by the user. Regular expressions match patterns of characters in text and are used for extracting custom fields in text.

(195) In one or more embodiments, as noted above, a field extractor may be configured to automatically generate extraction rules for certain field values in the events when the events are being created, indexed, or stored, or possibly at a later time. In one embodiment, a user may be able to dynamically create custom fields by highlighting portions of a sample event that should be extracted as fields using a graphical user interface. The system would then generate a regular expression that extracts those fields from similar events and store the regular expression as an extraction rule for the associated field in the configuration file **716**.

(196) In some embodiments, the indexers may automatically discover certain custom fields at index time and the regular expressions for those fields will be automatically generated at index time and stored as part of extraction rules in configuration file **716**. For example, fields that appear in the event data as "key=value" pairs may be automatically extracted as part of an automatic field discovery process. Note that there may be several other ways of adding field definitions to configuration files in addition to the methods discussed herein.

(197) The search head **210** can apply the extraction rules derived from configuration file **716** to event data that it receives from indexers **206**. Indexers **206** may apply the extraction rules from the configuration file to events in an associated data store **208**. Extraction rules can be applied to all the events in a data store, or to a subset of the events that have been filtered based on some criteria (e.g., event time stamp values, etc.). Extraction rules can be used to extract one or more values for a field from events by parsing the event data and examining the event data for one or more patterns of characters, numbers, delimiters, etc., that indicate where the field begins and, optionally, ends.

(198) In one more embodiments, the extraction rule in configuration file **716** will also need to define the type or set of events that the rule applies to. Because the raw record data store will contain events from multiple heterogeneous sources, multiple events may contain the same fields in different locations because of discrepancies in the format of the data generated by the various sources. Furthermore, certain events may not contain a particular field at all. For example, event **715** also contains "clientip" field, however, the "clientip" field is in a different format from the events **712**, **713**, and **714**. To address the discrepancies in the format and content of the different types of events, the configuration file will also need to specify the set of events that an extraction rule applies to, e.g., extraction rule **717** specifies a rule for filtering by the type of event and contains a regular expression for parsing out the field value. Accordingly, each extraction rule will pertain to only a particular type of event. If a particular field, e.g., "clientip" occurs in multiple

events, each of those types of events would need its own corresponding extraction rule in the configuration file **716** and each of the extraction rules would comprise a different regular expression to parse out the associated field value. The most common way to categorize events is by source type because events generated by a particular source can have the same format.

(199) The field extraction rules stored in configuration file **716** perform search-time field extractions. For example, for a query that requests a list of events with source type "access_combined" where the "clientip" field equals "127.0.0.1," the query search engine would first locate the configuration file **716** to retrieve extraction rule **717** that would allow it to extract values associated with the "clientip" field from the event data **720** "where the source type is "access_combined. After the "clientip" field has been extracted from all the events comprising the "clientip" field where the source type is "access_combined," the query search engine can then execute the field criteria by performing the compare operation to filter out the events where the "clientip" field equals "127.0.0.1." In the example shown in FIG. **7**B, the events **712**, **713**, and **714** would be returned in response to the user query. In this manner, the search engine can service queries containing field criteria in addition to queries containing keyword criteria (as explained above).

(200) The configuration file can be created during indexing. It may either be manually created by the user or automatically generated with certain predetermined field extraction rules. As discussed above, the events may be distributed across several indexers, wherein each indexer may be responsible for storing and searching a subset of the events contained in a corresponding data store. In a distributed indexer system, each indexer would need to maintain a local copy of the configuration file that is synchronized periodically across the various indexers.

(201) The ability to add schema to the configuration file at search time results in increased efficiency. A user can create new fields at search time and simply add field definitions to the configuration file. As a user learns more about the data in the events, the user can continue to refine the late-binding schema by adding new fields, deleting fields, or modifying the field extraction rules in the configuration file for use the next time the schema is used by the system. Because the data intake and query system maintains the underlying raw data and uses late-binding schema for searching the raw data, it enables a user to continue investigating and learn valuable insights about the raw data long after data ingestion time.

(202) The ability to add multiple field definitions to the configuration file at search time also results in increased flexibility. For example, multiple field definitions can be added to the configuration file to capture the same field across events generated by different source types. This allows the data intake and query system to search and correlate data across heterogeneous sources flexibly and efficiently.

(203) Further, by providing the field definitions for the queried fields at search time, the configuration file **716** allows the record data store to be field searchable. In other words, the raw record data store can be searched using keywords as well as fields, wherein the fields are searchable name/value pairings that distinguish one event from another and can be defined in configuration file **716** using extraction rules. In comparison to a search containing field names, a keyword search does not need the configuration file and can search the event data directly as shown in FIG. **7**B.

(204) It should also be noted that any events filtered out by performing a search-time field extraction using a configuration file can be further processed by directing the results of the filtering step to a processing step using a pipelined search language. Using the prior example, a user could pipeline the results of the compare step to an aggregate function by asking the query search engine to count the number of events where the "clientip" field equals "127.0.0.1."

2.11. Example Search Screen

(205) FIG. **8**A is an interface diagram of an example user interface for a search screen **800**, in accordance with example embodiments. Search screen **800** includes a search bar **802** that accepts

user input in the form of a search string. It also includes a time range picker **812** that enables the user to specify a time range for the search. For historical searches (e.g., searches based on a particular historical time range), the user can select a specific time range, or alternatively a relative time range, such as “today,” “yesterday” or “last week.” For real-time searches (e.g., searches whose results are based on data received in real-time), the user can select the size of a preceding time window to search for real-time events. Search screen **800** also initially displays a “data summary” dialog as is illustrated in FIG. **8**B that enables the user to select different sources for the events, such as by selecting specific hosts and log files.

(206) After the search is executed, the search screen **800** in FIG. **8**A can display the results through search results tabs **804**, wherein search results tabs **804** includes: an “events tab” that displays various information about events returned by the search; a “statistics tab” that displays statistics about the search results; and a “visualization tab” that displays various visualizations of the search results. The events tab illustrated in FIG. **8**A displays a timeline graph **805** that graphically illustrates the number of events that occurred in one-hour intervals over the selected time range. The events tab also displays an events list **808** that enables a user to view the machine data in each of the returned events.

(207) The events tab additionally displays a sidebar that is an interactive field picker **806**. The field picker **806** may be displayed to a user in response to the search being executed and allows the user to further analyze the search results based on the fields in the events of the search results. The field picker **806** includes field names that reference fields present in the events in the search results. The field picker may display any Selected Fields **820** that a user has pre-selected for display (e.g., host, source, sourcetype) and may also display any Interesting Fields **822** that the system determines may be interesting to the user based on pre-specified criteria (e.g., action, bytes, categoryid, clientip, date_hour, date_mday, date_minute, etc.). The field picker also provides an option to display field names for all the fields present in the events of the search results using the All Fields control **824**.

(208) Each field name in the field picker **806** has a value type identifier to the left of the field name, such as value type identifier **826**. A value type identifier identifies the type of value for the respective field, such as an “a” for fields that include literal values or a “#” for fields that include numerical values.

(209) Each field name in the field picker also has a unique value count to the right of the field name, such as unique value count **828**. The unique value count indicates the number of unique values for the respective field in the events of the search results.

(210) Each field name is selectable to view the events in the search results that have the field referenced by that field name. For example, a user can select the “host” field name, and the events shown in the events list **808** will be updated with events in the search results that have the field that is reference by the field name “host.”

2.12. Data Models

(211) A data model is a hierarchically structured search-time mapping of semantic knowledge about one or more datasets. It encodes the domain knowledge used to build a variety of specialized searches of those datasets. Those searches, in turn, can be used to generate reports.

(212) A data model is composed of one or more “objects” (or “data model objects”) that define or otherwise correspond to a specific set of data. An object is defined by constraints and attributes. An object's constraints are search criteria that define the set of events to be operated on by running a search having that search criteria at the time the data model is selected. An object's attributes are the set of fields to be exposed for operating on that set of events generated by the search criteria.

(213) Objects in data models can be arranged hierarchically in parent/child relationships. Each child object represents a subset of the dataset covered by its parent object. The top-level objects in data models are collectively referred to as “root objects.”

(214) Child objects have inheritance. Child objects inherit constraints and attributes from their

parent objects and may have additional constraints and attributes of their own. Child objects provide a way of filtering events from parent objects. Because a child object may provide an additional constraint in addition to the constraints it has inherited from its parent object, the dataset it represents may be a subset of the dataset that its parent represents. For example, a first data model object may define a broad set of data pertaining to e-mail activity generally, and another data model object may define specific datasets within the broad dataset, such as a subset of the e-mail data pertaining specifically to e-mails sent. For example, a user can simply select an "e-mail activity" data model object to access a dataset relating to e-mails generally (e.g., sent or received), or select an "e-mails sent" data model object (or data sub-model object) to access a dataset relating to e-mails sent.

(215) Because a data model object is defined by its constraints (e.g., a set of search criteria) and attributes (e.g., a set of fields), a data model object can be used to quickly search data to identify a set of events and to identify a set of fields to be associated with the set of events. For example, an "e-mails sent" data model object may specify a search for events relating to e-mails that have been sent, and specify a set of fields that are associated with the events. Thus, a user can retrieve and use the "e-mails sent" data model object to quickly search source data for events relating to sent e-mails, and may be provided with a listing of the set of fields relevant to the events in a user interface screen.

(216) Examples of data models can include electronic mail, authentication, databases, intrusion detection, malware, application state, alerts, compute inventory, network sessions, network traffic, performance, audits, updates, vulnerabilities, etc. Data models and their objects can be designed by knowledge managers in an organization, and they can enable downstream users to quickly focus on a specific set of data. A user iteratively applies a model development tool (not shown in FIG. **8**A) to prepare a query that defines a subset of events and assigns an object name to that subset. A child subset is created by further limiting a query that generated a parent subset.

(217) Data definitions in associated schemas can be taken from the common information model (CIM) or can be devised for a particular schema and optionally added to the CIM. Child objects inherit fields from parents and can include fields not present in parents. A model developer can select fewer extraction rules than are available for the sources returned by the query that defines events belonging to a model. Selecting a limited set of extraction rules can be a tool for simplifying and focusing the data model, while allowing a user flexibility to explore the data subset. Development of a data model is further explained in U.S. Pat. Nos. 8,788,525 and 8,788,526, both entitled "DATA MODEL FOR MACHINE DATA FOR SEMANTIC SEARCH", both issued on 22 Jul. 2014, U.S. Pat. No. 8,983,994, entitled "GENERATION OF A DATA MODEL FOR SEARCHING MACHINE DATA", issued on 17 Mar. 2015, U.S. Pat. No. 9,128,980, entitled "GENERATION OF A DATA MODEL APPLIED TO QUERIES", issued on 8 Sep. 2015, and U.S. Pat. No. 9,589,012, entitled "GENERATION OF A DATA MODEL APPLIED TO OBJECT QUERIES", issued on 7 Mar. 2017, each of which is hereby incorporated by reference in its entirety for all purposes.

(218) A data model can also include reports. One or more report formats can be associated with a particular data model and be made available to run against the data model. A user can use child objects to design reports with object datasets that already have extraneous data pre-filtered out. In some embodiments, the data intake and query system **108** provides the user with the ability to produce reports (e.g., a table, chart, visualization, etc.) without having to enter SPL, SQL, or other query language terms into a search screen. Data models are used as the basis for the search feature.

(219) Data models may be selected in a report generation interface. The report generator supports drag-and-drop organization of fields to be summarized in a report. When a model is selected, the fields with available extraction rules are made available for use in the report. The user may refine and/or filter search results to produce more precise reports. The user may select some fields for organizing the report and select other fields for providing detail according to the report

organization. For example, “region” and “salesperson” are fields used for organizing the report and sales data can be summarized (subtotaled and totaled) within this organization. The report generator allows the user to specify one or more fields within events and apply statistical analysis on values extracted from the specified one or more fields. The report generator may aggregate search results across sets of events and generate statistics based on aggregated search results. Building reports using the report generation interface is further explained in U.S. patent application Ser. No. 14/503,335, entitled “GENERATING REPORTS FROM UNSTRUCTURED DATA”, filed on 30 Sep. 2014, and which is hereby incorporated by reference in its entirety for all purposes. Data visualizations also can be generated in a variety of formats, by reference to the data model. Reports, data visualizations, and data model objects can be saved and associated with the data model for future use. The data model object may be used to perform searches of other data.

(220) FIGS. **9-15** are interface diagrams of example report generation user interfaces, in accordance with example embodiments. The report generation process may be driven by a predefined data model object, such as a data model object defined and/or saved via a reporting application or a data model object obtained from another source. A user can load a saved data model object using a report editor. For example, the initial search query and fields used to drive the report editor may be obtained from a data model object. The data model object that is used to drive a report generation process may define a search and a set of fields. Upon loading of the data model object, the report generation process may enable a user to use the fields (e.g., the fields defined by the data model object) to define criteria for a report (e.g., filters, split rows/columns, aggregates, etc.) and the search may be used to identify events (e.g., to identify events responsive to the search) used to generate the report. That is, for example, if a data model object is selected to drive a report editor, the graphical user interface of the report editor may enable a user to define reporting criteria for the report using the fields associated with the selected data model object, and the events used to generate the report may be constrained to the events that match, or otherwise satisfy, the search constraints of the selected data model object.

(221) The selection of a data model object for use in driving a report generation may be facilitated by a data model object selection interface. FIG. **9** illustrates an example interactive data model selection graphical user interface **900** of a report editor that displays a listing of available data models **901**. The user may select one of the data models **902**.

(222) FIG. **10** illustrates an example data model object selection graphical user interface **1000** that displays available data objects **1001** for the selected data object model **902**. The user may select one of the displayed data model objects **1002** for use in driving the report generation process.

(223) Once a data model object is selected by the user, a user interface screen **1100** shown in FIG. **11**A may display an interactive listing of automatic field identification options **1101** based on the selected data model object. For example, a user may select one of the three illustrated options (e.g., the “All Fields” option **1102**, the “Selected Fields” option **1103**, or the “Coverage” option (e.g., fields with at least a specified % of coverage) **1104**). If the user selects the “All Fields” option **1102**, all of the fields identified from the events that were returned in response to an initial search query may be selected. That is, for example, all of the fields of the identified data model object fields may be selected. If the user selects the “Selected Fields” option **1103**, only the fields from the fields of the identified data model object fields that are selected by the user may be used. If the user selects the “Coverage” option **1104**, only the fields of the identified data model object fields meeting a specified coverage criteria may be selected. A percent coverage may refer to the percentage of events returned by the initial search query that a given field appears in. Thus, for example, if an object dataset includes 10,000 events returned in response to an initial search query, and the “avg_age” field appears in **854** of those 10,000 events, then the “avg_age” field would have a coverage of 8.54% for that object dataset. If, for example, the user selects the “Coverage” option and specifies a coverage value of 2%, only fields having a coverage value equal to or greater than 2% may be selected. The number of fields corresponding to each selectable option may be

displayed in association with each option. For example, “97” displayed next to the “All Fields” option **1102** indicates that 97 fields will be selected if the “All Fields” option is selected. The “3” displayed next to the “Selected Fields” option **1103** indicates that 3 of the 97 fields will be selected if the “Selected Fields” option is selected. The “49” displayed next to the “Coverage” option **1104** indicates that 49 of the 97 fields (e.g., the 49 fields having a coverage of 2% or greater) will be selected if the “Coverage” option is selected. The number of fields corresponding to the “Coverage” option may be dynamically updated based on the specified percent of coverage.

(224) FIG. **11**B illustrates an example graphical user interface screen **1105** displaying the reporting application's “Report Editor” page. The screen may display interactive elements for defining various elements of a report. For example, the page includes a “Filters” element **1106**, a “Split Rows” element **1107**, a “Split Columns” element **1108**, and a “Column Values” element **1109**. The page may include a list of search results **1111**. In this example, the Split Rows element **1107** is expanded, revealing a listing of fields **1110** that can be used to define additional criteria (e.g., reporting criteria). The listing of fields **1110** may correspond to the selected fields. That is, the listing of fields **1110** may list only the fields previously selected, either automatically and/or manually by a user. FIG. **11**C illustrates a formatting dialogue **1112** that may be displayed upon selecting a field from the listing of fields **1110**. The dialogue can be used to format the display of the results of the selection (e.g., label the column for the selected field to be displayed as “component”).

(225) FIG. **11**D illustrates an example graphical user interface screen **1105** including a table of results **1113** based on the selected criteria including splitting the rows by the “component” field. A column **1114** having an associated count for each component listed in the table may be displayed that indicates an aggregate count of the number of times that the particular field-value pair (e.g., the value in a row for a particular field, such as the value “BucketMover” for the field “component”) occurs in the set of events responsive to the initial search query.

(226) FIG. **12** illustrates an example graphical user interface screen **1200** that allows the user to filter search results and to perform statistical analysis on values extracted from specific fields in the set of events. In this example, the top ten product names ranked by price are selected as a filter **1201** that causes the display of the ten most popular products sorted by price. Each row is displayed by product name and price **1202**. This results in each product displayed in a column labeled “product name” along with an associated price in a column labeled “price” **1206**. Statistical analysis of other fields in the events associated with the ten most popular products have been specified as column values **1203**. A count of the number of successful purchases for each product is displayed in column **1204**. These statistics may be produced by filtering the search results by the product name, finding all occurrences of a successful purchase in a field within the events and generating a total of the number of occurrences. A sum of the total sales is displayed in column **1205**, which is a result of the multiplication of the price and the number of successful purchases for each product.

(227) The reporting application allows the user to create graphical visualizations of the statistics generated for a report. For example, FIG. **13** illustrates an example graphical user interface **1300** that displays a set of components and associated statistics **1301**. The reporting application allows the user to select a visualization of the statistics in a graph (e.g., bar chart, scatter plot, area chart, line chart, pie chart, radial gauge, marker gauge, filler gauge, etc.), where the format of the graph may be selected using the user interface controls **1302** along the left panel of the user interface **1300**. FIG. **14** illustrates an example of a bar chart visualization **1400** of an aspect of the statistical data **1301**. FIG. **15** illustrates a scatter plot visualization **1500** of an aspect of the statistical data **1301**.

2.13. Acceleration Technique

(228) The above-described system provides significant flexibility by enabling a user to analyze massive quantities of minimally-processed data “on the fly” at search time using a late-binding

schema, instead of storing pre-specified portions of the data in a database at ingestion time. This flexibility enables a user to see valuable insights, correlate data, and perform subsequent queries to examine interesting aspects of the data that may not have been apparent at ingestion time.
(229) However, performing extraction and analysis operations at search time can involve a large amount of data and require a large number of computational operations, which can cause delays in processing the queries. Advantageously, the data intake and query system also employs a number of unique acceleration techniques that have been developed to speed up analysis operations performed at search time. These techniques include: (1) performing search operations in parallel across multiple indexers; (2) using a keyword index; (3) using a high performance analytics store; and (4) accelerating the process of generating reports. These novel techniques are described in more detail below.

2.13.1. Aggregation Technique

(230) To facilitate faster query processing, a query can be structured such that multiple indexers perform the query in parallel, while aggregation of search results from the multiple indexers is performed locally at the search head. For example, FIG. **16** is an example search query received from a client and executed by search peers, in accordance with example embodiments. FIG. **16** illustrates how a search query **1602** received from a client at a search head **210** can split into two phases, including: (1) subtasks **1604** (e.g., data retrieval or simple filtering) that may be performed in parallel by indexers **206** for execution, and (2) a search results aggregation operation **1606** to be executed by the search head when the results are ultimately collected from the indexers.
(231) During operation, upon receiving search query **1602**, a search head **210** determines that a portion of the operations involved with the search query may be performed locally by the search head. The search head modifies search query **1602** by substituting “stats” (create aggregate statistics over results sets received from the indexers at the search head) with “prestats” (create statistics by the indexer from local results set) to produce search query **1604**, and then distributes search query **1604** to distributed indexers, which are also referred to as “search peers” or “peer indexers.” Note that search queries may generally specify search criteria or operations to be performed on events that meet the search criteria. Search queries may also specify field names, as well as search criteria for the values in the fields or operations to be performed on the values in the fields. Moreover, the search head may distribute the full search query to the search peers as illustrated in FIG. **6**A, or may alternatively distribute a modified version (e.g., a more restricted version) of the search query to the search peers. In this example, the indexers are responsible for producing the results and sending them to the search head. After the indexers return the results to the search head, the search head aggregates the received results **1606** to form a single search result set. By executing the query in this manner, the system effectively distributes the computational operations across the indexers while minimizing data transfers.

2.13.2. Keyword Index

(232) As described above with reference to the flow charts in FIG. **5**A and FIG. **6**A, data intake and query system **108** can construct and maintain one or more keyword indices to quickly identify events containing specific keywords. This technique can greatly speed up the processing of queries involving specific keywords. As mentioned above, to build a keyword index, an indexer first identifies a set of keywords. Then, the indexer includes the identified keywords in an index, which associates each stored keyword with references to events containing that keyword, or to locations within events where that keyword is located. When an indexer subsequently receives a keyword-based query, the indexer can access the keyword index to quickly identify events containing the keyword.

2.13.3. High Performance Analytics Store

(233) To speed up certain types of queries, some embodiments of system **108** create a high performance analytics store, which is referred to as a “summarization table,” that contains entries for specific field-value pairs. Each of these entries keeps track of instances of a specific value in a

specific field in the events and includes references to events containing the specific value in the specific field. For example, an example entry in a summarization table can keep track of occurrences of the value “94107” in a “ZIP code” field of a set of events and the entry includes references to all of the events that contain the value “94107” in the ZIP code field. This optimization technique enables the system to quickly process queries that seek to determine how many events have a particular value for a particular field. To this end, the system can examine the entry in the summarization table to count instances of the specific value in the field without having to go through the individual events or perform data extractions at search time. Also, if the system needs to process all events that have a specific field-value combination, the system can use the references in the summarization table entry to directly access the events to extract further information without having to search all of the events to find the specific field-value combination at search time.

(234) In some embodiments, the system maintains a separate summarization table for each of the above-described time-specific buckets that stores events for a specific time range. A bucket-specific summarization table includes entries for specific field-value combinations that occur in events in the specific bucket. Alternatively, the system can maintain a separate summarization table for each indexer. The indexer-specific summarization table includes entries for the events in a data store that are managed by the specific indexer. Indexer-specific summarization tables may also be bucket-specific.

(235) The summarization table can be populated by running a periodic query that scans a set of events to find instances of a specific field-value combination, or alternatively instances of all field-value combinations for a specific field. A periodic query can be initiated by a user, or can be scheduled to occur automatically at specific time intervals. A periodic query can also be automatically launched in response to a query that asks for a specific field-value combination.

(236) In some cases, when the summarization tables may not cover all of the events that are relevant to a query, the system can use the summarization tables to obtain partial results for the events that are covered by summarization tables, but may also have to search through other events that are not covered by the summarization tables to produce additional results. These additional results can then be combined with the partial results to produce a final set of results for the query. The summarization table and associated techniques are described in more detail in U.S. Pat. No. 8,682,925, entitled “DISTRIBUTED HIGH PERFORMANCE ANALYTICS STORE”, issued on 25 Mar. 2014, U.S. Pat. No. 9,128,985, entitled “SUPPLEMENTING A HIGH PERFORMANCE ANALYTICS STORE WITH EVALUATION OF INDIVIDUAL EVENTS TO RESPOND TO AN EVENT QUERY”, issued on 8 Sep. 2015, and U.S. patent application Ser. No. 14/815,973, entitled “GENERATING AND STORING SUMMARIZATION TABLES FOR SETS OF SEARCHABLE EVENTS”, filed on 1 Aug. 2015, each of which is hereby incorporated by reference in its entirety for all purposes.

(237) To speed up certain types of queries, e.g., frequently encountered queries or computationally intensive queries, some embodiments of system **108** create a high performance analytics store, which is referred to as a “summarization table,” (also referred to as a “lexicon” or “inverted index”) that contains entries for specific field-value pairs. Each of these entries keeps track of instances of a specific value in a specific field in the event data and includes references to events containing the specific value in the specific field. For example, an example entry in an inverted index can keep track of occurrences of the value “94107” in a “ZIP code” field of a set of events and the entry includes references to all of the events that contain the value “94107” in the ZIP code field. Creating the inverted index data structure avoids needing to incur the computational overhead each time a statistical query needs to be run on a frequently encountered field-value pair. In order to expedite queries, in most embodiments, the search engine will employ the inverted index separate from the raw record data store to generate responses to the received queries.

(238) Note that the term “summarization table” or “inverted index” as used herein is a data

structure that may be generated by an indexer that includes at least field names and field values that have been extracted and/or indexed from event records. An inverted index may also include reference values that point to the location(s) in the field searchable data store where the event records that include the field may be found. Also, an inverted index may be stored using well-known compression techniques to reduce its storage size.

(239) Further, note that the term “reference value” (also referred to as a “posting value”) as used herein is a value that references the location of a source record in the field searchable data store. In some embodiments, the reference value may include additional information about each record, such as timestamps, record size, meta-data, or the like. Each reference value may be a unique identifier which may be used to access the event data directly in the field searchable data store. In some embodiments, the reference values may be ordered based on each event record's timestamp. For example, if numbers are used as identifiers, they may be sorted so event records having a later timestamp always have a lower valued identifier than event records with an earlier timestamp, or vice-versa. Reference values are often included in inverted indexes for retrieving and/or identifying event records.

(240) In one or more embodiments, an inverted index is generated in response to a user-initiated collection query. The term “collection query” as used herein refers to queries that include commands that generate summarization information and inverted indexes (or summarization tables) from event records stored in the field searchable data store.

(241) Note that a collection query is a special type of query that can be user-generated and is used to create an inverted index. A collection query is not the same as a query that is used to call up or invoke a pre-existing inverted index. In one or more embodiments, a query can comprise an initial step that calls up a pre-generated inverted index on which further filtering and processing can be performed. For example, referring back to FIG. **6**B, a set of events can be generated at block **640** by either using a “collection” query to create a new inverted index or by calling up a pre-generated inverted index. A query with several pipelined steps will start with a pre-generated index to accelerate the query.

(242) FIG. **7**C illustrates the manner in which an inverted index is created and used in accordance with the disclosed embodiments. As shown in FIG. **7**C, an inverted index **722** can be created in response to a user-initiated collection query using the event data **723** stored in the raw record data store. For example, a non-limiting example of a collection query may include “collect clientip=127.0.0.1” which may result in an inverted index **722** being generated from the event data **723** as shown in FIG. **7**C. Each entry in inverted index **722** includes an event reference value that references the location of a source record in the field searchable data store. The reference value may be used to access the original event record directly from the field searchable data store.

(243) In one or more embodiments, if one or more of the queries is a collection query, the responsive indexers may generate summarization information based on the fields of the event records located in the field searchable data store. In at least one of the various embodiments, one or more of the fields used in the summarization information may be listed in the collection query and/or they may be determined based on terms included in the collection query. For example, a collection query may include an explicit list of fields to summarize. Or, in at least one of the various embodiments, a collection query may include terms or expressions that explicitly define the fields, e.g., using regex rules. In FIG. **7**C, prior to running the collection query that generates the inverted index **722**, the field name “clientip” may need to be defined in a configuration file by specifying the “access_combined” source type and a regular expression rule to parse out the client IP address. Alternatively, the collection query may contain an explicit definition for the field name “clientip” which may obviate the need to reference the configuration file at search time.

(244) In one or more embodiments, collection queries may be saved and scheduled to run periodically. These scheduled collection queries may periodically update the summarization information corresponding to the query. For example, if the collection query that generates inverted

index **722** is scheduled to run periodically, one or more indexers would periodically search through the relevant buckets to update inverted index **722** with event data for any new events with the "clientip" value of "127.0.0.1."

(245) In some embodiments, the inverted indexes that include fields, values, and reference value (e.g., inverted index **722**) for event records may be included in the summarization information provided to the user. In other embodiments, a user may not be interested in specific fields and values contained in the inverted index, but may need to perform a statistical query on the data in the inverted index. For example, referencing the example of FIG. **7**C rather than viewing the fields within summarization table **722**, a user may want to generate a count of all client requests from IP address "127.0.0.1." In this case, the search engine would simply return a result of "4" rather than including details about the inverted index **722** in the information provided to the user.

(246) The pipelined search language, e.g., SPL of the SPLUNK® ENTERPRISE system can be used to pipe the contents of an inverted index to a statistical query using the "stats" command for example. A "stats" query refers to queries that generate result sets that may produce aggregate and statistical results from event records, e.g., average, mean, max, min, rms, etc. Where sufficient information is available in an inverted index, a "stats" query may generate their result sets rapidly from the summarization information available in the inverted index rather than directly scanning event records. For example, the contents of inverted index **722** can be pipelined to a stats query, e.g., a "count" function that counts the number of entries in the inverted index and returns a value of "4." In this way, inverted indexes may enable various stats queries to be performed absent scanning or search the event records. Accordingly, this optimization technique enables the system to quickly process queries that seek to determine how many events have a particular value for a particular field. To this end, the system can examine the entry in the inverted index to count instances of the specific value in the field without having to go through the individual events or perform data extractions at search time.

(247) In some embodiments, the system maintains a separate inverted index for each of the above-described time-specific buckets that stores events for a specific time range. A bucket-specific inverted index includes entries for specific field-value combinations that occur in events in the specific bucket. Alternatively, the system can maintain a separate inverted index for each indexer. The indexer-specific inverted index includes entries for the events in a data store that are managed by the specific indexer. Indexer-specific inverted indexes may also be bucket-specific. In at least one or more embodiments, if one or more of the queries is a stats query, each indexer may generate a partial result set from previously generated summarization information. The partial result sets may be returned to the search head that received the query and combined into a single result set for the query

(248) As mentioned above, the inverted index can be populated by running a periodic query that scans a set of events to find instances of a specific field-value combination, or alternatively instances of all field-value combinations for a specific field. A periodic query can be initiated by a user, or can be scheduled to occur automatically at specific time intervals. A periodic query can also be automatically launched in response to a query that asks for a specific field-value combination. In some embodiments, if summarization information is absent from an indexer that includes responsive event records, further actions may be taken, such as, the summarization information may generated on the fly, warnings may be provided the user, the collection query operation may be halted, the absence of summarization information may be ignored, or the like, or combination thereof.

(249) In one or more embodiments, an inverted index may be set up to update continually. For example, the query may ask for the inverted index to update its result periodically, e.g., every hour. In such instances, the inverted index may be a dynamic data structure that is regularly updated to include information regarding incoming events.

(250) In some cases, e.g., where a query is executed before an inverted index updates, when the

inverted index may not cover all of the events that are relevant to a query, the system can use the inverted index to obtain partial results for the events that are covered by inverted index, but may also have to search through other events that are not covered by the inverted index to produce additional results on the fly. In other words, an indexer would need to search through event data on the data store to supplement the partial results. These additional results can then be combined with the partial results to produce a final set of results for the query. Note that in typical instances where an inverted index is not completely up to date, the number of events that an indexer would need to search through to supplement the results from the inverted index would be relatively small. In other words, the search to get the most recent results can be quick and efficient because only a small number of event records will be searched through to supplement the information from the inverted index. The inverted index and associated techniques are described in more detail in U.S. Pat. No. 8,682,925, entitled "DISTRIBUTED HIGH PERFORMANCE ANALYTICS STORE", issued on 25 Mar. 2014, U.S. Pat. No. 9,128,985, entitled "SUPPLEMENTING A HIGH PERFORMANCE ANALYTICS STORE WITH EVALUATION OF INDIVIDUAL EVENTS TO RESPOND TO AN EVENT QUERY", filed on 31 Jan. 2014, and U.S. patent application Ser. No. 14/815,973, entitled "STORAGE MEDIUM AND CONTROL DEVICE", filed on 21 Feb. 2014, each of which is hereby incorporated by reference in its entirety.

2.13.3.1 Extracting Event Data Using Posting

(251) In one or more embodiments, if the system needs to process all events that have a specific field-value combination, the system can use the references in the inverted index entry to directly access the events to extract further information without having to search all of the events to find the specific field-value combination at search time. In other words, the system can use the reference values to locate the associated event data in the field searchable data store and extract further information from those events, e.g., extract further field values from the events for purposes of filtering or processing or both.

(252) The information extracted from the event data using the reference values can be directed for further filtering or processing in a query using the pipeline search language. The pipelined search language will, in one embodiment, include syntax that can direct the initial filtering step in a query to an inverted index. In one embodiment, a user would include syntax in the query that explicitly directs the initial searching or filtering step to the inverted index.

(253) Referencing the example in FIG. **15**, if the user determines that she needs the user id fields associated with the client requests from IP address "127.0.0.1," instead of incurring the computational overhead of performing a brand new search or re-generating the inverted index with an additional field, the user can generate a query that explicitly directs or pipes the contents of the already generated inverted index **722** to another filtering step requesting the user ids for the entries in inverted index **722** where the server response time is greater than "0.0900" microseconds. The search engine would use the reference values stored in inverted index **722** to retrieve the event data from the field searchable data store, filter the results based on the "response time" field values and, further, extract the user id field from the resulting event data to return to the user. In the present instance, the user ids "frank" and "carlos" would be returned to the user from the generated results table **722**.

(254) In one embodiment, the same methodology can be used to pipe the contents of the inverted index to a processing step. In other words, the user is able to use the inverted index to efficiently and quickly perform aggregate functions on field values that were not part of the initially generated inverted index. For example, a user may want to determine an average object size (size of the requested gif) requested by clients from IP address "127.0.0.1." In this case, the search engine would again use the reference values stored in inverted index **722** to retrieve the event data from the field searchable data store and, further, extract the object size field values from the associated events **731**, **732**, **733** and **734**. Once, the corresponding object sizes have been extracted (i.e. **2326**, **2900**, **2920**, and **5000**), the average can be computed and returned to the user.

(255) In one embodiment, instead of explicitly invoking the inverted index in a user-generated query, e.g., by the use of special commands or syntax, the SPLUNK® ENTERPRISE system can be configured to automatically determine if any prior-generated inverted index can be used to expedite a user query. For example, the user's query may request the average object size (size of the requested gif) requested by clients from IP address “127.0.0.1.” without any reference to or use of inverted index **722**. The search engine, in this case, would automatically determine that an inverted index **722** already exists in the system that could expedite this query. In one embodiment, prior to running any search comprising a field-value pair, for example, a search engine may search though all the existing inverted indexes to determine if a pre-generated inverted index could be used to expedite the search comprising the field-value pair. Accordingly, the search engine would automatically use the pre-generated inverted index, e.g., index **722** to generate the results without any user-involvement that directs the use of the index.
(256) Using the reference values in an inverted index to be able to directly access the event data in the field searchable data store and extract further information from the associated event data for further filtering and processing is highly advantageous because it avoids incurring the computation overhead of regenerating the inverted index with additional fields or performing a new search.
(257) The data intake and query system includes one or more forwarders that receive raw machine data from a variety of input data sources, and one or more indexers that process and store the data in one or more data stores. By distributing events among the indexers and data stores, the indexers can analyze events for a query in parallel. In one or more embodiments, a multiple indexer implementation of the search system would maintain a separate and respective inverted index for each of the above-described time-specific buckets that stores events for a specific time range. A bucket-specific inverted index includes entries for specific field-value combinations that occur in events in the specific bucket. As explained above, a search head would be able to correlate and synthesize data from across the various buckets and indexers.
(258) This feature advantageously expedites searches because instead of performing a computationally intensive search in a centrally located inverted index that catalogues all the relevant events, an indexer is able to directly search an inverted index stored in a bucket associated with the time-range specified in the query. This allows the search to be performed in parallel across the various indexers. Further, if the query requests further filtering or processing to be conducted on the event data referenced by the locally stored bucket-specific inverted index, the indexer is able to simply access the event records stored in the associated bucket for further filtering and processing instead of needing to access a central repository of event records, which would dramatically add to the computational overhead.
(259) In one embodiment, there may be multiple buckets associated with the time-range specified in a query. If the query is directed to an inverted index, or if the search engine automatically determines that using an inverted index would expedite the processing of the query, the indexers will search through each of the inverted indexes associated with the buckets for the specified time-range. This feature allows the High Performance Analytics Store to be scaled easily.
(260) In certain instances, where a query is executed before a bucket-specific inverted index updates, when the bucket-specific inverted index may not cover all of the events that are relevant to a query, the system can use the bucket-specific inverted index to obtain partial results for the events that are covered by bucket-specific inverted index, but may also have to search through the event data in the bucket associated with the bucket-specific inverted index to produce additional results on the fly. In other words, an indexer would need to search through event data stored in the bucket (that was not yet processed by the indexer for the corresponding inverted index) to supplement the partial results from the bucket-specific inverted index.
(261) FIG. **7**D presents a flowchart illustrating how an inverted index in a pipelined search query can be used to determine a set of event data that can be further limited by filtering or processing in accordance with the disclosed embodiments.

(262) At block **742**, a query is received by a data intake and query system. In some embodiments, the query can be receive as a user generated query entered into search bar of a graphical user search interface. The search interface also includes a time range control element that enables specification of a time range for the query.
(263) At block **744**, an inverted index is retrieved. Note, that the inverted index can be retrieved in response to an explicit user search command inputted as part of the user generated query. Alternatively, the search engine can be configured to automatically use an inverted index if it determines that using the inverted index would expedite the servicing of the user generated query. Each of the entries in an inverted index keeps track of instances of a specific value in a specific field in the event data and includes references to events containing the specific value in the specific field. In order to expedite queries, in most embodiments, the search engine will employ the inverted index separate from the raw record data store to generate responses to the received queries.
(264) At block **746**, the query engine determines if the query contains further filtering and processing steps. If the query contains no further commands, then, in one embodiment, summarization information can be provided to the user at block **754**.
(265) If, however, the query does contain further filtering and processing commands, then at block **750**, the query engine determines if the commands relate to further filtering or processing of the data extracted as part of the inverted index or whether the commands are directed to using the inverted index as an initial filtering step to further filter and process event data referenced by the entries in the inverted index. If the query can be completed using data already in the generated inverted index, then the further filtering or processing steps, e.g., a “count” number of records function, “average” number of records per hour etc. are performed and the results are provided to the user at block **752**.
(266) If, however, the query references fields that are not extracted in the inverted index, then the indexers will access event data pointed to by the reference values in the inverted index to retrieve any further information required at block **756**. Subsequently, any further filtering or processing steps are performed on the fields extracted directly from the event data and the results are provided to the user at step **758**.
2.13.4. Accelerating Report Generation
(267) In some embodiments, a data server system such as the data intake and query system can accelerate the process of periodically generating updated reports based on query results. To accelerate this process, a summarization engine automatically examines the query to determine whether generation of updated reports can be accelerated by creating intermediate summaries. If reports can be accelerated, the summarization engine periodically generates a summary covering data obtained during a latest non-overlapping time period. For example, where the query seeks events meeting a specified criteria, a summary for the time period includes only events within the time period that meet the specified criteria. Similarly, if the query seeks statistics calculated from the events, such as the number of events that match the specified criteria, then the summary for the time period includes the number of events in the period that match the specified criteria.
(268) In addition to the creation of the summaries, the summarization engine schedules the periodic updating of the report associated with the query. During each scheduled report update, the query engine determines whether intermediate summaries have been generated covering portions of the time period covered by the report update. If so, then the report is generated based on the information contained in the summaries. Also, if additional event data has been received and has not yet been summarized, and is required to generate the complete report, the query can be run on these additional events. Then, the results returned by this query on the additional events, along with the partial results obtained from the intermediate summaries, can be combined to generate the updated report. This process is repeated each time the report is updated. Alternatively, if the system stores events in buckets covering specific time ranges, then the summaries can be generated on a bucket-by-bucket basis. Note that producing intermediate summaries can save the work involved in

re-running the query for previous time periods, so advantageously only the newer events needs to be processed while generating an updated report. These report acceleration techniques are described in more detail in U.S. Pat. No. 8,589,403, entitled “COMPRESSED JOURNALING IN EVENT TRACKING FILES FOR METADATA RECOVERY AND REPLICATION”, issued on 19 Nov. 2013, U.S. Pat. No. 8,412,696, entitled “REAL TIME SEARCHING AND REPORTING”, issued on 2 Apr. 2011, and U.S. Pat. Nos. 8,589,375 and 8,589,432, both also entitled “REAL TIME SEARCHING AND REPORTING”, both issued on 19 Nov. 2013, each of which is hereby incorporated by reference in its entirety for all purposes.

2.14. Security Features

(269) The data intake and query system provides various schemas, dashboards, and visualizations that simplify developers' tasks to create applications with additional capabilities. One such application is the an enterprise security application, such as SPLUNK® ENTERPRISE SECURITY, which performs monitoring and alerting operations and includes analytics to facilitate identifying both known and unknown security threats based on large volumes of data stored by the data intake and query system. The enterprise security application provides the security practitioner with visibility into security-relevant threats found in the enterprise infrastructure by capturing, monitoring, and reporting on data from enterprise security devices, systems, and applications. Through the use of the data intake and query system searching and reporting capabilities, the enterprise security application provides a top-down and bottom-up view of an organization's security posture.

(270) The enterprise security application leverages the data intake and query system search-time normalization techniques, saved searches, and correlation searches to provide visibility into security-relevant threats and activity and generate notable events for tracking. The enterprise security application enables the security practitioner to investigate and explore the data to find new or unknown threats that do not follow signature-based patterns.

(271) Conventional Security Information and Event Management (SIEM) systems lack the infrastructure to effectively store and analyze large volumes of security-related data. Traditional SIEM systems typically use fixed schemas to extract data from pre-defined security-related fields at data ingestion time and store the extracted data in a relational database. This traditional data extraction process (and associated reduction in data size) that occurs at data ingestion time inevitably hampers future incident investigations that may need original data to determine the root cause of a security issue, or to detect the onset of an impending security threat.

(272) In contrast, the enterprise security application system stores large volumes of minimally-processed security-related data at ingestion time for later retrieval and analysis at search time when a live security threat is being investigated. To facilitate this data retrieval process, the enterprise security application provides pre-specified schemas for extracting relevant values from the different types of security-related events and enables a user to define such schemas.

(273) The enterprise security application can process many types of security-related information. In general, this security-related information can include any information that can be used to identify security threats. For example, the security-related information can include network-related information, such as IP addresses, domain names, asset identifiers, network traffic volume, uniform resource locator strings, and source addresses. The process of detecting security threats for network-related information is further described in U.S. Pat. No. 8,826,434, entitled “SECURITY THREAT DETECTION BASED ON INDICATIONS IN BIG DATA OF ACCESS TO NEWLY REGISTERED DOMAINS”, issued on 2 Sep. 2014, U.S. Pat. No. 9,215,240, entitled “INVESTIGATIVE AND DYNAMIC DETECTION OF POTENTIAL SECURITY-THREAT INDICATORS FROM EVENTS IN BIG DATA”, issued on 15 Dec. 2015, U.S. Pat. No. 9,173,801, entitled “GRAPHIC DISPLAY OF SECURITY THREATS BASED ON INDICATIONS OF ACCESS TO NEWLY REGISTERED DOMAINS”, issued on 3 Nov. 2015, U.S. Pat. No. 9,248,068, entitled “SECURITY THREAT DETECTION OF NEWLY REGISTERED

DOMAINS", issued on 2 Feb. 2016, U.S. Pat. No. 9,426,172, entitled "SECURITY THREAT DETECTION USING DOMAIN NAME ACCESSES", issued on 23 Aug. 2016, and U.S. Pat. No. 9,432,396, entitled "SECURITY THREAT DETECTION USING DOMAIN NAME REGISTRATIONS", issued on 30 Aug. 2016, each of which is hereby incorporated by reference in its entirety for all purposes. Security-related information can also include malware infection data and system configuration information, as well as access control information, such as login/logout information and access failure notifications. The security-related information can originate from various sources within a data center, such as hosts, virtual machines, storage devices and sensors. The security-related information can also originate from various sources in a network, such as routers, switches, email servers, proxy servers, gateways, firewalls and intrusion-detection systems.

(274) During operation, the enterprise security application facilitates detecting "notable events" that are likely to indicate a security threat. A notable event represents one or more anomalous incidents, the occurrence of which can be identified based on one or more events (e.g., time stamped portions of raw machine data) fulfilling pre-specified and/or dynamically-determined (e.g., based on machine-learning) criteria defined for that notable event. Examples of notable events include the repeated occurrence of an abnormal spike in network usage over a period of time, a single occurrence of unauthorized access to system, a host communicating with a server on a known threat list, and the like. These notable events can be detected in a number of ways, such as: (1) a user can notice a correlation in events and can manually identify that a corresponding group of one or more events amounts to a notable event; or (2) a user can define a "correlation search" specifying criteria for a notable event, and every time one or more events satisfy the criteria, the application can indicate that the one or more events correspond to a notable event; and the like. A user can alternatively select a pre-defined correlation search provided by the application. Note that correlation searches can be run continuously or at regular intervals (e.g., every hour) to search for notable events. Upon detection, notable events can be stored in a dedicated "notable events index," which can be subsequently accessed to generate various visualizations containing security-related information. Also, alerts can be generated to notify system operators when important notable events are discovered.

(275) The enterprise security application provides various visualizations to aid in discovering security threats, such as a "key indicators view" that enables a user to view security metrics, such as counts of different types of notable events. For example, FIG. **17**A illustrates an example key indicators view **1700** that comprises a dashboard, which can display a value **1701**, for various security-related metrics, such as malware infections **1702**. It can also display a change in a metric value **1703**, which indicates that the number of malware infections increased by 63 during the preceding interval. Key indicators view **1700** additionally displays a histogram panel **1704** that displays a histogram of notable events organized by urgency values, and a histogram of notable events organized by time intervals. This key indicators view is described in further detail in pending U.S. patent application Ser. No. 13/956,338, entitled "KEY INDICATORS VIEW", filed on 31 Jul. 2013, and which is hereby incorporated by reference in its entirety for all purposes.

(276) These visualizations can also include an "incident review dashboard" that enables a user to view and act on "notable events." These notable events can include: (1) a single event of high importance, such as any activity from a known web attacker; or (2) multiple events that collectively warrant review, such as a large number of authentication failures on a host followed by a successful authentication. For example, FIG. **17**B illustrates an example incident review dashboard **1710** that includes a set of incident attribute fields **1711** that, for example, enables a user to specify a time range field **1712** for the displayed events. It also includes a timeline **1713** that graphically illustrates the number of incidents that occurred in time intervals over the selected time range. It additionally displays an events list **1714** that enables a user to view a list of all of the notable events that match the criteria in the incident attributes fields **1711**. To facilitate identifying patterns among the notable events, each notable event can be associated with an urgency value (e.g., low, medium,

high, critical), which is indicated in the incident review dashboard. The urgency value for a detected event can be determined based on the severity of the event and the priority of the system component associated with the event.

2.15. Data Center Monitoring

(277) As mentioned above, the data intake and query platform provides various features that simplify the developer's task to create various applications. One such application is a virtual machine monitoring application, such as SPLUNK® APP FOR VMWARE® that provides operational visibility into granular performance metrics, logs, tasks and events, and topology from hosts, virtual machines and virtual centers. It empowers administrators with an accurate real-time picture of the health of the environment, proactively identifying performance and capacity bottlenecks.

(278) Conventional data-center-monitoring systems lack the infrastructure to effectively store and analyze large volumes of machine-generated data, such as performance information and log data obtained from the data center. In conventional data-center-monitoring systems, machine-generated data is typically pre-processed prior to being stored, for example, by extracting pre-specified data items and storing them in a database to facilitate subsequent retrieval and analysis at search time. However, the rest of the data is not saved and discarded during pre-processing.

(279) In contrast, the virtual machine monitoring application stores large volumes of minimally processed machine data, such as performance information and log data, at ingestion time for later retrieval and analysis at search time when a live performance issue is being investigated. In addition to data obtained from various log files, this performance-related information can include values for performance metrics obtained through an application programming interface (API) provided as part of the vSphere Hypervisor™ system distributed by VMware, Inc. of Palo Alto, California. For example, these performance metrics can include: (1) CPU-related performance metrics; (2) disk-related performance metrics; (3) memory-related performance metrics; (4) network-related performance metrics; (5) energy-usage statistics; (6) data-traffic-related performance metrics; (7) overall system availability performance metrics; (8) cluster-related performance metrics; and (9) virtual machine performance statistics. Such performance metrics are described in U.S. patent application Ser. No. 14/167,316, entitled “CORRELATION FOR USER-SELECTED TIME RANGES OF VALUES FOR PERFORMANCE METRICS OF COMPONENTS IN AN INFORMATION-TECHNOLOGY ENVIRONMENT WITH LOG DATA FROM THAT INFORMATION-TECHNOLOGY ENVIRONMENT”, filed on 29 Jan. 2014, and which is hereby incorporated by reference in its entirety for all purposes.

(280) To facilitate retrieving information of interest from performance data and log files, the virtual machine monitoring application provides pre-specified schemas for extracting relevant values from different types of performance-related events, and also enables a user to define such schemas.

(281) The virtual machine monitoring application additionally provides various visualizations to facilitate detecting and diagnosing the root cause of performance problems. For example, one such visualization is a “proactive monitoring tree” that enables a user to easily view and understand relationships among various factors that affect the performance of a hierarchically structured computing system. This proactive monitoring tree enables a user to easily navigate the hierarchy by selectively expanding nodes representing various entities (e.g., virtual centers or computing clusters) to view performance information for lower-level nodes associated with lower-level entities (e.g., virtual machines or host systems). Example node-expansion operations are illustrated in FIG. **17**C, wherein nodes **1733** and **1734** are selectively expanded. Note that nodes **1731**-**1739** can be displayed using different patterns or colors to represent different performance states, such as a critical state, a warning state, a normal state or an unknown/offline state. The ease of navigation provided by selective expansion in combination with the associated performance-state information enables a user to quickly diagnose the root cause of a performance problem. The proactive monitoring tree is described in further detail in U.S. Pat. No. 9,185,007, entitled “PROACTIVE

MONITORING TREE WITH SEVERITY STATE SORTING", issued on 10 Nov. 2015, and U.S. Pat. No. 9,426,045, also entitled "PROACTIVE MONITORING TREE WITH SEVERITY STATE SORTING", issued on 23 Aug. 2016, each of which is hereby incorporated by reference in its entirety for all purposes.

(282) The virtual machine monitoring application also provides a user interface that enables a user to select a specific time range and then view heterogeneous data comprising events, log data, and associated performance metrics for the selected time range. For example, the screen illustrated in FIG. **17**D displays a listing of recent "tasks and events" and a listing of recent "log entries" for a selected time range above a performance-metric graph for "average CPU core utilization" for the selected time range. Note that a user is able to operate pull-down menus **1742** to selectively display different performance metric graphs for the selected time range. This enables the user to correlate trends in the performance-metric graph with corresponding event and log data to quickly determine the root cause of a performance problem. This user interface is described in more detail in U.S. patent application Ser. No. 14/167,316, entitled "CORRELATION FOR USER-SELECTED TIME RANGES OF VALUES FOR PERFORMANCE METRICS OF COMPONENTS IN AN INFORMATION-TECHNOLOGY ENVIRONMENT WITH LOG DATA FROM THAT INFORMATION-TECHNOLOGY ENVIRONMENT", filed on 29 Jan. 2014, and which is hereby incorporated by reference in its entirety for all purposes.

2.16. IT Service Monitoring

(283) As previously mentioned, the data intake and query platform provides various schemas, dashboards and visualizations that make it easy for developers to create applications to provide additional capabilities. One such application is an IT monitoring application, such as SPLUNK® IT SERVICE INTELLIGENCE™, which performs monitoring and alerting operations. The IT monitoring application also includes analytics to help an analyst diagnose the root cause of performance problems based on large volumes of data stored by the data intake and query system as correlated to the various services an IT organization provides (a service-centric view). This differs significantly from conventional IT monitoring systems that lack the infrastructure to effectively store and analyze large volumes of service-related events. Traditional service monitoring systems typically use fixed schemas to extract data from pre-defined fields at data ingestion time, wherein the extracted data is typically stored in a relational database. This data extraction process and associated reduction in data content that occurs at data ingestion time inevitably hampers future investigations, when all of the original data may be needed to determine the root cause of or contributing factors to a service issue.

(284) In contrast, an IT monitoring application system stores large volumes of minimally-processed service-related data at ingestion time for later retrieval and analysis at search time, to perform regular monitoring, or to investigate a service issue. To facilitate this data retrieval process, the IT monitoring application enables a user to define an IT operations infrastructure from the perspective of the services it provides. In this service-centric approach, a service such as corporate e-mail may be defined in terms of the entities employed to provide the service, such as host machines and network devices. Each entity is defined to include information for identifying all of the events that pertains to the entity, whether produced by the entity itself or by another machine, and considering the many various ways the entity may be identified in machine data (such as by a URL, an IP address, or machine name). The service and entity definitions can organize events around a service so that all of the events pertaining to that service can be easily identified. This capability provides a foundation for the implementation of Key Performance Indicators.

(285) One or more Key Performance Indicators (KPI's) are defined for a service within the IT monitoring application. Each KPI measures an aspect of service performance at a point in time or over a period of time (aspect KPI's). Each KPI is defined by a search query that derives a KPI value from the machine data of events associated with the entities that provide the service. Information in the entity definitions may be used to identify the appropriate events at the time a KPI is defined or

whenever a KPI value is being determined. The KPI values derived over time may be stored to build a valuable repository of current and historical performance information for the service, and the repository, itself, may be subject to search query processing. Aggregate KPIs may be defined to provide a measure of service performance calculated from a set of service aspect KPI values; this aggregate may even be taken across defined timeframes and/or across multiple services. A particular service may have an aggregate KPI derived from substantially all of the aspect KPI's of the service to indicate an overall health score for the service.

(286) The IT monitoring application facilitates the production of meaningful aggregate KPI's through a system of KPI thresholds and state values. Different KPI definitions may produce values in different ranges, and so the same value may mean something very different from one KPI definition to another. To address this, the IT monitoring application implements a translation of individual KPI values to a common domain of “state” values. For example, a KPI range of values may be 1-100, or 50-275, while values in the state domain may be ‘critical,’ ‘warning,’ ‘normal,’ and ‘informational’. Thresholds associated with a particular KPI definition determine ranges of values for that KPI that correspond to the various state values. In one case, KPI values 95-100 may be set to correspond to ‘critical’ in the state domain. KPI values from disparate KPI's can be processed uniformly once they are translated into the common state values using the thresholds. For example, “normal 80% of the time” can be applied across various KPI's. To provide meaningful aggregate KPI's, a weighting value can be assigned to each KPI so that its influence on the calculated aggregate KPI value is increased or decreased relative to the other KPI's.

(287) One service in an IT environment often impacts, or is impacted by, another service. The IT monitoring application can reflect these dependencies. For example, a dependency relationship between a corporate e-mail service and a centralized authentication service can be reflected by recording an association between their respective service definitions. The recorded associations establish a service dependency topology that informs the data or selection options presented in a GUI, for example. (The service dependency topology is like a “map” showing how services are connected based on their dependencies.) The service topology may itself be depicted in a GUI and may be interactive to allow navigation among related services.

(288) Entity definitions in the IT monitoring application can include informational fields that can serve as metadata, implied data fields, or attributed data fields for the events identified by other aspects of the entity definition. Entity definitions in the IT monitoring application can also be created and updated by an import of tabular data (as represented in a CSV, another delimited file, or a search query result set). The import may be GUI-mediated or processed using import parameters from a GUI-based import definition process. Entity definitions in the IT monitoring application can also be associated with a service by means of a service definition rule. Processing the rule results in the matching entity definitions being associated with the service definition. The rule can be processed at creation time, and thereafter on a scheduled or on-demand basis. This allows dynamic, rule-based updates to the service definition.

(289) During operation, the IT monitoring application can recognize notable events that may indicate a service performance problem or other situation of interest. These notable events can be recognized by a “correlation search” specifying trigger criteria for a notable event: every time KPI values satisfy the criteria, the application indicates a notable event. A severity level for the notable event may also be specified. Furthermore, when trigger criteria are satisfied, the correlation search may additionally or alternatively cause a service ticket to be created in an IT service management (ITSM) system, such as a systems available from ServiceNow, Inc., of Santa Clara, California.

(290) SPLUNK® IT SERVICE INTELLIGENCE™ provides various visualizations built on its service-centric organization of events and the KPI values generated and collected. Visualizations can be particularly useful for monitoring or investigating service performance. The IT monitoring application provides a service monitoring interface suitable as the home page for ongoing IT service monitoring. The interface is appropriate for settings such as desktop use or for a wall-

mounted display in a network operations center (NOC). The interface may prominently display a services health section with tiles for the aggregate KPI's indicating overall health for defined services and a general KPI section with tiles for KPI's related to individual service aspects. These tiles may display KPI information in a variety of ways, such as by being colored and ordered according to factors like the KPI state value. They also can be interactive and navigate to visualizations of more detailed KPI information.

(291) The IT monitoring application provides a service-monitoring dashboard visualization based on a user-defined template. The template can include user-selectable widgets of varying types and styles to display KPI information. The content and the appearance of widgets can respond dynamically to changing KPI information. The KPI widgets can appear in conjunction with a background image, user drawing objects, or other visual elements, that depict the IT operations environment, for example. The KPI widgets or other GUI elements can be interactive so as to provide navigation to visualizations of more detailed KPI information.

(292) The IT monitoring application provides a visualization showing detailed time-series information for multiple KPI's in parallel graph lanes. The length of each lane can correspond to a uniform time range, while the width of each lane may be automatically adjusted to fit the displayed KPI data. Data within each lane may be displayed in a user selectable style, such as a line, area, or bar chart. During operation a user may select a position in the time range of the graph lanes to activate lane inspection at that point in time. Lane inspection may display an indicator for the selected time across the graph lanes and display the KPI value associated with that point in time for each of the graph lanes. The visualization may also provide navigation to an interface for defining a correlation search, using information from the visualization to pre-populate the definition.

(293) The IT monitoring application provides a visualization for incident review showing detailed information for notable events. The incident review visualization may also show summary information for the notable events over a time frame, such as an indication of the number of notable events at each of a number of severity levels. The severity level display may be presented as a rainbow chart with the warmest color associated with the highest severity classification. The incident review visualization may also show summary information for the notable events over a time frame, such as the number of notable events occurring within segments of the time frame. The incident review visualization may display a list of notable events within the time frame ordered by any number of factors, such as time or severity. The selection of a particular notable event from the list may display detailed information about that notable event, including an identification of the correlation search that generated the notable event.

(294) The IT monitoring application provides pre-specified schemas for extracting relevant values from the different types of service-related events. It also enables a user to define such schemas.

3.0. Container Log Ingestion

(295) In hosted computing environments, a data source **202** may correspond to one or more host computing systems (or data stored on the host computing systems) that host a number of isolated execution environments, such as containers, pods, virtual machines, etc. The data sources **202** can include data generated by an application or isolated execution environment (also referred to herein as pre-transformed data or pre-transformed log data) that has been transformed by the host computing system or by an isolated execution environment manager (also referred to herein as transformed data or chunks of data). The transformed data may be significantly larger than the pre-transformed data and one or more characteristics of the pre-transformed data or system that generated the pre-transformed data may be lost or obfuscated during the transformation process. The increased size of the transformed data can increase the processing time and resources used by the data intake and query system **108**, and the obfuscation or loss of information about the pre-transformed data can decrease the processing efficiency of the data intake and query system **108** and/or result in certain characteristics about the pre-transformed data or system that generated the pre-transformed data being lost.

(296) As a non-limiting example, in a containerized environment (non-limiting example: Kubernetes), a particular container or pod may generate log data or raw machine data, which is then transformed by a container manager or container orchestration service (non-limiting example: transformed by Docker into a JSON object) and stored in a log file maintained by the container manager. The transformed data in the log file maintained by the container manager may be significantly larger than the pre-transformed log data generated by the container, and certain information about the container that generated the pre-transformed log data may be lost or obfuscated by the container manager during the transformation process. For example, the container manager may not include information about the specific container that generated the pre-transformed data in the log file managed by the container manager.
(297) To address this issue, a data adapter can aid in processing data from data sources **202** that include data to be processed by the data intake and query system **108** that is encapsulated within or associated with a wrapper or a larger chunk of data. In some embodiments, the data adapter can aid in identifying and extracting pre-transformed data from chunks of data in log files managed by the container manager and/or obtaining characteristics about the system or isolated execution environment that generated the pre-transformed data. In this way, the data adapter can facilitate the processing and storing of data from the data sources **202** by the data intake and query system **108**.
(298) FIG. **18** is a block diagram of an embodiment of the data processing environment **200** described previously with reference to FIG. **2** that includes a host computing system **1801** as a data source **202**. In the illustrated embodiment, the host computing system **1801** includes a data adapter **1802**, an isolated execution environment manager **1804**, and one or more isolated execution environments **1806**.
(299) Although illustrated in the data source **202**, it will be understood that the data adapter **1802** can be located in any one or any combination of the data sources **202**. Furthermore, it will be understood that the data adapter **1802**, or its functionality, can be included as part of one or more components of the data intake and query system **108**. For example, functions described herein as being performed by the data adapter **1802** can be performed by one or more forwarders **204** and/or one or more indexers **206**. Furthermore, although located in a data source **202**, in certain embodiments, the data adapter **1802** can form part of the data intake and query system **108**.
(300) In some embodiments, the data adapter **1802** can be implemented as a module, plug-in, or controller at the data source **202** that interfaces with one or more forwarders **204**. In certain embodiments, the data adapter **1802** can be implemented as part of a forwarder **204** located at the data source **202**. For example, if the data source **202** is a container, a forwarder **204**, in certain embodiments, can be implemented using a sidecar container associated with the container of the data source **202**.
(301) As mentioned, the data source **202** can correspond to a host computing system **1801** that hosts isolated execution environments **1806** or data or data files, directories of files, event logs, registries, etc., from the host computing system **1801**. The isolated execution environments **1806** can include, but are not limited, to containerization or operating-system-level virtualizations, or other virtualization techniques. For example, the host computing system **1801** can include one or more separate software containers or container instances. Each container instance can have certain resources (e.g., memory, processor, etc.) of the underlying host computing system **1801** assigned to it, but may share the same operating system and may use the operating system's system call interface. Each container may provide an isolated execution environment **1806** on the host system **1801**, such as by providing a memory space of the host system **1801** that is logically isolated from memory space of other containers. Further, each container may run the same or different computer applications concurrently or separately, and may interact with each other.
(302) Furthermore, the isolated execution environment manager **1804** can interface with the isolated execution environments **1806** to handle system calls and manage resource allocation of the host system's **1801** resources.

(303) Although reference is made herein to containerization and container instances, it will be understood that other virtualization techniques can be used. For example, virtual machines using full virtualization or paravirtualization, etc., can be instantiated on the host system **1801**. Thus, where reference is made to “containerized” components, it should be understood that such components may additionally or alternatively be implemented in other isolated execution environments **1806**, such as a virtual machine environment.
(304) Non-limiting examples of commercially available isolated execution environments **1806** include, but are not limited to, Kubernetes, Docker, Elastic Container Service (ECS), FreeBSD jail, Solaris Containers, Linux namespaces, Windows Server Containers, VMware, Microsoft Virtual Server, Hyper-V, etc. For example, the data sources **202** can include one or more containers, pods, or clusters managed by a container run-time layer, such as Docker, that interfaces or is managed by Kubernetes or Docker Swarm, etc.
(305) The isolated execution environments **1806** can be instantiated on one or more host computing systems **1801** that are local to a user or on a remote distributed computing system. In some embodiments, a remote distributed computing system or cloud-based service can refer to a service hosted by one more computing resources that are accessible to end users over a network, for example, by using a web browser or other application on a client device to interface with the remote computing resources. Typically, a user may pay a subscription or other fee to use such a service. Each subscribing user of the cloud-based service may be provided with an account that enables the user to configure customized cloud-based computing systems based on the user's preferences.
(306) In some cases, individual isolated execution environments **1806** can generate logs of data related to the applications or functions being executed thereon. For example, if an isolated execution environment **1806** is configured as an API server, it can generate logs of data as it processes requests and performs other functions. The logs of data can correspond to one or more log entries. Moreover, each log entry can include raw machine data.
(307) The logs of data generated by individual isolated execution environments **1806** can be communicated to the isolated execution environment manager **1804** that manages or interfaces with the various isolated execution environments **1806** instantiated on one or more host computing systems **1801**. For example, if the isolated execution environment **1806** is a pod or container, the isolated execution environment manager **1804** can be a Docker instance.
(308) The isolated execution environment manager **1804** can store the log data from the isolated execution environment **1806** in a log file of a data store **1902**, described in greater detail herein with reference to FIG. **19**. In some embodiments, the isolated execution environment manager **1804** can transform the log data received from the isolated execution environment **1806**, and store the transformed log data in the log file. In some cases, the transformed log data (also referred to herein as a chunk of data) can include at least a portion of the log data received from the isolated execution environment **1806**, as well as wrapper information, log data generated by the isolated execution environment manager **1804** or other data. Accordingly, the transformed data or chunks of data in the log file associated with the isolated execution environment manager **1804** may not match, accurately reflect, or directly correspond to the actual log data generated by the isolated execution environment **1806** or the application executing within the isolated execution environment **1806**.
(309) In some cases, a log entry generated by an isolated execution environment **1806** can be transformed into one or more chunks of data. For example, an error message generated by an application executing in the isolated execution environment **1806** can be transformed into one chunk of data by the isolated execution environment manager **1804** and stored in the log file. As another non-limiting example, a multi-line log entry generated by an application executing in the isolated execution environment **1806** can be transformed into multiple chunks of data by the isolated execution environment manager **1804** and stored in the log file. In some cases, the isolated

execution environment **1806** generates a chunk of data for each line of log data received from the isolated execution environment **1806**. A new line of data can be determined based on a new line command (e.g., hard return) in the log data, carriage return, or end of a line, etc.
(310) As part of the transformation process, the isolated execution environment manager **1804** can add information to the log data, break apart the log data, change the format of the log data, etc. In some embodiments, the isolated execution environment manager **1804** converts the received log data into one or more JSON objects, and stores the JSON objects in the log file associated with the isolated execution environment **1806** that generated the log data. In certain embodiments, the isolated execution environment manager **1804** stores the log data received from the isolated execution environment **1806** as a field value in the chunk of data. For example, the chunk of data can include a "log" field and store the log data received from the isolated execution environment **1806** as a field value for the "log" field.
(311) In certain embodiments, the isolated execution environment manager **1804** generates multiple log files and/or directories for the isolated execution environment **1806** instantiated on one or more computing devices. For example, the isolated execution environment manager **1804** can generate a separate folder or file for each container, pod, or cluster instantiated on one or more computing devices. Furthermore, each directory or file name can include information or characteristics about the isolated execution environment **1806** associated therewith. For example, the directory or file name can include information related to a container name or ID, namespace name or ID, pod name or ID, etc.
(312) The transformed log data or chunks of data can significantly increase the amount of data in a log file and/or make it more difficult for a data intake and query system **108** to parse and store the log data generated by an isolated execution environment **1806**. Furthermore, the wrapper information in the chunk of data can make it more difficult for the data intake and query system **108** to identify a host, source, or sourcetype of the log data. For example, where the log data may have originated at an API server, the wrapper information may indicate that the chunk of data originated from the isolated execution environment manager **1804**. Moreover, as the isolated execution environment manager **1804** generates chunks of data from multiple isolated execution environments **1806**, the identity of the individual isolated execution environments **1806** from which the log data originated can be lost. Thus, rather than identifying the log data from different isolated execution environments **1806** as originating from or being associated with different isolated execution environment **1806**, the data intake and query system **108** may identify the log data from different isolated execution environments **1806** as originating from a single device, system, isolated execution environment **1806**, or isolated execution environment manager **1804**.
(313) With continued reference to FIG. **18**, the data adapter **1802** can parse the chunks of data in the log files of the data store **1902**, extract data from the chunks of data, and extract relevant information about the isolated execution environment **1806** associated with the chunks of data. The data intake and query system **108** can use the extracted data and/or extracted information to generate one or more events.
(314) In certain embodiments, the data adapter **1802** can extract the data from the chunks of data or determine the relevant information about the isolated execution environment **1806** associated with the chunks of data using one or more rules, regex rules, transformers, etc. For example, different isolated execution environment managers **1804** can generate and store chunks of data differently or in various formats or structures. Accordingly, the data adapter **1802** can identify and use different regex rules to parse chunks of data based on the format or structure of the chunks of data, based on an identifier or characteristic of the isolated execution environment manager **1804** that generated or is associated with the chunk of data.
(315) In certain embodiments, the data adapter **1802** extracts only pre-transformed data (some or all of the pre-transformed data) from the chunk of data. In some cases, the data adapter **1802** extracts some or all of the pre-transformed data and other data from the chunk of data. For

example, the data adapter **1802** can extract a timestamp in the chunk of data that is generated by the isolated execution environment manager **1804**, but may not be part of the pre-transformed data.
(316) As another non-limiting example, in certain embodiments, the data adapter **1802** can extract characteristics of the pre-transformed data or the isolated execution environment **1806** associated with the pre-transformed data from the chunk of data, such as, but not limited to, an identification of the isolated execution environment **1806** that generated the pre-transformed data. In some cases, a chunk of data may include one or more names or identifiers of the isolated execution environment **1806** which can be extracted, such as, but not limited to, a pod name or identifier, a namespace name or identifier, a container name or identifier, a virtual machine name or identifier, etc.
(317) In some cases, the data adapter **1802** can determine characteristics of the pre-transformed data or the isolated execution environment **1806** associated with the pre-transformed data from sources other than the chunks of data. As a non-limiting example, the data adapter **1802** can extract the characteristics from the file name of the log file storing the chunks of data, the directory path or name of the log files, the location or name of the folder where the log files are located, other location information, etc. Similar to parsing chunks of data and extracting pre-transformed data from the chunks of data, the data adapter **1802** can use one or more regex rules to parse the location information to extract characteristics of an isolated execution environment **1806**. In some cases, the data adapter **1802** can use an identification of the isolated execution environment **1806** to identify a particular regex rule to parse the location information associated with a particular isolated execution environment **1806** or isolated execution environment manager **1804**.
(318) In certain embodiments, the data adapter **1802** can determine characteristics of the pre-transformed data or the isolated execution environment **1806** associated with the pre-transformed data by performing a function call to an API of the isolated execution environment manager **1804**. For example, the data adapter **1802** can request the isolated execution environment manager **1804** to send it one or more characteristics of the data or isolated execution environment **1806**.
(319) The data adapter **1802** can communicate the extracted information to a component of the data intake and query system **108** for further processing. The data intake and query system **108** can generate one or more events and associated metadata using the extracted information. For example, the data intake and query system **108** can generate one or more events using the data extracted from the chunks of data and generate metadata associated with the events using the data extracted from the location information, such as the characteristics of the isolated execution environment **1806**.
(320) As described herein, as part of generating the events, the data intake and query system **108** can parse the data extracted from the chunks of data using one or more regex rules. In some cases, the data intake and query system **108** identifies a regex rule to process the extracted data (non-limiting example: pre-transformed data extracted from the chunk of data) based on the location information and/or the determined characteristics of the isolated execution environment **1806**. Furthermore, in some cases, the data intake and query system **108** can determine a source or sourcetype of the extracted data based on the location information and/or the characteristic of the isolated execution environment **1806**. For example, in some embodiments, the data intake and query system **108** identifies the directory path or other location information as the source of the extracted data. In certain embodiments, the data intake and query system **108** identifies a portion of the directory path or location information as the sourcetype of the extracted data.
(321) Moreover, it will be understood that the format, structure, source and/or sourcetype of the chunk of data can be different from the format, structure, source and/or sourcetype of the data extracted from the chunk of data. For example, where the source and/or sourcetype of the extracted data may be associated with, derived from, or depend on the isolated execution environment **1806** or application executing in the isolated execution environment **1806**, the source and/or sourcetype of the chunk of data may be associated with, derived from, or depend on the isolated execution environment manager **1804**. Similarly, isolated execution environment **1806** and isolated execution environment manager **1804** may use different formats or structures to generate pre-transformed

data or chunks of data, respectively. Furthermore, given that the source and/or source type of the chunk of data may be different from the source and/or sourcetype of the extracted data, it will be understood that in some embodiments, the data adapter **1802** uses one or more sets of regex rules to parse or extract data from the chunk of data and the location information and the data intake and query system **108** uses a different set of regex rules to parse, extract data from, or generate events based on the extracted data.

(322) FIG. **19** is a block diagram of an embodiment of a data store **1902** that stores log files associated with a data source **202**. In the illustrated embodiment, the data store includes a directory **1904** that stores multiple log files **1906**A . . . **1906**N (generically referred to as log file(s) **1906**) associated with different isolated execution environments **1806** of one or more host computing systems **1801**.

(323) Although illustrated as a single directory **1904**, it will be understood that the data store **1902** can include multiple directories **1904**, each storing one or more log files **1906**. In some embodiments, the data store **1902** can include a directory **1904** for each isolated execution environment **1806** or for a group of isolated execution environments **1806** (non-limiting examples: one directory per pod of containers or cluster of pods/containers, etc.). In certain embodiments, the data store **1902** can include a directory for each isolated execution environment **1806** instantiated on a particular host computing system **1801** or group of host computing systems **1801**. In some cases, the data store **1902** can include one directory for all (or a subset of) isolated execution environment **1806** instantiated on a particular host computing system **1801** or group of host computing systems **1801**. Similarly, the data store **1902** can include a log file **1906** for each isolated execution environment **1806** or for a group of isolated execution environments **1806**.

(324) In some embodiments, each log file **1906** in the directory **1904** can include log data and/or include location information for a different file that may be in a different location that includes the log data. For simplicity, reference herein will be made to the log files **1906** storing log data or chunks of data. However, it will be understood that the actual log data or chunks of data may be stored in a different file than the log files **1906**. For example, the files that store the log data or chunks of data may be accessed using a file path, location information, or link in the log files **1906**.

(325) In some cases, the path to the directory **1904** or files **1906** can provide information about one or more characteristics of the isolated execution environment managers **1804**, isolated execution environments **1806** associated with the files **1906**, the chunks of data, or the pre-transformed data. For example, the directory path or name or file name can include one or more identifiers or other information about the isolated execution environment **1806** that generated at least a portion of the data in the log files **1906**. This information can include, but is not limited to, a name or identifier of the isolated execution environment **1806**, an IP address of the isolated execution environment **1806**, time range information for the data in the log files, partition information, etc. As a non-limiting example, in a containerized environment, the information can include, but is not limited to, a container name or identifier, a pod name or identifier, a namespace name or identifier, etc.

(326) In the illustrated embodiment of FIG. **19**, the path **1908** includes the directory path "/var/log/containers/" and file name "kube-apiserver-ip-172-20-43-173.ec2.internal--kube-system--kube-apiserver--f7360a148a670c4c257f4ee024be81284b6017d72ae4lea 8ee 5d.log" of the file **1906**N. However, the path **1908** can include fewer or more information as desired. For example, in some embodiments, such as where the data store **1902** includes a directory **1904** for each isolated execution environment **1806**, the path **1908** may only include the directory path, etc.

(327) In the illustrated embodiment FIG. **19**, the name of the file **1906**N includes information about the isolated execution environment **1806** that generated at least a portion of the data in the file **1906**N, such as, but not limited to, the pre-transformed data included in the chunks of data of the file **1906**N. Specifically, as shown in box **1910**, the name of the file **1906**N includes a pod name, name space name, container name and container identifier for the isolated execution environment **1806** that generated at least a portion of the data in the file **1906**N. However, as described herein, it

will be understood that the path **1908** or name of the files **1906**N can include more, less, or different information than what is shown in box **1910**.
(328) As described herein, the data adapter **1802** can use one or more rules, regex rules, configuration files, etc., to parse the path **1908** and identify one or more characteristics of the isolated execution environment **1806** that generated at least a portion of the data in the file **1906**N. The data adapter **1802** can communicate the determined information to a forwarder **204** or indexer **206** for processing along with data from the log file **1906**N. In some cases, by parsing the path **1908** to identify the characteristics, the data adapter **1802** can avoid requesting the information from the isolated execution environment manager **1804** via an API call. As such, the data adapter **1802** can reduce processing delay related to identifying characteristics of the isolated execution environment **1806** that generated pre-transformed data and processing the pre-transformed data.
(329) FIG. **19** further includes an expanded view of some of the content of the log file **1906**N. In the illustrated embodiment, multiple chunks of data **1912**A, **1912**B, **1914**A, **1914**B, **1914**C are illustrated. As described herein, the chunks of data **1912**A, **1912**B, **1914**A, **1914**B, **1914**C can include data generated by an isolated execution environment **1806** and transformed by an isolated execution environment manager **1804** or the host computing system **1801**. In the illustrated embodiment of FIG. **19**, the chunks of data **1912**A, **1912**B, **1914**A, **1914**B, **1914**C are shown in a JSON structure, however, it will be understood that the chunks of data can be implemented in any number of structures or formats.
(330) The chunks of data **1912**A, **1912**B correspond to the same log entry or pre-transformed data generated by an isolated execution environment **1806**, but are shown in different formats for illustrative purposes. Specifically, both chunks of data **1912**A, **1912**B include pre-transformed data **1920** (I0503 23:04:12.595203 1 wrap.go:42] GET /apis/admissionregistrations.k8s.io/v1beta1/validatingwebhookconfiguration 200 [[kube-apiserver/v1.9.3 (linux/amd64) kubernetes/d283541] 127.0.0.1:55026) as a field value for a “log” field and respective additional data **1922**, **1924**.
(331) As shown, the additional data **1924** of the chunk of data **1912**B includes more or different information than the additional data **1922** of the chunk of data **1912**A. Specifically, the additional data **1924** of the chunk of data **1912**B includes field-value pairs for a container name, host IP, labels, master URL, namespace ID and name, pod ID and name, stream type, and timestamp in addition to the pre-transformed data **1920**, whereas the additional data **1922** of the chunk of data **1912**A only includes the stream type and a timestamp. In some embodiments, the additional data **1922**, **1924** can refer to any information in the chunks of data **1912**A, **1912**B other than the pre-transformed data **1920** and/or may refer to the field values of the fields other than the “log” field.
(332) In some embodiments, the additional data **1922**, **1924** can be added to the chunks of data **1912**A, **1912**B by the isolated execution environment manager **1804** and/or by the data adapter **1802**. For example, as the isolated execution environment manager **1804** transforms the pre-transformed data it can collect or generate the additional data **1922**, **1924** for inclusion in the chunks of data **1912**A, **1912**B. In some cases, the container name, host IP, labels, master URL, namespace ID and name, and/or pod ID and name can be collected from (and identify a characteristic of) the isolated execution environment **1806** that generated the pre-transformed data as the isolated execution environment manager **1804** receives and processes the pre-transformed data. In certain embodiments, the timestamp of the additional data **1922**, **1924** can be generated by the isolated execution environment manager **1804** to indicate the time when the pre-transformed data was received and/or when the chunk of data **1912**A, **1912**B was created.
(333) In certain embodiments, the data adapter **1802** can add some or all of the additional data **1922**, **1924** to the chunks of data. For example, in some embodiments, as part of processing the chunks of data **1912**A, **1912**B, the data adapter **1802** can determine characteristics of the isolated execution environment **1806** that generated the pre-transformed data **1920** and add one or more of the characteristics to the chunks of data **1912**A, **1912**B. With reference to chunks of data **1912**A

and **1912**B, the data adapter **1802** can parse the chunk of data **1912**A, determine the host IP, labels, master URL, namespace ID and name, and/or pod ID and name associated with the isolated execution environment **1806** that generated the pre-transformed data **1920** and add that information to create chunk of data **1912**B. In certain cases, the data adapter **1802** can generate the chunk of data **1912**B and store it in the file **1906**N, generate the chunk of data **1912**B and send it to a forwarder **204** or indexer **206** for processing without storing it in the file **1906**N, or append the determined characteristics to the chunk of data **1912**A and communicate it to a forwarder **204** or indexer **206**. As described herein, the data adapter **1802** can determine the characteristics based on an API function call, parsing location information of the file **1906**N, other mechanisms, etc.

(334) The chunks of data **1914**A, **1914**B, **1914**C can correspond to the same log entry of an isolated execution environment **1806** that during the transformation process by the isolated execution environment manager **1804** or host computing system **1801** was transformed into three separate chunks of data. As described herein, in some cases, the isolated execution environment manager **1804** transforms a log entry (non-limiting example: multi-line log entry, formatted log entry) into multiple chunks of data. As such, processing the chunks of data **1914**A, **1914**B, **1914**C by the data intake and query system **108** can result in multiple events being generated by the data intake and query system **108** even though the chunks of data originated from the same log entry. To address this, the data adapter **1802** can extract the pre-transformed data from each of the chunks of data **1914**A, **1914**B, **1914**C and combine them so that the data intake and query system **108** generates one event for the combined pre-transformed data.

(335) As mentioned, the data adapter **1802** can use one or more rules, regex rules, configuration files, etc. to determine that the chunks of data **1914**A, **1914**B, **1914**C are part of a single log entry. In certain embodiments, the data adapter can determine that the chunks of data **1914**A, **1914**B, **1914**C are part of a single log entry based on the format or pattern of the pre-transformed data of the chunks of data **1914**A, **1914**B, **1914**C. For example, based on a regex rule, the data adapter **1802** can determine that if a chunk of data includes pre-transformed data that does not have a timestamp (e.g., chunks of data **1914**B, **1914**C) then they are to be appended to a chunk of data that includes a timestamp (e.g., chunk of data **1914**A). In certain embodiments, based on a format of the timestamp of a chunk of data, the data adapter **1802** can determine that multiple chunks of data are to be combined until another timestamp of the same format is identified.

(336) In certain cases, the data adapter **1802** can append the pre-transformed data from multiple chunks of data together and send the combined data to a forwarder **204** or indexer **206** for processing. Furthermore, in certain embodiments, the data adapter **1802** can determine that each appended pre-transformed data corresponds to a different line and can append a line breaker to the data, such that the data can be displayed as a multi-line or formatted event by the data intake and query system **108**. However, as described herein, the forwarder **204** or indexer **206** can parse the chunks of data and append pre-transformed data from multiple chunks of data together for processing as part of a single event.

(337) As described herein, the data adapter **1802** can parse and extract data from the chunks of data **1912**A, **1912**B, **1914**A, **1914**B, **1914**C using one or more rules, regex rules, or configuration files. In some cases, data adapter **1802** can identify the rules, regex rules, or configuration files to use based on one or more characteristics of the isolated execution environment **1806**, an identity of the isolated execution environment **1806** or isolated execution environment manager **1804**, an identification of the source or sourcetype of the chunks of data or the data to be extracted from the chunks of data, etc. For example, as described herein, different isolated execution environments **1806** (or applications running thereon) or isolated execution environment managers **1804** can generate and store data in different formats, schemes, etc. The data adapter **1802** can use a regex rule that corresponds to the particular isolated execution environments **1806** (or applications running thereon) or isolated execution environment managers **1804** to parse and/or extract data from chunks of data associated with the isolated execution environments **1806** (or applications

running thereon) or isolated execution environment managers **1804**.
(338) As a non-limiting example, the data adapter **1802** can use different regex rules to parse or extract data from the chunk of data **1912**A and the chunk of data **1912**B due to the different formats and content of the chunks of data **1912**A, **1912**B. Similarly, the data adapter **1802** can use a regex rule different from the regex rules used for chunks of data **1912**A, **1912**B to parse or extract data from the chunks of data **1914**A, **1914**B, **1914**C.
(339) As the data adapter **1802** parses the log files **1906** and/or path **1908**, it can track its location using one or more markers or files. The data adapter **1802** can determine that new data has been added to a file based on polling or a notification, such as a notification received from an isolated execution environment manager **1804**. The data adapter **1802** can use the markers or files to identify chunks of data that have not been processed. In certain embodiments, the notification received by the data adapter **1802** can include location or identification information to enable the data adapter **1802** to locate the chunks of data that have not been processed. In certain embodiments, as the data adapter **1802** processes the chunks of data, it deletes them from the log file **1906**. In this way, the data adapter **1802** can reduce the amount of data in the files **1906** and the amount of storage used to store the files **1906**.
(340) In addition, as the data adapter **1802** parses the log files **1906** and/or path **1908**, it can communicate information to the data intake and query system **108**. In some cases, such as when the data adapter **1802** is implemented as a plug-in at the data source **202**, it can communicate the information to a forwarder **204** for further processing. In certain embodiments, such as when the data adapter **1802** is implemented as part of a forwarder **204**, it can communicate the information to an indexer **206**.
(341) As mentioned, the data adapter **1802** can communicate some or all of the chunks of data to the data intake and query system **108**. For example, the data adapter **1802** can extract some or all of the pre-transformed data from a chunk of data and communicate the extracted data to the data intake and query system **108**. In some embodiments, the data adapter **1802** can include other portions of the chunk of data, such as a timestamp or other data that can be used by the data intake and query system **108** to process the data. In certain embodiments, a timestamp of a chunk of data that is not part of the pre-transformed data can be used by the data intake and query system **108** to indicate an indexing or processing time of the pre-transformed data by the isolated execution environment manager **1804**, etc. In certain cases, the data adapter **1802** can discard the data that is not communicated to the data intake and query system **108**.
(342) As described herein, by extracting portions of data from the chunks of data and communicating those portions to the data intake and query system **108**, the data adapter **1802** can reduce the amount of data communicated to and processed by the data intake and query system **108**. As such, the data adapter **108** can improve the efficiency and speed of the data intake and query system **108**.
(343) As another non-limiting example, the data adapter **1802** can communicate entire chunks of data to the data intake and query system **108**. The data intake and query system **108** can extract data, such as the pre-transformed data, from the chunks of data, etc. In such embodiments, other components of the data intake and query system **108** can parse the chunks of data. In some cases, the data intake and query system **108** parses the chunks of data based on an identification of the associated isolated execution environment manager **1804** and/or the isolated execution environment **1806**.
(344) Furthermore, in some cases, the data adapter **1802** can communicate characteristic information about the pre-transformed data, chunks of data, isolated execution environment **1806** that generated the pre-transformed data, or the isolated execution environment manager **1804**, to the data intake and query system **108**. The characteristics can be used to process the chunks of data and/or the data extracted from the chunks of data.
(345) The data intake and query system **108** can process and generate events from the chunks of

data or data extracted from the chunks of data. As describe herein, in some cases, the data intake and query system can generate a formatted event or multi-line event based on the data extracted from the chunks of data.
(346) As described herein, the data intake and query system **108** can process the chunks of data or data extracted from the chunks of data using one or more rules, regex rules, configuration files, etc. In some cases, the data intake and query system identifies the regex rules to process the chunks of data or data extracted from the chunks of data based on an identified source or sourcetype of the chunks of data or data extracted from the chunks of data or based on the location information or characteristics of the isolated execution environment **1806** or isolated execution environment manager **1804**. For example, an identifier of the isolated execution environment manager **1804** can be used to identify one or regex rules to parse and extract data from a chunk of data and an identifier of the isolated execution environment **1806** can be used to identify one or more regex rules to parse and process data extracted from a chunk of data. Furthermore, as described herein, the regex rules used to process the data extracted from the chunk of data can be different from the regex rules used to extract data from and/or parse the chunks of data. Additionally, the regex rules used to parse the chunks of data and the regex rules used to process the data extracted from the chunk of data can be used to identify or extract different data. For example, the regex rules used to parse the chunks of data can be used to extract pre-transformed data and/or other data, such as a timestamp or field values for field-value pairs from the chunk of data, and the regex rules used to process the data extracted from the chunk of data can be used to extract a timestamp, field values for field-value pairs, keywords, tokens, etc., from the extracted data.
(347) In certain embodiments, the data intake and query system **108** can use the characteristic information to process the chunks of data or pre-transformed data, or to generate one or more field-value pairs for the received data. For example, if the data intake and query system **108** receives a container ID associated with pre-transformed data, the data intake and query system **108** can generate a container ID field-value pair for the event generated based on the pre-transformed data. In certain embodiments, the data intake and query system can store the generated field-value pairs in one or more inverted indexes as described herein.
(348) FIGS. **20**A and **20**B are interface diagrams of an example user interface **2000** displaying event data associated with events generated by the data intake and query system **108**. In the illustrated embodiment, the user interface **2000** includes display objects similar to those described previously with reference to search screen **800**. However, the displayed event data **2004**, **2006** shown in the events list **2002** corresponds to events generated based on the chunks of data **1912**A and/or **1912**B and the chunks of data **1914**A, **1914**B, **1914**C, described herein with reference to FIG. **19**.
(349) Specifically, with reference to the displayed event data **2004**, the user interface **2000** includes a timestamp **2008** and data **2010**, which may be raw machine data, of an event. As will be noted, the data **2010** for the event corresponds to the pre-transformed data **1920** extracted from the chunk of data **1912**A and/or **1912**B, and the timestamp **2008** corresponds to the timestamp found in the pre-transformed data **1920**.
(350) In addition, based on a selection of a user to expand the displayed event data **2004** that is shown, the user interface **2000** includes various field-value pairs **2012** associated with the event. As described herein, one or more field values for the field-value pairs can correspond to data extracted from the chunks of data **1912**A and/or **1912**B, correspond to data determined from the path **1908** or other location information, or correspond to data obtained from an isolated execution environment manager **1804**.
(351) In some cases, based on parsing the chunks of data by the data adapter **1802**, the data intake and query system **108** can modify or adjust one or more field values of the field-value pairs **2012**. For example, without parsing the chunks of data, the data intake and query system **108** may determine that the sourcetype of the event is a log file of an isolated execution environment

manager **1804**. However, by parsing the chunk of data, the data intake and query system **108** can determine that the sourcetype of the event is a log entry of a particular isolated execution environment **1806** associated with the isolated execution environment manager **1804** or an application executing in the isolated execution environment **1806**.

(352) In certain cases, based on the characteristics of the isolated execution environment **1806**, the data intake and query system **108** generates or automatically generates one or more field-value pairs **2012**. For example, in some cases, the data intake and query system **108** generates field-value pairs based on data in an event or data in the pre-transformed data.

(353) In some cases, using the characteristics of the isolated execution environment **1806** obtained by the data adapter **1802**, the data intake and query system **108** can generate field-value pairs based on information that may not be found in the data **2010** of the event. Further, these field-value pairs can significantly increase a user's ability to filter events to identify relevant events. As a non-limiting example and with reference to the path **1908**, the field values for the pod name, namespace name, container name, and container id fields of the field-value pairs **2012** can be determined from the path **1908** and, in some embodiments, without reference to the data **2010** of the event.

(354) In the illustrated embodiment, the data intake and query system **108** generates field-value pairs for at least the container ID and name, namespace ID and name, pod ID and name, based on the characteristics obtained by the data adapter **1802**. Furthermore, in some embodiments, the data intake and query system **108** generate field-value pairs for the host, source, and sourcetype based on the characteristics obtained by the data adapter **1802**. For example, as shown in FIGS. **19** and **20**A, the source, sourcetype, pod name, namespace name, container name, and container ID can all be derived from the path **1908**. Using the above-identified field value pairs a user can filter or analyze events to identify related events more easily.

(355) With reference to displayed event data **2006**, a user has not selected to expand the event so less information is shown. However, similar to the displayed event data **2004**, the displayed event data **2006** includes a timestamp **2014** of the event and some of the data **2016** of the event. As described herein, based on the processing of the chunks of data **1914**A, **1914**B, **1914**C, the data intake and query system **108** determined that the chunks of data **1914**A, **1914**B, **1914**C relate to a single log entry of an isolated execution environment **1806**. As such, the data intake and query system **108** combined the pre-transformed data from the chunks of data **1914**A, **1914**B, **1914**C to generate a single event. In addition, the data intake and query system **108** determined that the single event is a multi-line or formatted event. As such, the data intake and query system **108** retains the formatting or multi-line nature of the event for display.

(356) FIG. **20**B includes additional displayed event data **2006** showing additional lines of data **2016** of the corresponding event. As described herein, in some cases, each line of data **2016** of the multi-line event can correspond to a chunk of data in a log file. Thus, the event corresponding to the displayed event data **2006** may be associated with upwards of 24 chunks of data in a log file **1906**. As described herein, the data adapter **1802** can combine the data from the different chunks of data and the data intake and query system **108** can treat the combined data as a single event.

(357) In addition to displaying additional data **2016** of the event, FIG. **20**B includes certain field value pairs **2018** of the event, including host, source, and sourcetype field-value pairs. In the illustrated embodiment, the data intake and query system **108** derives the field values for the host, source and sourcetype from the location path **1908**. It will be understood that fewer or more information or field-value pairs can be included as desired and that the field-value pairs can be determined or derived in a variety of ways.

(358) FIG. **21** is a flow diagram illustrative of an embodiment of a routine **2100** implemented by the data intake and query system **108** for parsing chunks of data. Although described as being implemented by the data intake and query system **108**, it will be understood that the elements outlined for routine **2100** can be implemented by one or more computing devices/components that are associated with the data intake and query system **108**, such as, but not limited to, the data

adapter **1802**, the forwarder **204**, the indexer **206**, etc. Thus, the following illustrative embodiment should not be construed as limiting.

(359) At block **2102**, the data intake and query system **108** obtains a chunk of data. As described herein, in some embodiments, the data intake and query system **108** can obtain a chunk of data by opening or parsing a file from a data source, such as a log file **1906** in the data store **1902**. In certain embodiments, the data intake and query system **108** obtains the chunk of data by receiving it from a data source **202**. For example, a forwarder **204** or an indexer **206** can receive the chunks of data from the data source **202**. In certain embodiments, the chunk of data is associated with an isolated execution environment **1806**.

(360) At block **2104**, the data intake and query system **108** parses the chunk of data. As described herein the chunk of data can correspond to an entry in a log file generated or managed by an isolated execution environment manager **1804**. Furthermore, the chunk of data can include pre-transformed data that corresponds to data generated by an isolated execution environment **1806** or an application executing in the isolated execution environment **1806** (non-limiting examples: log entry, raw machine data, etc.). In certain embodiments, the pre-transformed data can be a field value for a field in the chunk of data and/or the chunk of data can be a JSON object. In addition, the chunk of data can include other data, generated by a data adapter **1802** and/or the isolated execution environment manager **1804**. The other data can include, but is not limited to, wrapper information, field names, field-value pairs, timestamps, log data generated by the isolated execution environment manager **1804**, host computing system **1801**, data adapter **1802**, etc.

(361) As described herein, the data intake and query system **108** can identify log data and non-log data in the chunk of data. In some cases, the data intake and query system **108** can identify the log data based on a pattern, particular location, or field name in the chunk of data. For example, the data intake and query system **108** can determine that field values of field-value pairs correspond to log data and field names and other information in the chunk of data is not log data. Thus, in some embodiments, the identified log data in the chunk of data can include log data generated by an isolated execution environment **1806**, or pre-transformed data, and log data generated by an isolated execution environment manager **1804**. In certain embodiments, the data intake and query system **108** can identify pre-transformed data in the chunk of data. For example, the data intake and query system **108** can determine that the field value of one or more particular field-value pairs corresponds to pre-transformed data.

(362) As described herein, the data intake and query system **108** can parse the chunk of data and identify relevant portions thereof using one or more rules, regex rules, configuration files, etc. For example, chunks of data from different isolated execution environments **1806**, isolated execution environment manager **1804**, log files, sources, or sourcetypes can be formatted differently or include different information. The data intake and query system **108** can include rules, regex rules, and configuration files for each of the different formats of data that are to be processed. The rules, regex rules, configuration files can provide the data intake and query system **108** with information regarding where certain data (non-limiting examples: log data, pre-transformed data) is located within the chunk of data from a particular source or sourcetype or from a particular isolated execution environment manager **1804**. For example, the rules or regex rules can indicate that relevant data is located after a particular sequence of characters or as a field value of a particular field, etc. For instance, the rules or regex rules can indicate that pre-transformed data is located between quotation marks and after “log:,” etc. Accordingly, the data intake and query system **108** can parse the chunk of data based on a determined source or sourcetype of the chunk of data.

(363) At block **2106**, the data intake and query system **108** extracts at least a portion of the chunk of data. In some cases, the extracted portion of the chunk of data can correspond to a portion of the identified log data. In certain embodiments, the extracted portion of the chunk of data can correspond to a portion or all of the pre-transformed data. In certain cases, the at least a portion of the chunk of data can include log data generated by the isolated execution environment manager

1804, such as a timestamp or other information that can be used by the data intake and query system **108** to process the data.
(364) At block **2108**, the data intake and query system **108** processes the extracted data. In certain embodiments, such as when the routine **2100** is implemented as a plug-in or controller on the data source **202**, processing the data can include communicating the data to a forwarder **204** for further processing. In some embodiments, such as when the routine **2100** is executed as part of a forwarder **204**, processing the data can include communicating the data to an indexer **206** for further processing. In some embodiments, such as when the routine **2100** is executed as part of an indexer **206**, processing the data can include generating one or more events and/or storing the processed data in a data store of the data intake and query system **108**.
(365) As described herein, in some cases, the data intake and query system **108** uses one or more regex rules or configuration files to process data and generate events or generate field-value pairs from the data. However, the regex rules or configuration files used to process data to generate the events may be different from the regex rules or configuration files used to process the chunks of data. Accordingly, the data intake and query system **108** may use one set of regex rules or configuration files to process a chunk of data and extract relevant data therefrom (e.g., pre-transformed data) and a second set of set of regex rules or configuration files to process the data extracted from the chunk of data.
(366) Fewer, more, or different blocks can be used as part of the routine **2100**. In some cases, one or more blocks can be omitted. For example, in some embodiments, the data intake and query system **108** can combine extracted data from multiple chunks of data and communicate the combined data for further processing.
(367) In certain cases, the data intake and query system **108** can track its location in a log file. Based on an indication that additional chunks of data have been added to the log file, the data intake and query system can use the tracked location to open the log file and read from it. As another non-limiting example, the data intake and query system **108** can discard the portions of the chunk of data that are not used and/or delete chunks of data as they are processed.
(368) In addition, as described herein at least with reference to FIG. **22**, the data intake and query system **108** can obtain characteristic information about the chunks of data, isolated execution environment **1806** that generated the pre-transformation data, isolated execution environment manager **1804**, etc., and use the information to generate metadata and/or field-value pairs for generated events. In certain embodiments, the data intake and query system **108** can determine the characteristic information from the chunk of data or from location information associated with the chunk of data. In some embodiments, a data adapter **1802** can add the characteristic information to a chunk of data and a forwarder **204** or indexer **206** can extract the characteristic information from the chunk of data.
(369) Furthermore, it will be understood that the various blocks described herein with reference to FIG. **21** can be implemented in a variety of orders. For example, the data intake and query system **108** can concurrently parse and extract data from the chunk of data and so forth.
(370) FIG. **22** is a flow diagram illustrative of an embodiment of a routine **2100** implemented by the data intake and query system **108** for identifying and associating metadata with one or more events. Although described as being implemented by the data intake and query system **108**, it will be understood that the elements outlined for routine **2100** can be implemented by one or more computing devices/components that are associated with the data intake and query system **108**, such as, but not limited to, the data adapter **1802** the forwarder **204**, the indexer **206**, etc. Thus, the following illustrative embodiment should not be construed as limiting.
(371) At block **2202**, the data intake and query system **108** obtains data and location information. As described herein, the data intake and query system **108** can obtain the data from a data source **202** by parsing one or more files in the data source **202** or receiving the data from the data source **202**. Further, the data can correspond to chunks of data that include pre-transformed data generated

by an isolated execution environment **1806**. In certain embodiments, the data intake and query system **108** can obtain the location information by identifying a path of a log file that stores chunks of data.

(372) At block **2204**, the data adapter **1802** parses the location information. As described herein, the data intake and query system **108** can use one or more rules, regex rules, configuration files, etc. to parse the location information. For example, data from different sources can include different information in the location information. Accordingly, the rules, regex rules, configuration files can provide the data intake and query system **108** with information regarding where certain data (non-limiting examples: isolated execution environment **1806** identifier information) is located within the location information. For example, the rules or regex rules can indicate that relevant data is located after a particular sequence of characters, etc. For instance, the rules or regex rules can indicate that the name of an associated isolated execution environment **1806** identifier is located after "/containers/:" etc. As described herein, in some cases, by parsing the location information and extracting relevant data or characteristics therefrom, the data intake and query system **108** can avoid querying an isolated execution environment manager **1804**. In this way, the data intake and query system **108** can reduce the amount of overhead and processing resources used to process the data.

(373) At block **2206**, the data intake and query system **108** generates metadata based on the location information. As described herein, the location information can include various characteristics, names and/or identifiers associated with the isolated execution environment **1806** or isolated execution environment manager **1804** associated with the obtained data. The data intake and query system **108** can use the identified information to generate one or more field-value pairs. For example, the data intake and query system **108** can determine that a pod name associated with the isolated execution environment **1806** is "kube-apiserver-ip-172-20-43-173.ec2.internal," and generate a pod name field value or pod name field-value pair with that information. Similarly, the data intake and query system **108** can use other information or characteristics extracted from the location information to generate field values or field-value pairs.

(374) At block **2208**, the data intake and query system **108** generates events. As described herein, the data intake and query system **108** generates events based on the obtained data. The obtained data can be chunks of data or data extracted from the chunks of data. Accordingly, the data intake and query system **108** can generate events based on chunks of data or pre-transformed data extracted from the chunks of data. In certain cases, the data intake and query system **108** combines data from multiple chunks of data to generate one event.

(375) As described herein, in some cases, the data intake and query system **108** uses one or more regex rules or configuration files to process data and generate events. However, the regex rules or configuration files used to process the data to generate the events may be different from the regex rules or configuration files used to parse the location information. Accordingly, the data intake and query system may use one set of regex rules or configuration files to process the location information and extract relevant data therefrom (e.g., one or more isolated execution environment **1806** identifier(s)) and a second set of set of regex rules or configuration files to process data to generate an event. Furthermore, a third set of regex rules or configuration files may be used to parse and extract data from the chunks of data. Accordingly, multiple sets of regex rules or configuration files can be used by the data intake and query system **108** to process chunks of data, process location information associated with the chunks of data, and process the data extracted from the chunks of data.

(376) At block **2210**, the data intake and query system **108** associates the metadata with the events. As described herein, as the events are generated, the data intake and query system **108** can associate the generated metadata with the events. Furthermore, as described herein the metadata can be used to filter, parse, or analyze the events.

(377) Fewer, more, or different blocks can be used as part of the routine **2200**. In some cases, one

or more blocks can be omitted. For example, in some embodiments, routine **2200** can include displaying the events and/or metadata associated with the events. Furthermore, it will be understood that the various blocks described herein with reference to FIG. **22** can be implemented in a variety of orders. For example, the data intake and query system **108** can concurrently generate metadata and events.

4.0. Additional Embodiments

(378) Although reference is made to extracting pre-transformed data generated by an isolated execution environment **1806** from a chunk of data generated by an isolated execution environment manager **1804** and extracting log data generated by a container or pod from a JSON object, it will be understood that the systems and methods described herein can be used in a variety of systems and architectures to extract data that is to be processed from a wrapper or chunk of data that encapsulates the data to be processed.

(379) Moreover, in some embodiments, an isolated execution environment **1806** can store log entries in a log file of a data store, such as a volume. In such embodiments, the data in the log file may be pre-transformed data. For example, the isolated execution environment **1806** can store the data without it being transformed by an isolated execution environment manager **1804**. Accordingly, in some embodiments, the data adapter **1802** can communicate pre-transformed data to a component of the data intake and query system **108** without parsing and extracting the pre-transformed data from a chunk of data. However, as described herein, in certain embodiments, the data adapter **1802** can use location information of the pre-transformed data in the log file to determine one or more characteristics of the isolated execution environment **1806**. As described herein, the characteristics can be used to generate metadata, determine field-value pairs, and/or process the pre-transformed data, etc.

(380) Further, although reference is made herein to forwarders **204** and indexers **206** of the data intake and query system **108**, it will be understood that the routines and functionality described herein can be implemented in a variety of architectures. For example, U.S. application Ser. No. 15/967,570, entitled “MESSAGE-BASED DATA INGESTION TO A DATA INTAKE AND QUERY SYSTEM” and U.S. application Ser. No. 15/665,159, entitled “MULTI-LAYER PARTITION ALLOCATION FOR QUERY EXECUTION,” each of which is incorporated herein by reference in its entirety, describe a variety of architectures of the data intake and query system. It will be understood that the routines and functionality described herein can be implemented in any one or any combination of the architectures described therein. For example, the routines and functionality described herein can be implemented as part of one or more components of the intake system **210** (e.g., forwarder, **302**, streaming data processor **308**, etc.), or indexing system **212** (e.g., indexing nodes **404**, indexer **410**, etc.). Thus reference to certain components of the data intake and query system **108** described herein should not be construed as limiting the routines and functionality to a particular architecture.

5.0. Terminology

(381) Computer programs typically comprise one or more instructions set at various times in various memory devices of a computing device, which, when read and executed by at least one processor, will cause a computing device to execute functions involving the disclosed techniques. In some embodiments, a carrier containing the aforementioned computer program product is provided. The carrier is one of an electronic signal, an optical signal, a radio signal, or a non-transitory computer-readable storage medium.

(382) Any or all of the features and functions described above can be combined with each other, except to the extent it may be otherwise stated above or to the extent that any such embodiments may be incompatible by virtue of their function or structure, as will be apparent to persons of ordinary skill in the art. Unless contrary to physical possibility, it is envisioned that (i) the methods/steps described herein may be performed in any sequence and/or in any combination, and (ii) the components of respective embodiments may be combined in any manner.

(383) Although the subject matter has been described in language specific to structural features and/or acts, it is to be understood that the subject matter defined in the appended claims is not necessarily limited to the specific features or acts described above. Rather, the specific features and acts described above are disclosed as examples of implementing the claims, and other equivalent features and acts are intended to be within the scope of the claims.
(384) Conditional language, such as, among others, “can,” “could,” “might,” or “may,” unless specifically stated otherwise, or otherwise understood within the context as used, is generally intended to convey that certain embodiments include, while other embodiments do not include, certain features, elements and/or steps. Thus, such conditional language is not generally intended to imply that features, elements and/or steps are in any way required for one or more embodiments or that one or more embodiments necessarily include logic for deciding, with or without user input or prompting, whether these features, elements and/or steps are included or are to be performed in any particular embodiment.
(385) Unless the context clearly requires otherwise, throughout the description and the claims, the words “comprise,” “comprising,” and the like are to be construed in an inclusive sense, as opposed to an exclusive or exhaustive sense, i.e., in the sense of “including, but not limited to.” As used herein, the terms “connected,” “coupled,” or any variant thereof means any connection or coupling, either direct or indirect, between two or more elements; the coupling or connection between the elements can be physical, logical, or a combination thereof. Additionally, the words “herein,” “above,” “below,” and words of similar import, when used in this application, refer to this application as a whole and not to any particular portions of this application. Where the context permits, words using the singular or plural number may also include the plural or singular number respectively. The word “or” in reference to a list of two or more items, covers all of the following interpretations of the word: any one of the items in the list, all of the items in the list, and any combination of the items in the list. Likewise the term “and/or” in reference to a list of two or more items, covers all of the following interpretations of the word: any one of the items in the list, all of the items in the list, and any combination of the items in the list.
(386) Conjunctive language such as the phrase “at least one of X, Y and Z,” unless specifically stated otherwise, is otherwise understood with the context as used in general to convey that an item, term, etc. may be either X, Y or Z, or any combination thereof. Thus, such conjunctive language is not generally intended to imply that certain embodiments require at least one of X, at least one of Y and at least one of Z to each be present. Further, use of the phrase “at least one of X, Y or Z” as used in general is to convey that an item, term, etc. may be either X, Y or Z, or any combination thereof.
(387) In some embodiments, certain operations, acts, events, or functions of any of the algorithms described herein can be performed in a different sequence, can be added, merged, or left out altogether (e.g., not all are necessary for the practice of the algorithms). In certain embodiments, operations, acts, functions, or events can be performed concurrently, e.g., through multi-threaded processing, interrupt processing, or multiple processors or processor cores or on other parallel architectures, rather than sequentially.
(388) Systems and modules described herein may comprise software, firmware, hardware, or any combination(s) of software, firmware, or hardware suitable for the purposes described. Software and other modules may reside and execute on servers, workstations, personal computers, computerized tablets, PDAs, and other computing devices suitable for the purposes described herein. Software and other modules may be accessible via local computer memory, via a network, via a browser, or via other means suitable for the purposes described herein. Data structures described herein may comprise computer files, variables, programming arrays, programming structures, or any electronic information storage schemes or methods, or any combinations thereof, suitable for the purposes described herein. User interface elements described herein may comprise elements from graphical user interfaces, interactive voice response, command line interfaces, and

other suitable interfaces.
(389) Further, processing of the various components of the illustrated systems can be distributed across multiple machines, networks, and other computing resources. Two or more components of a system can be combined into fewer components. Various components of the illustrated systems can be implemented in one or more virtual machines or an isolated execution environment **1806**, rather than in dedicated computer hardware systems and/or computing devices. Likewise, the data repositories shown can represent physical and/or logical data storage, including, e.g., storage area networks or other distributed storage systems. Moreover, in some embodiments the connections between the components shown represent possible paths of data flow, rather than actual connections between hardware. While some examples of possible connections are shown, any of the subset of the components shown can communicate with any other subset of components in various implementations.
(390) Embodiments are also described above with reference to flow chart illustrations and/or block diagrams of methods, apparatus (systems) and computer program products. Each block of the flow chart illustrations and/or block diagrams, and combinations of blocks in the flow chart illustrations and/or block diagrams, may be implemented by computer program instructions. Such instructions may be provided to a processor of a general purpose computer, special purpose computer, specially-equipped computer (e.g., comprising a high-performance database server, a graphics subsystem, etc.) or other programmable data processing apparatus to produce a machine, such that the instructions, which execute via the processor(s) of the computer or other programmable data processing apparatus, create means for implementing the acts specified in the flow chart and/or block diagram block or blocks. These computer program instructions may also be stored in a non-transitory computer-readable memory that can direct a computer or other programmable data processing apparatus to operate in a particular manner, such that the instructions stored in the computer-readable memory produce an article of manufacture including instruction means which implement the acts specified in the flow chart and/or block diagram block or blocks. The computer program instructions may also be loaded to a computing device or other programmable data processing apparatus to cause operations to be performed on the computing device or other programmable apparatus to produce a computer implemented process such that the instructions which execute on the computing device or other programmable apparatus provide steps for implementing the acts specified in the flow chart and/or block diagram block or blocks.
(391) Any patents and applications and other references noted above, including any that may be listed in accompanying filing papers, are incorporated herein by reference. Aspects of the invention can be modified, if necessary, to employ the systems, functions, and concepts of the various references described above to provide yet further implementations of the invention. These and other changes can be made to the invention in light of the above Detailed Description. While the above description describes certain examples of the invention, and describes the best mode contemplated, no matter how detailed the above appears in text, the invention can be practiced in many ways. Details of the system may vary considerably in its specific implementation, while still being encompassed by the invention disclosed herein. As noted above, particular terminology used when describing certain features or aspects of the invention should not be taken to imply that the terminology is being redefined herein to be restricted to any specific characteristics, features, or aspects of the invention with which that terminology is associated. In general, the terms used in the following claims should not be construed to limit the invention to the specific examples disclosed in the specification, unless the above Detailed Description section explicitly defines such terms. Accordingly, the actual scope of the invention encompasses not only the disclosed examples, but also all equivalent ways of practicing or implementing the invention under the claims.
(392) To reduce the number of claims, certain aspects of the invention are presented below in certain claim forms, but the applicant contemplates other aspects of the invention in any number of claim forms. For example, while only one aspect of the invention is recited as a means-plus-

function claim under 35 U.S.C. sec. 112(f) (AIA), other aspects may likewise be embodied as a means-plus-function claim, or in other forms, such as being embodied in a computer-readable medium. Any claims intended to be treated under 35 U.S.C. § 112(f) will begin with the words “means for,” but use of the term “for” in any other context is not intended to invoke treatment under 35 U.S.C. § 112(f). Accordingly, the applicant reserves the right to pursue additional claims after filing this application, in either this application or in a continuing application.

Claims

1. A method, comprising: identifying a portion of a log file stored in a data store of a host computing device in a hosted computing environment, wherein the host computing device includes a plurality of software containers instantiated thereon, wherein the plurality of software containers share hardware resources of the host computing device; obtaining a filesystem directory of a location within the data store where the log file is stored; parsing the filesystem directory to determine an identity of a software container that generated the portion of the log file; generating metadata using the determined identity of the software container; and communicating the portion of the log file and the generated metadata to a computing device of a distributed processing system, wherein the computing device generates an event using the portion of the log file and associated the generated metadata with the generated event.
2. The method of claim 1, wherein identifying the portion of the log file comprises identifying a directory path to the log file that includes a chunk of data.
3. The method of claim 1, further comprising identifying one or more regex rules associated with a software container manager of the plurality of software containers and parsing the filesystem directory using the one or more regex rules.
4. The method of claim 1, further comprising identifying one or more regex rules associated with the software container and parsing the filesystem directory of the location using the one or more regex rules.
5. The method of claim 1, wherein the determined identity of the software container includes at least one of a container name, a container ID, a pod name, a pod ID, a namespace name or a namespace identifier.
6. The method of claim 1, further comprising determining a source of the log file based on the filesystem directory of the location.
7. The method of claim 1, further comprising determining a source type of the log file based on the filesystem directory of the location, wherein the source type corresponds to at least one identifier of the software container extracted from the filesystem directory of the location.
8. The method of claim 1, further comprising parsing the log file based on the determined identity of the software container.
9. The method of claim 1, further comprising determining a source of the portion of the log file based on the filesystem directory of the location, wherein the source of the portion of the log file is different from a source of the log file.
10. The method of claim 1, further comprising determining a source type of the portion of the log file based on the filesystem directory of the location, wherein the source type of the portion of the log file is different from a source type of the log file.
11. The method of claim 1, further comprising: identifying a regex rule based on the determined identity of the software container; and processing the portion of the log file based on the regex rule.
12. The method of claim 1, wherein the software container is an isolated execution environment.
13. The method of claim 1, wherein the portion of the log file is log data generated by the software container.
14. The method of claim 1, wherein the portion of the log file is raw machine data generated by the software container.

15. A computing system, comprising: memory; and one or more first processing devices coupled to the memory and configured to: identify a portion of a log file stored in a data store of a host computing device in a hosted computing environment, wherein the host computing device includes a plurality of software containers instantiated thereon, wherein the plurality of software containers share hardware resources of the host computing device; obtain a filesystem directory of a location within the data store where the log file is stored; parse the filesystem directory to determine an identity of a software container that generated the portion of the log file; generate metadata using the determined identity of the software container; and communicate the portion of the log file and the generated metadata to a computing device of a distributed processing system, wherein the computing device generates an event using the portion of the log file and associated the generated metadata with the generated event.

16. The computing system of claim 15, wherein the filesystem directory of the location comprises a directory path to a file that includes the log file.

17. The computing system of claim 15, wherein the one or more first processing devices are further configured to identify one or more regex rules associated with a software container manager of the plurality of software containers and parse the filesystem directory of the location using the one or more regex rules.

18. A non-transitory computer readable media comprising computer-executable instructions that, when executed by a computing system of a data intake and query system, cause the computing system to: identify a portion of a log file stored in a data store of a host computing device in a hosted computing environment, wherein the host computing device includes a plurality of software containers instantiated thereon, wherein the plurality of software containers share hardware resources of the host computing device; obtain a filesystem directory of a location within the data store where the log file is stored; parse the filesystem directory to determine an identity of a software container that generated the portion of the log file; generate metadata using the determined identity of the software container; and communicate the portion of the log file and the generated metadata to a computing device of a distributed processing system, wherein the computing device generates an event using the portion of the log file and associated the generated metadata with the generated event.

19. The non-transitory computer readable media of claim 18, wherein the filesystem directory of the location comprises a directory path to a file that includes the log file.

20. The non-transitory computer readable media of claim 18, wherein the determined identity of the software container includes at least one of a container name, a container ID, a pod name, a pod ID, a namespace name or a namespace identifier.